

ELI — Complete Technical Manual

ELI v2.5.34

November 2026

Contents

ELI — The Complete User Manual **3**
Table of contents . . . 3
1. What ELI is (in one minute) . . . 4
2. How ELI works when you ask it something . . . 5
3. Getting started . . . 5
4. Your first hour with ELI . . . 7
5. Talking to ELI . . . 7
6. A tour of the window . . . 7
7. What ELI can do — by everyday task . . . 8
8. ELI remembers you (memory) . . . 11
9. ELI learns your routines (habits) . . . 12
10. ELI helps before you ask (proactive) . . . 12
11. Voice, wake word, and dictation . . . 13
12. Seeing your screen and images (vision) . . . 13
13. The ELI server & using it from your phone . . . 14
14. Models: picking, switching, and training your own . . . 16
15. How hard ELI thinks (reasoning modes) . . . 18
16. Settings you'll actually touch . . . 18
17. Privacy & staying offline . . . 18
18. Troubleshooting . . . 19
19. Quick phrasebook (cheat sheet) . . . 19
20. Add-ons: plugins & the community marketplace . . . 20
21. Making ELI your own: custom agents . . . 21
22. Glossary . . . 21
23. Appendix A — Under the hood: how ELI thinks (the full pipeline) . . . 22
24. Appendix B — Under the hood: how ELI remembers (the full memory system) . . . 24
25. Appendix C — Under the hood: how ELI stays safe and yours (security) . . . 26

Blueprint — ELI Architecture **27**
0. Design principles . . . 27
1. Repository map . . . 28
2. Entry points and launch . . . 29
3. The request lifecycle . . . 30
4. Routing layer (`eli/execution`) . . . 30
5–6. Agent bus and agents (`eli/cognition/agent_bus.py`) . . . 31

7. The orchestrator (eli/cognition/orchestrator.py) . . . 32
8. Inference layer (eli/cognition) . . . 32
9. Grounding and anti-confabulation (eli/runtime, eli/core/grounding.py) . . . 33
10. Execution layer (eli/execution/executor_enhanced.py) . . . 33
11. Memory subsystem (eli/memory) . . . 34
12. Perception (eli/perception) . . . 34
13. Persona (eli/cognition/persona*) . . . 34
14. Daemons and background loops . . . 34
15. Learning and self-training (eli/learning) . . . 35
16. EliWorld (eli/world, the EliWorld tab, kernel/world_model.py) . . . 35
17. Core infrastructure (eli/core) . . . 35
18. GUI (eli/gui) . . . 35
19. On-disk layout (artifacts/, runtime) . . . 36
20. Known seams and weak points . . . 36
21. Where to make a change . . . 36
Plugins, agent specifications and training surfaces . . . 37

Blueprint — ELI Full Architecture (ASCII) 38
A. Full system — layered block diagram . . . 38
B. The 15 agents . . . 39
C. Module tree (LOC · key files · role) . . . 40
D. On-disk data layout . . . 41
E. The flow in one line . . . 42
Update — 2026-06-09 (scheduled-task dedup; clean shutdown; STT VRAM) . . . 106

Running ELI at Scale 106
1. The core principle — your design is not the bottleneck . . . 106
2. What “a trillion-parameter model” really means . . . 106
3. You don’t need to own the datacenter — rent it by the hour . . . 107
4. The tiers — what to run, on what, for roughly how much . . . 107
5. How ELI scales — the mechanisms (all already built) . . . 107
6. Runbook A — single big GPU (simplest): run a 70B . . . 108
7. Runbook B — multi-GPU: run a 405B / 671B / ~1T MoE . . . 108
8. Privacy and ethos at scale . . . 109
9. The hardware is coming to you . . . 109
10. Cost cheat-sheet and “see it big this weekend” checklist . . . 109

ELI Security Posture 109
1. Shell command gate (runtime/security.py SecurityManager) . . . 110
2. Filesystem & app gates (SecurityManager) . . . 110
3. Prompt-injection guard (kernel/engine.py) . . . 110
4. SQL identifier validation (memory/memory.py) . . . 110
5. Custom-agent trust chain (cognition/agent_trust.py) . . . 110
6. Code generation & self-modification — three paths . . . 111
Honest assessment . . . 111
RUN_CMD terminal is real; mock leak fenced . . . 112
Web app & multi-user security . . . 112
Stable phone token, admin-gated model switch . . . 114
Token moved to the URL fragment; test reds cleared . . . 114
Community plugin marketplace: consent, verification, scanning . . . 114

Blueprint — ELI ASCII Diagrams 119
1. Full pipeline (request lifecycle) . . . 119
2. Memory subsystem eli/memory + cognition/working_memory.py . . . 120
3. Gating stack (in path order — every guard the request passes) . . . 121
4. One-screen overview (how it all connects) . . . 121

ELI — Capabilities & Actions (with activation phrases) **122**
Conversation & persona . . . 122
App & window control . . . 123
Input control . . . 124
Media . . . 124
Vision & screen . . . 125
Gaze (eye-tracking) . . . 125
Voice . . . 126
Files & notes . . . 127
Generation (docs/code) . . . 127
System status & info . . . 128
Grounded introspection . . . 129
Memory & profile . . . 130
Self-maintenance . . . 131
Plugins . . . 132
Tasks, time & planning . . . 133
Proactive . . . 133
Shell & advanced . . . 133
Aliases & internal actions (not directly user-triggered) . . . 134
Coverage . . . 134

ELI — The Complete User Manual

Plain-English Edition · with flowcharts

A friendly, no-jargon guide to everything ELI can do and how to make it do it.

This manual assumes **zero** technical background. You talk to ELI in normal English — by **typing** in the chat box or **speaking** — and it does things on your computer, answers questions, remembers what matters to you, and learns your routines. Everything runs **on your own machine**; nothing is sent to the cloud.

> **The one rule worth knowing:** ELI is **offline by default**. It only touches the internet when *you* ask for something that needs it (a web search, the news, downloading a model). Your conversations, files, and memories never leave your computer.

How to read this manual: - **Just want to use it?** Read §1–§5 and the cheat sheet (§18). That's enough to be productive. - **Want the whole picture?** Read straight through — every feature is here, in plain terms. - **[Everyone] / [Some setup] / [Advanced] tags** at the start of some sections mean *anyone / a little technical / advanced*.

Table of contents

1. What ELI is (in one minute)
2. How ELI works when you ask it something
3. Getting started
4. Your first hour with ELI
5. Talking to ELI
6. A tour of the window
7. What ELI can do — by everyday task
8. ELI remembers you (memory)
9. ELI learns your routines (habits)
10. ELI helps before you ask (proactive)

11. Voice, wake word, and dictation
12. Seeing your screen and images (vision)
13. The ELI server & using it from your phone
14. Models: picking, switching, and training your own
15. How hard ELI thinks (reasoning modes)
16. Settings you'll actually touch
17. Privacy & staying offline
18. Troubleshooting
19. Quick phrasebook (cheat sheet)
20. Add-ons: plugins & the community marketplace
21. Making ELI your own: custom agents
22. Glossary
23. Appendix A — Under the hood: how ELI thinks (the full pipeline)
24. Appendix B — Under the hood: how ELI remembers (the full memory system)
25. Appendix C — Under the hood: how ELI stays safe and yours (security)

---

1. What ELI is (in one minute)

ELI is a **personal AI assistant that lives on your computer**. Think of it as a capable helper that can:

- **Talk with you** like a knowledgeable person — by text or voice.
- **Operate your computer** — open apps, play music, manage windows, type for you.
- **Look at things** — your screen, images, PDFs, spreadsheets — and explain them.
- **Write for you** — documents, notes, code, scripts.
- **Remember** what matters about you, and **learn** your daily routines.
- **Act on its own** when helpful — a morning briefing, a heads-up, a suggestion.

The big difference from cloud assistants: **it's yours**. It runs locally, stays private, has no subscription, and you can even retrain its "voice" yourself (§14).

Here's the whole thing at a glance:

```
mindmap
  root((ELI))
    Talk
      Type or speak
      Wake word
    Do
      Open apps
      Play music
      Manage windows
      Type for you
    See
      Your screen
      Images and PDFs
      Spreadsheets
    Create
      Documents
      Notes
      Code and scripts
    Know you
      Memory of facts
      Living profile
      Learned habits
    Act on its own
      Morning briefing
```

```
  Suggestions
  Overnight tasks
Private
  Runs locally
  Offline by default
  No accounts
```

---

2. How ELI works when you ask it something

[Everyone] *Anyone — this is the mental model that makes everything else click.*

When you say or type something, ELI follows the same honest loop every time:

```
flowchart TD
    A([You ask ELI something<br/>type or speak]) --> B{Is it a command<br/>or a conversation?}
    B -->|A command<br/>'open spotify'| C{Does it need<br/>the internet?}
    B -->|A conversation<br/>'how are you?'| H[ELI replies in its<br/>own voice]
    C -->|No| D[ELI does it<br/>on your computer]
    C -->|Yes, e.g. news/search| E{You allowed<br/>online for this?}
    E -->|Yes| F[ELI fetches it,<br/>tells you it went online]
    E -->|No| G[ELI stays offline<br/>and says so]
    D --> I{Did it actually<br/>work?}
    F --> I
    I -->|Yes| J([ELI confirms what it did])
    I -->|No / unsure| K([ELI tells you honestly —<br/>never pretends it worked])
    H --> J
```

The promise in that diagram: ELI is **built not to fake an action**. When it says it played a song or fixed a file, that claim comes from the code that actually ran it — and when it couldn't, it is designed to say so plainly instead of bluffing. This is enforced in software, not just good manners. It is a strong guard rather than a mathematical guarantee: no filter over a language model can promise perfection, so ELI is built to fail loudly rather than quietly invent.

---

3. Getting started

[Some setup] *A little technical, but only once.*

Installing (one download per platform)

Get the latest from **GitHub Releases**. Every release is built in CI and **launch-tested on Windows, macOS and Linux** before it can publish.

- **Windows:** run `ELI-Setup-<version>.exe` — installs per-user, no admin password. If it sees an NVIDIA card it offers GPU acceleration during setup; if it finds an older ELI it offers a **fresh install** (resets settings & memory; keeps downloaded models). You get three shortcuts: **ELI** (desktop + Start Menu), **ELI Server** (Start Menu only — not on the desktop; that used to open two windows), **Uninstall ELI**. v2.4.23+ refuses a second GUI.
- **macOS (Apple Silicon):** open the `.dmg`, drag **ELI** into Applications. First launch is right-click → **Open** (the app is unsigned). Apple-GPU acceleration is built in — nothing to set up.
- **Linux:** `chmod +x ELI_v2-<version>-x86_64.AppImage` and run it. On first launch it offers **applications-menu entries** (ELI, ELI Server, and a working Uninstall).

```
flowchart TD
    A([Run the installer / AppImage]) --> B[First launch]
    B --> C{GPU detected?}
```

```
C -->|NVIDIA| D[Offer: CUDA engine<br/>downloaded, verified, kept in step with ELI]
C -->|AMD / Intel / Qualcomm| E[Offer: Vulkan engine<br/>downloaded, verified, kept in step with ELI]
C -->|Apple| F[Metal built in]
C -->|None / decline| G[CPU engine - always works]
D --> H{Chat model installed?}
E --> H
F --> H
G --> H
H -->|No| I[Offer: starter model<br/>sized to your hardware]
H -->|Yes| J
I --> J[Quick interview]
J --> K([Ready - say hello])
```

What arrives on first launch

Asset	When	Size
Piper voices (speech out)	already inside the installer	—
Nomic embedder (memory/recall)	fetched automatically	~80 MiB
Whisper (speech in)	fetched on first voice use	~460 MiB
GPU engine (optional)	if you accept the GPU offer; again by itself when an update moves llama.cpp on	depends on backend and OS, shown while it downloads
Chat model	if you accept the model offer	sized to your hardware (e.g. ~4.7 GiB)

Everything ELI knows and downloads lives in **one per-user folder** that **survives upgrades** — like a save-game: `%LOCALAPPDATA%\ELI_v2` (Windows), `~/Library/Application Support/ELI_v2` (macOS), `~/.local/share/ELI_v2` (Linux). Databases start as a **true blank slate** — no personal data ships in any installer.

Starting over, or uninstalling

- **Clean slate:** tick *Fresh install* in the Windows setup, or run `ELI --fresh-start` anywhere (keeps downloaded models unless you add `--purge-models`).
- **Uninstall:** Windows → the **Uninstall ELI** shortcut. Linux → the **Uninstall ELI** menu entry (removes the entries, offers to delete your data, then delete the AppImage file). macOS → drag the app to Trash; your data folder is listed above.
- **GPU later / undo:** `ELI --install-gpu-pack` (add `--vulkan` for Intel Arc) · `ELI --remove-gpu-pack`.
- **From source (developers):** `bash install.sh` from a clone works exactly as before.

First launch — the quick interview

The very first time you open ELI, it doesn't know you yet, so it asks **three short questions** (your name, what you do, how you like to be spoken to). Ten seconds, and you can **skip it** entirely. From there, ELI builds its understanding of you naturally as you talk.

Opening ELI

- **Desktop app (full experience):** the **ELI** shortcut / menu entry (installer builds), the AppImage itself (Linux), or `scripts/eli_launch.sh` from a source clone.
- **From your phone:** the **ELI Server** shortcut (or `ELI --server`) starts the home server with the connect QR — see §13.

4. Your first hour with ELI

[Everyone] A gentle on-ramp. Try these in order — each builds confidence.

1. **Say hello.** Type `how are you?` — get a feel for ELI's voice.
2. **Ask what it can do.** `what can you do?` lists its capabilities.
3. **Open something.** `open the file manager` or `open firefox`.
4. **Play music.** `play lo-fi on YouTube`, then `pause`, then `skip`.
5. **Look at your screen.** `what's on my screen?`
6. **Tell it a fact.** `remember that my dog's name is Rufus`. Later: `what's my dog's name?`
7. **Set a reminder.** `set a timer for 5 minutes`.
8. **Get a briefing.** `what's the news?` (this one goes online — ELI will say so).
9. **Check on ELI itself.** `what are you running on?` shows its model and hardware.
10. **Go hands-free.** Say the wake word **"computer"**, then a request.

By the end you've touched conversation, control, vision, memory, reminders, the web, voice, and introspection — the whole surface.

---

5. Talking to ELI

There are two ways, freely mixed:

- **Type** in the chat box and press Enter.
- **Speak** — click the microphone, or say the **wake word** (default **"computer"**) then your request. Change the wake word to anything (§11).

You don't need special phrasing. Talk normally:

> "open spotify and play some lo-fi" "what's on my screen?" "remember that my sister's name is Anna" "set an alarm for 7am"

Chaining: ask for several things at once — > "close steam and set an alarm for 7am" > "open spotify then play Juicy by Notorious B.I.G."

If ELI isn't sure what you meant, it asks — it won't guess wildly or pretend. And if a request needs the internet, it tells you before going online.

If you're having a hard time: ELI is tuned to notice genuine first-person distress (it's built to ignore ambient audio and dark jokes, not to over-react). When it does, it steps out of task-mode and gently checks in — pointing you toward real human support rather than ploughing on with commands or reciting a canned line. It's a steer, not a script.

---

6. A tour of the window

[Everyone] ELI Pro is organised into **tabs**. You'll mostly live in **Chat**; here's the rest in plain terms:

Tab	What it's for
Chat	Talk to ELI, by voice or text. Home base.
Quick Actions	One-tap buttons for the things you do most.
Habits	Routines ELI has learned; turn on/off, edit times, add your own.
Proactive	Controls for ELI's "act on its own" behaviour (suggestions, summaries, insights) — includes a nested **Memory** sub-tab to browse/add/remove what ELI remembers about you.
Tasks	Scheduled & overnight jobs and their status.

Tab	What it's for
Images	Generate images — procedural (fast) or diffusion (photoreal).
Screen	Watch/read your screen, take screenshots, OCR.
IDE	A built-in code editor.
Coding	Write, examine, and fix code via the coding agent.
Files	Browse and act on your local files.
Self-Improve	What ELI is proposing to improve in *itself* — approval-gated.
Report Builder	Generate longer documents/reports with sources (evidence → outline → draft → revise).
Elis World	ELI's own world-model view.
Labs	Power-user workshop — notebook, Jupyter, calculator, physics, file-chat, **projects/workspaces**, sim-IDE, orchestration, test review, **training** (§14).
Experimental	In-development features not yet promoted to a stable tab.
Settings	Model (incl. the model-switch dropdown), hardware, voice, privacy options (§16), plugins and the **Marketplace** (§20).

You rarely need the tabs directly — almost everything is reachable by **asking** in Chat: "show my habits", "what do you remember about me", "show background jobs", "make an image of …", and ELI opens or reports the right thing.

---

7. What ELI can do — by everyday task

[Everyone] Real things you can say. ELI understands many phrasings — these are examples.

Music & media

- "play lo-fi on YouTube" · "play Juicy by Notorious B.I.G. on Spotify"
- "pause" · "skip" · "next song" · "previous track" · "shuffle" · "repeat this song"
- "what's playing?" · "stop the music" · "skip the ad" (YouTube, when skippable)

Apps & windows

- "open spotify" · "launch firefox" · "open the file manager" · "open github.com"
- "close chrome" · "focus the browser" · "switch to workspace 2"
- "tile windows" · "maximise the window" · "show desktop" · "next window"

Volume, typing, mouse

- "volume up" · "mute" · "set volume to 30"
- "type hello world" · "press enter" · "left click" · "move the mouse up"

Files & notes

- "create a file notes.txt" · "make a folder called projects" · "read notes.txt"
- "list the files in ~/Documents" · "write a note saying buy milk" · "search notes for budget"
- "what's in my clipboard?" · "summarise report.docx" · "convert report.md to pdf"

Writing, documents & code

- “write a report on renewable energy” · “generate a document about X”
- “write a bash script to monitor the GPU” · “solve this: implement a function that …”
- “fix the bugs in foo.py” · “examine eli/memory/memory.py for errors” → ELI scans, **offers** fixes, and applies them only if you say “yes, fix it”. · “show the diff”
- “fabricate a test dataset for X” → ELI generates synthetic/test data on a topic.
- The **Report Builder** tab turns “generate a document on X” into a real report: it gathers evidence, plans an outline, drafts each section, then **reviews and revises** it — with document-type quality profiles. It’s generation-first, not a one-shot dump.

Screen, images, PDFs, spreadsheets

- “what’s on my screen?” · “find the submit button on screen” · “find the word compose” · “click compose” (finds the text and clicks it) · “take a screenshot”
- “describe cat.jpg” · “read the text in image.jpg” (OCR)
- “summarise report.pdf” · “analyse data.csv” · “watch my screen” / “stop watching”

Information

- “what time is it?” · “what’s the date?” · “weather in Paris”
- “what’s the weather in Dublin tomorrow?” → that day’s forecast. With no place ELI asks “Where?” once and remembers it for the session.
- “what’s the news?” / “catch me up” → a **synthesised** briefing, not a raw dump
- “search the web for X” (goes online — ELI tells you) · “gpu status” · “system stats”

[done] Saying yes, picking, answering

Whatever ELI offers or asks, your next message is read against it. This is the same for every topic, not only reminders. - **“yes” does it.** “Want me to walk you through how that works?” → “yes please” and ELI does exactly that. So do “go ahead”, “sounds good”, “do it”. - **Pick from a list.** When ELI lists steps and asks whether to go ahead, “do 2”, “1 and 3”, “the second one” or “the abstract one” runs just those; “yes” runs them all. Steps ELI can act on are carried out; steps it does by writing are written. - **Answer in a word.** After “Which one, A or B?” a bare “B” is the answer. After “Is it formal or casual?” so is “formal”. - **“no thanks” drops it.** - A few things ELI will not do on a “yes” to its own offer (clearing a persona lock, deleting memories, running a shell command). It tells you the words to say instead. - An offer lapses after 30 minutes, or as soon as ELI says something else.

Reminders, timers, calendar, scheduling

- “set an alarm for 7am” · “set a timer for 10 minutes” · “remind me at 4pm to go over my slides” · “remind me in 20 minutes to stretch”
- **Calendar.** ELI keeps its own calendar on your machine: no account, nothing leaves the computer. “add dentist appointment on friday at 2:30pm to my calendar” · “what’s on today?” · “what have I got on this week?”. A timed event gets a reminder 30 minutes before and one at the start: a desktop notification with a sound, and a line in the chat. Reminders survive a restart; one that came due while ELI was closed is shown as missed when ELI next starts (up to 18 hours later).
- **Changing it.** “i meant 8pm” or “move the dentist to next monday” moves the event and its reminders. “cancel the dentist appointment” removes it; “clear my reminders” removes them all. When more than one entry matches, ELI lists them and asks which.
- **Mention it and ELI offers.** Say “I have a presentation at 7.30pm” in passing and ELI asks whether to put it in the calendar; “yes” adds it.
- **When ELI has to ask, answer in a word.** “add dentist appointment to my calendar” gets “When is it?”; “friday at 3pm” finishes the job. “cancel the dentist” with two entries gets a numbered choice; “the second one” or “the check-up” picks.

- ELI knows what is coming up: “how long until the presentation?” is answered from the calendar and the clock, not from the web.
- The calendar is also written to `calendar.ics` in ELI’s data folder, which any calendar app can import. Repeating events are not supported.
- “start a pomodoro”
- “research the best solar inverters overnight” · “build me a script at 2am” → **background jobs**
- “show background jobs” · “check job 5”

Asking ELI about itself (honest introspection)

- “what are you running on?” (model, context, GPU) · “how does your memory work?”
- “what do you know about me?” · “what updates have you made recently?” · “status”

Eye control (gaze)

- “calibrate gaze”, then “enable gaze”. With it on, ELI clicks **where you’re looking** when you say “open” / “left click” / “right click” / “hit enter” — hands-free pointing.
- **Fully hands-free *and* voice-free — dwell-click.** Say **“enable gaze clicking”** (or turn on **Dwell-click** in Settings) and ELI clicks wherever you **rest your gaze for about a second** — no mouse, no voice needed. This is built for people who can’t use their hands *or* speak reliably. Tune the hold time, wander radius, and sensitivity in Settings (`gaze_dwell_seconds`, `gaze_dwell_radius`). Turn it off any time; it’s opt-in.
- “gaze status” · “disable gaze”

Making images

- “make an image of a red bike by the sea” · “generate a picture of …”
- Two engines: a fast **procedural** one (always available) and **diffusion** (SSD-1B) for photoreal results — ELI briefly frees the model’s VRAM to run it. The **Images** tab has the controls; results save locally.

Coding & building things

- “solve this: implement a function that …” → ELI plans it, writes it, then **verifies and repairs** it.
- “generate a project that does X” (a full multi-file scaffold) · “write a bash script to …”
- “examine eli/memory/memory.py for errors” → a tiered scan; it **offers** fixes and applies them only if you say “yes”. · “show the diff” · “generate tests for your code” (it writes *and* sandbox-verifies them). The **Coding** tab and the built-in **IDE** are home for this.

ELI improving itself

- “improve yourself” → a self-improvement patch cycle — proposed, and **gated by your approval**.
- “generate patch” · “improve yourself and propose verified fixes” → the coding agent drafts verified fixes for recent failures as a **background job** (it can take many model calls); “check job N” for the result. Say “in the foreground” to wait for it.
- “show your self-improvement log” · “patch yourself” · “upgrade yourself” (git pull → deps → rebuild indexes) · “run your self-tests”. The **Self-Improve** tab shows what it’s proposing.

Plugins & your own agents

- “list plugins” · “enable the X plugin” · “install the X plugin” · “plugin status”
- You can **build your own agent** just by describing it — ELI runs a short dialog, validates it, and registers it live (the create-agent flow).

Overnight & scheduled work

- “research the best solar inverters overnight” · “build me a script at 2am” · “every night, regenerate the test report” → durable **background jobs** that survive restarts.
- “show background jobs” · “check job 5”. The **Tasks** tab lists them.

Shaping ELI’s voice (persona)

ELI’s personality is **emergent** — it forms from your profile, your history, and the way you talk to it, not from a hand-written script. Most of the time you leave it alone and it just *becomes* itself with you. When you want to steer it, you can, without erasing that: - “lock your persona to a terse senior engineer” · “what persona are you locked to?” · “clear persona lock”. A lock **layers on top** of the emergent voice for the session; clearing it hands the wheel back to the natural persona. - “refresh your persona” reloads it from your latest profile (after ELI’s learned something new about you). - **Set how it talks to you** *(new, 2.1.23)* — say *“speak to me more bluntly”*, *“be funnier”*, or *“respond as Rick”* and it **sticks** (say *“reset your tone”* to clear it). You can also set it in **Settings → Identity → Tone / how ELI speaks**. This explicit preference takes priority over the tone ELI infers, but leaving it blank keeps ELI’s natural emergent voice in charge. - ELI also keeps a small set of **core values** (memory policy, verbosity, safety) that are part of its persona and evolve alongside it.

The rule of thumb: **the emergent voice is the default and the truth of who ELI is with you; a lock is a temporary hat it wears on request.** ELI won’t drift into a scripted character on its own, and a lock never overwrites the underlying personality — it sits on top until you clear it.

Tone & emotion — reading the room

ELI now **reads the emotional situation from two independent signals** — the *sound* of your voice (energy, mood) and the *content* of what you say — and shades **how** it delivers: its wording, its voice (pace, warmth, pauses), and its avatar expression all move together. If you’re clearly frustrated it gets calmer and more direct; if you’re playing, it plays back; if you’re upset, it softens. **This never changes who ELI is** — its core personality stays exactly the same; it just picks the tone that’s most useful for the moment.

You can also set the register yourself: *“be comedic”*, *“talk street”*, *“sound more professional”*, *“use a deadpan tone”* — and *“go back to normal”* (or *“be yourself”*) to hand it back to ELI’s own read of the room. There’s a growing palette of tones (comedic, professional, curious, deadpan, tender, street-smart, gremlin, and more), and you can add your own in `config/voices/tones.json`. Turn the whole thing off with `ELI_TONE_ADAPT=0`.

---

8. ELI remembers you (memory)

[Everyone] ELI builds a private, evolving understanding of **who you are** and reads it every time you talk, so you never repeat yourself.

```
flowchart TD
    A([You're talking to ELI]) --> B{What kind of thing<br/>did you say?}
    B -->|'Remember that ...'| C[Saved as a durable fact<br/>kept until you remove it]
    B -->|Just chatting<br/>interests, focus, tone| D[Quietly updates ELI's<br/>living profile of you]
    C --> E[(Local memory<br/>on your computer)]
    D --> E
    E --> F([Read every turn so ELI<br/>stays in context])
    G[/Old, abandoned topics<br/>fade over time/] --> E
```

Tell it something to keep: “remember that my sister’s name is Anna” · “my name is Alex” **Ask what it knows:** “what do you remember about my car?” · “what do you know about me?” · “explain everything you know about me and where it’s stored” (a sourced, detailed answer).

It remembers the thread, not just facts: when a conversation ends, ELI quietly summarises it and carries a digest of your recent sessions forward — so it recalls what you've been *working through* over days, not just isolated one-off facts.

Everything is stored **locally** in small private databases. Review and prune it in the **Memory** sub-tab (under **Proactive**). A brand-new install is a **blank slate** — ELI knows nothing about you until you talk.

---

9. ELI learns your routines (habits)

[Everyone] If you do the same thing at roughly the same time on **several different days**, ELI notices and **offers** to automate it. It never activates anything without your "yes".

```
flowchart TD
    A[You open your email app<br/>around 9am, several days] --> B{Same action, same time,<br/>on 3+ differe
    B -->|No| X[ELI stays quiet<br/>no nonsense suggestions]
    B -->|Yes| C[ELI offers:<br/>'Add this as a habit? yes/no']
    C --> D{Your answer}
    D -->|No| E[Dropped, won't ask again]
    D -->|Yes| F{Has today's time<br/>already passed?}
    F -->|Yes| G[Runs it now once,<br/>then daily from tomorrow]
    F -->|No| H[Runs at the time, daily]
    G --> I([Active habit — edit in Habits tab])
    H --> I
```

Manage everything in the **Habits** tab (add, edit times, turn off), or say "show my habits". The key safeguards: it only proposes routines you've **genuinely repeated across days**, and it **always asks first**.

---

10. ELI helps before you ask (proactive)

[Everyone] With **proactive mode** on, ELI can do helpful things on its own — a **morning briefing**, a gentle suggestion, or a goal it thinks would help. It's conservative and easy to control:

- "start proactive mode" / "stop proactive mode" · "proactive status"
- "morning report" / "give me my briefing" (news, weather, your agenda)
- "do you have any proposals for me?" / "what's on your agenda?"

Schedule real unattended work too: > "research X overnight" · "generate tests for your code tonight"

These appear in the **Tasks** tab and survive restarts.

The autonomy tick — what "on its own" actually means

When you allow it, ELI runs a quiet **autonomy tick** inside the proactive daemon — roughly every half hour, and only if it's switched on (kill switch: `ELI_AUTONOMY_TICK=0`). One tick does three things, all **read-only or proposal-only**:

1. **Code monitor** — it glances over its own code health and flags anything that's drifted.
2. **Self-model refresh** — it updates its understanding of itself (what it's running, what it can do) so its introspection stays honest.
3. **Goal autogenesis** — it may propose a goal, or a scheduled task, it reckons would help you.

The guardrail that matters: every autogenerated goal is **proposal-only**. It shows up as a suggestion you accept or dismiss — nothing destructive ever runs unattended, and anything risky still goes through the **approval gate** (Appendix C). Autonomy makes ELI *thoughtful* on its own, not *loose*.

---

11. Voice, wake word, and dictation

[Everyone] ELI is fully usable hands-free.

```
flowchart LR
    A([Say the wake word<br/>'computer']) --> B[ELI starts listening]
    B --> C[You speak your request]
    C --> D[ELI transcribes it<br/>on your computer]
    D --> E[ELI does the task]
    E --> F([Optionally speaks the answer back])
```

- **Wake word:** say **"computer"** then your request. Change it: "change the wake word to athena". ELI trains its listener on the new word automatically — works even over background music.
- **Personalise to your voice:** "enroll my wake word" (records you saying it and adapts).
- **Dictation:** "start dictation" / "stop dictating" — speak and ELI types into the active field.
- **Transcribe a recording:** "transcribe audio.wav"
- **ELI speaking back:** "say good morning"
- **A human-like voice:** at the **startup model picker** (before ELI loads its model) pick **Voice engine → "Natural neural (XTTS-v2)"** for an expressive, human-sounding voice — or say *"use the natural voice"* any time. It needs a one-time `pip install "eli-v2.0[natural]"` (downloads ~1.8 GB on first use) and a reasonable GPU; on a lighter machine leave it on **Piper**, which is fast and now noticeably more natural (it pauses on full stops and lifts on questions).
- **Change ELI's voice:** the voice dropdown next to the chat box, **Settings ▸ Runtime ▸ "VOICE / TTS"**, or just ask — *"use the alan voice"*, *"switch your voice to amy"*, *"use a natural male voice"*.
- **Get more voices and accents:** ask ELI directly — *"what voices do you have"*, *"download a British voice"*, *"get me a Scottish accent"* — or use **Settings ▸ Runtime ▸ "VOICE / TTS" > "Get more voices / accents…"**. Either way, ELI can download from a library of **166 voices across 45 languages** — 38 of them English (US, British, Scottish, Northern and Southern English). Each is verified, installed, and selectable immediately. A few voices are marked *personal use only*: their licences don't allow ELI to bundle them, but you can still download them yourself.
- **Character voices:** choose `char:calm`, `char:robotic`, `char:energetic`, `char:synthetic`, or `char:refined` from the same dropdown — each is a real voice plus a live effect chain. You can make your own, and downloading a character's "ideal base" voice (flagged in the voice library) sharpens it.
- **Make your own voice: Settings ▸ Runtime ▸ "VOICE / TTS"** — either **drop in a clip** (a short, clean ~6–20s .wav/.mp3/.mp4 of the voice you want) or click **"Record a voice now"** to capture 12 seconds from your mic on the spot. You can also say *"create a voice called Nova from clip.mp4"*. It sounds like your sample once the neural voice extra is installed (`pip install "eli-v2.0[natural]"`); until then it's saved and ELI uses a normal voice.
- **ELI's face reacts:** the little animated face shows how ELI feels, **moves its mouth while it speaks**, and shows a focused look while it's thinking.
- **Teach ELI your voice & tone:** "train my voice" — it learns your pitch/energy and even emotion cues, then adapts how it speaks to you.
- **If voice misbehaves:** "run voice diagnostics".

---

12. Seeing your screen and images (vision)

[Everyone] ELI can actually **look**:

- **Your screen:** "what's on my screen?" · "what does this error say?" · "find the submit button"
- **Image files:** "describe cat.jpg" · "read the text in this image" (OCR)
- **Documents:** "summarise report.pdf" · "analyse data.csv" · "analyse the pdfs in that folder"
- **Continuous watching:** "watch my screen" (and "stop watching")

The first time you use vision, ELI may fetch a small vision model (you can pre-download it — §14). With a webcam set up, ELI even supports **eye-tracking control**: "enable gaze control", "calibrate gaze", then

click where your eyes rest.

---

13. The ELI server & using it from your phone

[Some setup] *Slightly technical, but only at setup.*

ELI includes a built-in **server** and **web app**. Start the server on your computer and chat with ELI from your **phone or tablet's browser** over your home Wi-Fi — while all the actual thinking stays on your computer.

```
flowchart TD
    A([Start the ELI Server<br/>on your computer]) --> B{Who should reach it?}
    B -->|Just this computer<br/>default| C[Private to localhost]
    B -->|My phone/tablet<br/>on home Wi-Fi| D[Start in LAN mode]
    D --> E[It prints an address<br/>+ a security token]
    E --> F[Open that address in<br/>your phone's browser]
    F --> G[Enter the token once]
    G --> H([Chat with ELI from your phone<br/>thinking stays on the PC])
    C --> H
```

- Start it from the desktop app — **Settings → Web Server** (a "this computer" button and a "phone / Wi-Fi" button that shows the exact link to open). Or run `scripts/eli_serve.sh` (Mac/Linux) / `scripts/eli_serve.ps1` (Windows). Full details: `docs/SERVER_AND_WEB_APP.md`.
- The web app is **installable** — your phone's "Add to Home Screen" makes it open like an app, and the shell works offline. Light/dark themes; you can recolour the chat.

Starting the server, and what it prints

Start the server and it does two friendly things so you never have to hunt for the address: it **prints a clickable link** (Ctrl/Cmd-click it right in the terminal), and it **auto-opens your local browser** to the dashboard. (Set `ELI_API_NO_BROWSER=1` if you'd rather it didn't.)

- **This computer only (default):** the dashboard opens at `http://127.0.0.1:8081/`. No token — you're on the machine, so you're trusted automatically.
- **Phone / Wi-Fi (LAN mode):** it prints your computer's LAN address (e.g. `http://192.168.1.20:8081/`) with a **one-time token** baked into the link, plus a **QR code** in the Connect tab. Scan it or open the link and you're in.

The token, and the rotate button

The phone's token is **stable** — it's saved (permissions `0600`, under your config dir) and survives server restarts, so a paired phone stays paired; no re-scanning every reboot. Lost the phone, or just want a clean slate? Hit **rotate**: it mints a fresh token and instantly cuts off anything still using the old one.

How the link keeps your token safe

The token rides in the link's **fragment** — the part after the `#` (`…:8081/#token=…`). Browsers **never send the fragment to the server**, so your token can't land in the server's access log (or any proxy log in between). When the page loads it reads the token from the fragment, stores it locally, then **wipes it from the address bar** so it isn't left in your history either. Older links that used `?token=` still work — the page accepts both — but everything ELI prints or displays now uses the safe fragment form, on the desktop panel and both launcher scripts alike.

Voice from the phone needs HTTPS

Browsers block the microphone on a plain `http://` LAN address, so the phone **mic** needs a secure page. Start with `--https` (or the HTTPS toggle) and ELI runs a *second* server alongside on port **8443** with a self-signed certificate — accept the one-time "not private" warning and the mic works. Plain HTTP stays the primary path (it's what QR scanners open reliably); HTTPS runs *in addition*, purely for voice.

Switching the model from the dashboard

Settings → Model lists every model installed on your computer, with sizes. Pick one and ELI **hot-swaps** it live — no restart. Only real, installed files show up, so you can't point it at something that won't load, and only an **admin** can switch it.

Two servers, one machine

Worth knowing: "the server" is really **two** local services. The **web/API server** (port 8081) serves the dashboard, the phone app, and a small API. The **device server** is a separate MQTT service that talks to your smart-home gear (see the Home tab, below). Both run locally; both are yours. Everything about who's allowed in — tokens, roles, the loopback trust — is laid out in **Appendix C (Security)**.

Under the hood (for scripts & power users)

The web/API server is a **FastAPI** app served by **uvicorn** (`api/server.py`). You rarely touch it directly — the desktop app and the launcher scripts wire everything up — but if you script it, a handful of environment variables control it: - `ELI_API_HOST` — `127.0.0.1` (default, this-computer-only) or `0.0.0.0` (LAN mode). - `ELI_API_PORT` — the HTTP port (default `8081`); the HTTPS voice server uses `8443`. - `ELI_API_TOKEN` — set your own token; if unset, ELI uses the saved stable one (`config/api_token`, permissions `0600`) so a paired phone survives restarts. - `ELI_API_ALLOW_TOKENLESS` — honoured only on loopback; LAN mode always requires the token. - `ELI_API_NO_BROWSER=1` — start without auto-opening the local browser.

The launchers wrap all of this: `scripts/eli_serve.sh` (Mac/Linux) and `scripts/eli_serve.ps1` (Windows) start LAN mode, then print the phone link (with the safe `#token=` fragment) and the **exact** firewall command for your OS — the quickest way in from a terminal. The API itself is a small REST surface (chat, status/telemetry, model list/switch, voice, memory, research, audit); full endpoint detail lives in `docs/SERVER_AND_WEB_APP.md`.

If a phone can't connect

Nine times out of ten it's the firewall. When it starts in LAN mode ELI prints the **exact** command to open the port for your OS — run that, and make sure the phone is on the **same Wi-Fi**. It's all scoped to your local network; nothing is exposed to the internet.

What's in the web app

It has a sidebar with several sections: - **Overview** — a live dashboard (clock, GPU/CPU/RAM gauges, status, recent activity). - **Chat** — streamed replies with markdown/code, saved sessions, stop/regenerate, and voice in/out. - **Commands** — a searchable list of everything ELI can do. - **Home** — ELI's own **home-AI** (see below). - **System** — live, measured GPU/CPU/RAM/model telemetry. - **Research** — shared document corpora you can build and query **together**, with citations. - **Audit** — a tamper-evident log of every action. - **Admin** — (admin only) per-user activity, the risk-gate policy, and user management.

Roles (optional)

By default the person on the computer is the operator (full admin). If others share it, you can create accounts with roles — **viewer** (read-only dashboards), **member** (use everything), **admin** (also the Admin console) — in the Admin tab. Each gets a one-time link; the audit trail then attributes actions to the right person.

The Home tab — ELI as your home assistant

ELI controls smart devices **directly over MQTT** (ESPHome / Tasmota / Zigbee2MQTT) — no Home Assistant, no cloud: - **Find devices** on your network with one click (mDNS), or add one by its topics. - **Control** lights/switches, group them into **rooms**, and run room-wide on/off — from the app or by voice ("turn on the desk lamp", "turn off the kitchen"). - **Scenes** — save a set of device states as "Movie mode" and activate it (by voice too). - **Automations** — *when* something happens (a time, **sunrise/sunset**, or a device turning on/off), *do* something (control a device or run a scene), optionally *only if* a condition holds. - ELI **learns** how you use your home and **suggests** automations you can accept with one tap.

---

14. Models: picking, switching, and training your own

[Some setup] The **model** is the "brain" ELI thinks with — a file on your computer. Bigger = smarter but needs more graphics memory (VRAM). ELI picks a suitable one at install; change it anytime.

Which model should I get?

```
flowchart TD
    A([How much graphics memory<br/>VRAM do you have?]) --> B{Amount?}
    B -->|None / CPU only| C[qwen2.5-3b<br/>small and fast]
    B -->|~8 GB| D[qwen2.5-7b default<br/>or qwen3-8b]
    B -->|~12 GB| E[falcon3-10b<br/>or phi-4]
    B -->|24 GB+| F[qwen3.6-35b-a3b<br/>or falcon-h1-34b]
    C --> G([ELI auto-sizes it to your<br/>hardware — you tune nothing])
    D --> G
    E --> G
    F --> G
```

Downloading

- **Pick from a menu (choose any number of models):** `python -m eli.core.model_download --choose` — this is what the installer uses; you tick the ones you want (or `all` / `auto` / `none`).
- Let ELI choose one best-fit for your hardware: `python -m eli.core.model_download --auto`
- See the menu without downloading: `python -m eli.core.model_download --list`
- Grab a specific one: `python -m eli.core.model_download qwen2.5-7b`

You're not limited to one — download several and switch between them anytime (§"Switching models").

Command name	Model	Roughly needs
`qwen2.5-3b`	small & fast	4 GB graphics / or CPU
`qwen2.5-7b` *(default)*	great all-rounder	8 GB graphics
`qwen3-8b`	newer, big context, good reasoning	8 GB graphics
`falcon3-10b`	a step up	12 GB graphics
`phi-4`	strong reasoning for its size	12 GB graphics
`qwen3.6-35b-a3b`	very capable (mixture-of-experts)	24 GB / or slow on CPU
`falcon-h1-34b`	largest option	24 GB / or slow on CPU

You can also just **drop any `.gguf` file** into the `models/` folder and ELI finds it.

Switching models

In **Settings → Model** (or "Model menu / Auto Detect"), pick the file. ELI sizes it to your hardware automatically — you don't tune anything.

Using Ollama instead (Windows, macOS, Linux)

[Some setup] If you already run **Ollama**, ELI can use your Ollama models instead of a GGUF file — handy when you've curated models there, or want ELI to share one Ollama install.

1. Install and start Ollama (`https://ollama.com/download`). On Windows and macOS it starts itself; on Linux use `systemctl --user start ollama` (or `ollama serve`).
2. Pull at least one model, e.g. `ollama pull llama3.2`.
3. In ELI: **Settings → Model → Backend = Ollama**, then **Refresh Ollama Models** and pick one. It's also in the startup model picker and the toolbar dropdown.

ELI uses Ollama only when you choose it this way. With a GGUF model chosen it never starts the Ollama server and never falls back to it, even if Ollama happens to be running.

Host and port. Leave *Host* blank/default for a normal local install. Set it if Ollama runs elsewhere — a different port (`localhost:9999`), another machine (`192.168.1.20`), or a container. You can type it any way that makes sense (`localhost:11434`, `127.0.0.1:11434`, a bare IP) — ELI fills in the missing pieces. **Ollama on another machine is supported**: ELI treats a host you configured as a deliberate local service, so its offline-by-default guard won't block it and the Net toggle can stay off.

If ELI can't see your models: confirm Ollama is running (`ollama list` in a terminal). If it's on another machine, start it there with `OLLAMA_HOST=0.0.0.0:11434` so it accepts connections beyond that machine, and allow port `11434` through its firewall.

[Advanced] Training your own ELI-flavoured model (advanced, optional)

You can fine-tune a model so it speaks in **ELI's voice** out of the box.

See `blueprints/finetuning_guide.md` — a full step-by-step from choosing a base model, through extracting a voice dataset and training, to producing a ready-to-load `.gguf`. Crucially, ELI's **personality stays live** (never frozen into the model); training only shapes the *manner* of speech.

Training ELI on your own conversations

[Some setup] ELI can fine-tune a model on **the way you actually talk to it** — all on your computer, nothing uploaded. **Labs ▸ Training**, four steps.

1. Can this machine do it? ELI checks your graphics card, how much memory is free, and whether the training software is installed — then tells you plainly. If your card is too small it says how much it needed and what would fix it, rather than starting something that runs for days.

Works with NVIDIA and AMD cards. Apple and Intel graphics are detected but cannot run training; ELI says so instead of pretending.

2. What are you training? You need a **Hugging Face model folder**, which is a different thing from the `.gguf` file ELI chats with. ELI lists every one it can find on your computer. Pick one, give it a name, done.

3. Which conversations should it learn from? This is the important step. ELI gathers your past exchanges and sorts them for you:

- exchanges that were clearly broken are **removed automatically**;
- exchanges that look odd are **flagged for you to read**;
- the rest are shown as clean.

You approve the clean batch in one click and then read the flagged ones. **Nothing trains on anything you have not approved.** Typically that means reading a few hundred rather than every single exchange.

You can also edit an exchange — fix ELI's answer to what it *should* have said — and train on the corrected version.

4. Train. Pick Light (minutes), Standard, or Deep (hours). "Check readiness" runs every check without training. "Start training" runs it, showing progress and loss, and you can cancel.

> **After training finishes**, the result is a small "adapter" file, not a new brain. To actually chat with it, the adapter has to be merged into the base model and converted to `.gguf`. ELI tells you this when the run ends — it will not pretend the model has changed when it hasn't. See the fine-tuning guide for that step.

---

15. How hard ELI thinks (reasoning modes)

[Everyone] ELI has five effort levels. Faster modes answer quickly; deeper modes think longer and use more of ELI's internal helpers for hard problems.

Mode	Feel	Use it for
quick	snappy	greetings, simple commands, quick facts
normal	balanced (default)	everyday questions and tasks
advanced	more thorough	tricky, multi-step asks
research	digs deep	gathering and synthesising a lot
expert	maximum effort	the hardest problems

Ask "what reasoning mode are you in?" or "explain all your reasoning modes". For most people the default is right — ELI also deepens on its own when a problem clearly needs it.

---

16. Settings you'll actually touch

[Some setup] Most people never need these. The few that matter:

- **Model** — which brain ELI uses (§14).
- **Voice** — wake word, text-to-speech on/off, microphone.
- **Hardware / startup** — ELI auto-tunes; advanced users can nudge how much graphics memory to hold back, or how much context to use. Leave alone unless you know why.
- **Privacy** — ELI is offline by default; this is where anything network-related lives.

Two graphics cards? ELI can use both — set in the GPU profile settings; single-card just works.

---

17. Privacy & staying offline

[Everyone]

- **Nothing leaves your computer** unless you ask for something online (web search, news, weather, downloading a model). ELI tells you when it goes online.
- **Your data is yours** — conversations, memories, files, profile live in local databases. Delete anytime.
- **No accounts, no telemetry, no subscription.**
- A fresh install is a **blank slate** — ELI starts knowing nothing about you.

---

18. Troubleshooting

[Some setup] Most issues map to one of these. When in doubt, **ask ELI** — “run a full runtime audit” or “status” — it reports honestly.

“ELI is replying very slowly”

```
flowchart TD
    A([Replies feel slow]) --> B[Ask: 'what are you running on?']
    B --> C{Running on GPU<br/>or CPU?}
    C -->|CPU| D[Reinstall with graphics support,<br/>or pick a smaller model]
    C -->|GPU but big model| E[Switch to a smaller model<br/>see section 14]
    C -->|GPU, right-sized| F[That's normal speed for<br/>your hardware]
    D --> G([Faster])
    E --> G
```

Everything else

Problem	Try this
Replies look like gibberish	A model whose context is too small. Pick a recommended one (§14); ELI sizes context automatically.
“I couldn’t find a model”	`python -m eli.core.model_download --auto`
Memory/recall seems empty	Normal on a fresh install. Ensure the embedder downloaded: `python -m eli.core.model_download --aux`
Voice/wake word not responding	“run voice diagnostics”; check the microphone in Settings → Voice.
Won’t go online for news/search	By design (offline-first). The network is allowed per-request; check Settings → Privacy.
A habit didn’t run	Open **Habits** — confirm it’s enabled and the time is right; or “show my habits”.
Something seems broken	“run a full runtime audit” or “status”.

---

19. Quick phrasebook (cheat sheet)

Everyday - “open spotify and play lo-fi” · “pause” · “skip” - “what’s on my screen?” · “summarise report.pdf” - “set an alarm for 7am” · “set a timer for 10 minutes” - “what’s the news?” · “weather in Dublin” · “what time is it?”

Memory & you - “remember that …” · “what do you know about me?” · “my name is Alex”

Habits & proactive - “show my habits” · “yes” / “no” (to an offer) · “morning report” · “start proactive mode”

Writing & code - “write a report on X” · “write a script to …” · “fix the bugs in foo.py”

Voice - wake word “computer” + request · “change the wake word to athena” · “start dictation” · “say good morning” · “train my voice”

Check on ELI - “what are you running on?” · “status” · “what updates have you made recently?”

Models (in a terminal) - `python -m eli.core.model_download --list` · `python -m eli.core.model_download --auto`

---

20. Add-ons: plugins & the community marketplace

[Some setup] **Settings ▸ Marketplace.** Plugins add abilities to ELI. Some ship with it; others are written by **other people in the community**.

The honest bit, first

The marketplace belongs to the community, not to ELI's author. Anyone can host one and anyone can publish to it. **Nobody vets what goes in**, so ELI will never tell you a plugin is safe — it isn't in a position to know.

What it does instead, before a single file lands on your computer:

- **Checks the file is the file.** The listing publishes a fingerprint; ELI checks the download against it. If it doesn't match, it refuses — that means the file was changed somewhere between the publisher and you.
- **Checks who wrote it**, if they signed it and you have chosen to trust their key. Unsigned isn't blocked — it's clearly labelled *unverified*.
- **Reads the code.** If it uses an ability it never declared, ELI refuses.
- **Scans it for malware** — nine checks always run, looking for things like remote access backdoors, password stealing, hidden start-up entries, disguised code, and crypto miners; if you have ClamAV or YARA installed, those run too, for eleven in total.
- **Shows you everything it found**, then asks.

If a scanner couldn't run, ELI says the check was **partial**. It won't call something clean that it didn't fully check.

Permissions — like your phone

A newly installed plugin arrives **switched off with no permissions at all**.

When you enable it and it first wants to do something — reach the internet, read your files, use your microphone — ELI asks. The message names the plugin, says what it can do to you, and how risky that is. Four answers:

Answer	What happens
Allow once	Allowed now. You'll be asked again next time.
Always allow	Allowed from now on, no more asking.
Not now	Refused this time. You'll be asked again.
Never allow	Refused permanently. It can never ask again.

Every answer is recorded, and you can take any of them back in **Marketplace ▸ Permissions**.

If a plugin asks while nobody's there — an overnight scheduled job, or the phone server — the answer is automatically **no**. A plugin can't get permission by waiting until you're asleep.

Paid plugins

Some publishers charge. You buy from **them**, not from ELI, and paste the licence key in. ELI has no payment system, takes no cut, and **cannot confirm a purchase happened or that a refund is possible**. Treat it exactly like buying software from a stranger's website — because that's what it is.

MCP servers — one thing to know

MCP servers are add-ons that run as **their own separate program** alongside ELI. That means ELI's "stay offline" switch **cannot stop them** reaching the internet, and ELI can't see what they send. This isn't a flaw in ELI — it's how MCP works. Only add MCP servers you'd trust with that.

ELI does make sure they actually work: it checks the required software is installed, then genuinely starts the server and talks to it before saving anything. If it can't answer, it isn't added.

---

21. Making ELI your own: custom agents

[Advanced] An **agent** is a small specialist that runs alongside ELI's main reply — one that watches for a certain kind of request and contributes something extra.

You can write your own. A custom agent needs four things, and ELI won't accept it without them:

Objective	One sentence: what is this agent responsible for?
Instruction	What it should actually do — the prompt it runs on.
Triggers	When it should wake up (a keyword, a pattern, or a type of request).
How you'd know it worked	Checks ELI can run on the answer — must mention X, must not say Y, must be at least N characters, must be valid JSON.

That last one is the part people skip, and it's the reason it's required. Without it, an agent either seems fine or doesn't, and you can never tell which. With it, ELI **tests** the agent and an agent that fails its own test contributes nothing rather than quietly adding noise to your answers.

Agents are created switched off. You can edit one and reload without restarting ELI.

> If you write an agent as actual Python code rather than a specification, ELI treats it as untrusted until you approve it — and it scans the code first, recording what it found and when you approved it. Editing the file afterwards cancels the approval automatically.

---

22. Glossary

Term	Plain meaning
Model / GGUF	The "brain" file ELI thinks with. `.gguf` is just its file format.
VRAM	Memory on your graphics card. More VRAM → you can run a bigger, smarter model.
Embedder	A tiny helper model that powers ELI's memory/recall. Installed automatically.
Context	How much of the conversation ELI can "hold in mind" at once. ELI sets this for you.
Vision model	The model that lets ELI understand screens and images.
Wake word	The phrase that makes ELI start listening (default "computer").
Proactive mode	ELI doing helpful things on its own (briefings, suggestions).
Habit	A routine ELI learned and you approved, run on a schedule.
Fine-tuning	Optional advanced step to teach a model ELI's voice (§14).

Term	Plain meaning
Offline-by-default	ELI never uses the internet unless you ask for something that needs it.

---

23. Appendix A — Under the hood: how ELI thinks (the full pipeline)

[Advanced] *For the curious / technical. You never need this to use ELI — but here is the complete, verified path from your words to ELI's reply.*

Every request runs through one funnel — `CognitiveEngine.process()` — which takes one of three paths depending on what you asked and the reasoning mode. The diagram is the **real** pipeline (verified against the running code): a deterministic fast-path for plain commands, a **quick** path via the Agent Bus (the default), and a **deep** 12-stage Orchestrator for hard work.

```
flowchart TD
    U([Your input — text or voice]) --> R[Router<br/>regex priority pipeline<br/>+ LLM-intent fallback]
    R --> P[CognitiveEngine.process<br/>action · args · confidence · mode]
    P --> FP{Plain deterministic command?<br/>open app · media · volume · status · job}
    FP -->|Yes · PHASE45 fast-path| FX[execute_action] --> VERB([Reply returned verbatim<br/>no LLM, instant
    FP -->|No| MODE{Reasoning mode}

    MODE -->|Quick · Normal · Advanced · Research · Expert| ORCH[Gradient Orchestrator<br/>depth scales with

    subgraph DEEP[Orchestrator pipeline — S01–S12]
      direction TB
      O1[S05 · Planner — mode-aware budgets] --> O6[S06 · Shared retrieval + specialist bus]
      O6 --> O7[S07 · Context assembly + heuristic rerank]
      O7 --> O8[S08 · LLM generation via broker]
      O8 --> O9[S09 · Private reasoning algo for mode]
      O9 --> O12[S12 · Learning / state commit]
    end
    ORCH --> O1
    O12 --> GOV

    ORCH -.->|fallback if orchestrator fails| BUS[Agent Bus dispatch]
    BUS --> AG[15 specialist agents over a DAG]
    AG --> GOV

    GEN[LLM generation<br/>broker.infer / stream] --> GOV[Output governor<br/>+ No-Fake-Actions guard]
    GOV --> OUT([Reply to you])
    VERB --> OUT
```

The retrieval stages (5–7) are where memory plugs in — that's Appendix B. The two guarantees baked into the end of every path: the **Output governor** cleans the reply, and the **No-Fake-Actions guard** ensures ELI never claims it did something it didn't.

A.1 — The router (before any path is chosen)

Your words first hit the **router**. It runs a **regex-first priority pipeline**: an ordered set of fast, exact checks (web realtime lookups, media controls, file/PDF, memory, OS control, …), each carefully shaped so it only fires on a real command — e.g. media controls need a whole word in command shape, so "metabolise" never triggers a media "tab". If nothing matches, an **LLM intent classifier** is the fallback. The router emits `{action, args, confidence}` and hands off to `CognitiveEngine.process()`. The router also remembers the *last used path* so follow-ups ("do it again", "the second one") resolve correctly.

A.2 — The paths, and when each is taken (v2.3.37+)

- **Fast-path (PHASE45) — no AI involved.** Plain, deterministic actions (open an app, pause music, set volume, report status, check a job) are executed directly and the result is returned **verbatim**. There's no model call, so these are instant and can't be "hallucinated".
- **CHAT path — gradient orchestrator for every mode.** Quick, Normal, Advanced, Research, and Expert **all** run `AgentOrchestrator` at a depth chosen for that mode: Quick uses a **fast planner** and a **lean specialist fan-out**; Expert uses full retrieval budgets and the broadest agent profile. Quick no longer bypasses the orchestrator for a bare bus dispatch.
- **Fallback — Agent Bus.** If orchestration fails, the engine falls back to a direct `AgentBus.dispatch()` — the same 15-agent parallel fan-out documented below.

A.3 — The specialist bus (composed inside orchestration)

The bus runs **`dispatch_specialists()`** after shared retrieval on CHAT turns (memory agent skipped when evidence was already prefetched). It selects a mode-aware subset of agents, runs them concurrently over a dependency DAG, and aggregates into context. Key agents:

Agent	Contributes
Memory	recalls facts/turns from SQLite + full-text + vector search
Knowledge-Graph	entities and how they relate (who/what/where)
System	runs executor actions (web search, audits, status)
File-Code	searches the actual codebase for grounded answers about ELI itself
Introspection	live runtime/self state
Capability	what ELI can do (the capability manifest)
Habit / Proactive / Reflection / Self-Improve	routines, signals, insights, fixes
Plugin / Voice / Frontier / Orchestrator	plugin dispatch, speech state, deeper reasoning hooks, planning

After the bus returns, a **grounding-escalation hook** checks: *is this a factual question that came back weakly supported?* If so, ELI escalates — a web tier (if you allow online), a local tier, or an **honest hedge** ("I'm not certain, but…") rather than a confident guess. Offline, which is the default, ELI answers what does not change (what a river is, who wrote a play) from what its model knows, and says so plainly when a question is about something that changes (news, prices, results) and cannot be checked without the network. Then the result is finalised: a self-contained action returns directly, a grounded control action gets a short grounded synthesis, and a conversation goes to generation with the full persona + memory context.

A.4 — The 12 stages, one by one (canonical S01–S12)

Logged via `pipeline_trace.py`; enable with `ELI_PIPELINE_TRACE=1`.

1. **S01 PERCEIVE_INGEST** — normalise voice/text input.
2. **S02 INPUT_GUARDS** — safety and policy gates.
3. **S03 ROUTER** — intent → action.
4. **S04 GROUNDING_GATE** — pre-generation grounding check.
5. **S05 PLANNER** — mode-aware retrieval budgets (fast / balanced / deep).
6. **S06 AGENT_BUS** — `retrieve_for_turn()` (shared, 8 s turn cache) then `dispatch_specialists()` fan-out.
7. **S07 CONTEXT_ASSEMBLY** — merge channels + **heuristic rerank** (lexical × recency × importance; not a neural cross-encoder today).
8. **S08 INFERENCE_BROKER** — model generation (stream or one-shot).
9. **S09 REASONING_SYNTHESIS** — private mode algorithm (CoT / self-consistency / ToT / …).

10. **S10 OUTPUT_GOVERNOR** — post-generation policy and sanitisation.
11. **S11 RESPONSE_DELIVERY** — user-visible reply.
12. **S12 LEARNING_STATE_UPDATE** — `learning_coordinator.finalize_turn()`: store the assistant turn, publish meta, and run `_learn_from_result()`.

A.5 — The two end-of-path guarantees

Every path, no matter which, ends at: - **Output governor** — normalises and cleans the reply (strips artefacts, enforces formatting). - **No-Fake-Actions guard** — if ELI's reply *claims* it did something (played a song, fixed a file), the system verifies it actually happened; if not, the claim is replaced with the honest truth. This is enforced in code, not left to the model's goodwill.

---

24. Appendix B — Under the hood: how ELI remembers (the full memory system)

[Advanced] *For the curious / technical. ELI's memory is a genuine hybrid — relational + full-text + vector + graph — all local. This is the complete input → storage → recall path (verified against the code).*

```
flowchart TD
    subgraph IN[INPUT — everything ELI may write]
      direction LR
      I1[Conversation turns]
      I2["'Remember X' durable facts"]
      I3[Observations + user patterns]
      I4[Habits + habit events]
      I5[Failures · corrections · improvements]
      I6[Knowledge-graph entities + relations]
      I7[User Model brief]
    end
    IN --> MEM[Memory class<br/>memory.py routes every write]

    MEM --> D1[(user.sqlite3<br/>memories · turns · habits<br/>patterns · user_model · KG)]
    MEM --> D2[(agent.sqlite3<br/>dispatch + metrics)]
    MEM --> D3[(system_index.sqlite3<br/>apps · files · dirs)]
    MEM --> D4[(coding_memory.sqlite3<br/>bug fixes)]
    MEM -->|embeds new text| V[[FAISS vector index<br/>nomic embedder<br/>serialized — segfault-safe]]
    MEM -->|indexes text| F[[FTS5 full-text<br/>memories + turns + KG]]
    MEM -->|entities/relations| KG[[Knowledge graph<br/>subject - predicate - object]]

    Q([Recall request<br/>from pipeline stages 5-7]) --> RC[recall_memory<br/>hybrid retriever]
    V --> RC
    F --> RC
    RC --> S1[1 · FAISS vector search — primary]
    S1 --> S2{Cold index or<br/>fewer than limit/2 hits?}
    S2 -->|Yes| S3[2 · FTS5 / LIKE keyword supplement]
    S2 -->|No| S4
    S3 --> S4[3 · Noise filter<br/>drop ELI's own insights / reflections /<br/>session summaries / rows ove
    S4 --> S5[4 · Importance-weighted ordering<br/>COALESCE importance 0.5]
    S5 --> CTX([Grounded context handed to the prompt])
    KG -.->|context_for_prompt| CTX
    I7 -.->|read fresh every turn| CTX

    subgraph MECH[UPKEEP MECHANISMS]
      direction LR
```

```
  M1[Weight decay<br/>fades by AGE, half-life stretched by importance]
  M2[Embedder lock<br/>one-at-a-time embedding]
  M3[recall_log<br/>records what was recalled]
end
M1 -.-> D1
```

Why it's trustworthy: the **noise filter** stops ELI's own past replies and reflections from resurfacing as if they were *your* facts; the **embedder lock** keeps the vector engine from crashing under concurrent use; and **weight decay** is the gentle "forgetting" that lets stale topics fade while active ones stay fresh — a memory fades with its age, and every time one is recalled its half-life stretches, so what you keep coming back to keeps its place. Nothing is ever deleted by fading, and **consolidation** folds duplicate entries into one rather than letting the same thought accumulate. Nothing here leaves your computer.

B.1 — What ELI writes (the inputs)

Everything that flows into memory passes through one **Memory** module, which decides where each item belongs: - **Conversation turns** — what you said and what ELI replied, with timestamps. - **Durable facts** — anything you explicitly ask it to "remember". - **Observations & user patterns** — quiet signals about your interests, focus, and tone that build the living profile. - **Habits & habit events** — your repeated actions and the routines learned from them. - **Failures, corrections, improvements** — ELI's own mistakes and fixes, kept for self-learning (these are deliberately **excluded** from normal recall — see the noise filter). - **Knowledge-graph entities & relations** — named things and how they connect. - **User Model brief** — a compact, pre-rendered summary of who you are, refreshed over time.

B.2 — Where it's stored (four local databases + three indexes)

Everything is plain local files on your machine — no server, no cloud: - **`user.sqlite3`** — the main store: memories, conversation turns, habits, patterns, the User Model, and the knowledge graph. - **`agent.sqlite3`** — ELI's internal agent activity and performance metrics. - **`system_index.sqlite3`** — an index of your apps, executables, files, and folders (so "open the file manager" is instant). - **`coding_memory.sqlite3`** — remembered code bug-fixes.

On top of the relational data sit three search indexes: - **FAISS vector index** — every stored text is turned into an "embedding" (a list of numbers capturing meaning) by a small local **nomic embedder**, so ELI can find things by *meaning*, not just exact words. Embedding is done **one-at-a-time** behind a lock (the embedder isn't thread-safe; serialising it prevents crashes). - **FTS5 full-text index** — fast exact/keyword search over memories, conversation turns, and the knowledge graph. - **Knowledge graph** — a subject → predicate → object web (e.g. *you → own → a Tesla*) that ELI can walk to answer relational questions.

B.3 — How recall works (step by step)

When the cognition pipeline (stages 5–7) needs memory, it calls **`recall_memory`**, a genuine hybrid retriever that runs in this order: 1. **Vector search first (FAISS).** The semantic index is the primary path — it finds things that *mean* the same even if worded differently. 2. **Keyword search (FTS5) alongside it.** Both run every time and their rankings are combined, so a memory found by meaning and by the exact word comes first. The keyword search reads what was said, never the labels ELI files it under. A plain LIKE scan covers the case where it finds nothing. 3. **Noise filter.** Results are scrubbed of ELI's *own* output — assistant insights, reflections, session summaries, the "orchestrator" source, and any over-long blobs (>1500 chars). This is the safeguard that stops ELI's past musings from masquerading as *your* facts. 4. **Importance-weighted ordering.** What survives is ranked by an importance score so the most significant memories surface first.

When you ask what ELI remembers, every memory it shows has its date in front of it. Asking how its memory is doing ("how's your memory holding up?") gets a short health check instead: how much is stored and whether every store is readable.

Alongside this, the **knowledge graph** contributes a lightweight relational context, and your **User Model brief** is read **fresh every single turn** — that's why ELI stays in context about who you are without you

repeating yourself.

B.4 — Upkeep (the mechanisms that keep memory healthy)

- **Weight decay** — old, low-importance memories are gently faded (multiplied down over time), so abandoned topics naturally recede while active ones stay prominent. This is ELI's "forgetting".
- **Embedder serialization** — the single lock around the embedder that keeps the vector engine stable under concurrent access.
- **Recall log** — a record of what was retrieved, used for tuning and for honestly explaining *where* a piece of knowledge came from ("how do you know that?").

B.5 — Privacy, restated

Every database and index in this appendix is a local file under your user profile. There is no sync, no upload, no account. Delete the files (or use the **Memory** tab) and that knowledge is gone. A fresh install starts with the full structure but **zero** personal rows — a true blank slate.

---

25. Appendix C — Under the hood: how ELI stays safe and yours (security)

[Advanced] *For the curious / technical. ELI is meant to be safe by construction, not by good intentions — every guard below is enforced in code, not left to a setting you might forget to flip. All of this is verified against the running codebase.*

C.1 — Offline by default (the socket failsafe)

ELI doesn't just *prefer* to stay offline — it's physically stopped from reaching the internet unless you asked for something that needs it. At startup, `netguard` installs a **process-wide socket guard**: it patches the low-level `connect()` call so that *any* outbound connection to a non-loopback address raises an `OfflineError` while ELI is offline. Loopback and local sockets (the phone web server, the local model) always work. So even if some stray bit of code *tried* to phone home, the operating-system call itself is blocked — belt and braces. When you do turn networking on for a task, that flip is written to the audit ledger; "internet on" is never silent.

Ask for something online while offline and you don't get a crash or an invented answer — you get an honest refusal: *"I can't fetch that right now, I'm offline."*

C.2 — The fail-closed command gate

ELI can run shell commands and open apps, so that path is gated hard. Before any command runs it's checked against an **allowlist** by the SecurityManager. The part that matters: if the security layer can't load for any reason, the check **fails closed** — it returns *denied*, not *allowed*. A broken guard blocks the action; it never quietly waves one through. Sandboxed runs are limited to a fixed set of allowed executables.

C.3 — The approval gate (nothing risky happens behind your back)

Anything ELI proposes on its own — an autonomous action, a self-improvement patch, a scheduled job — goes through the **approval engine** before it can execute. Each proposal carries an *action class*, and the engine decides: auto-approve the harmless ones, or hold the riskier ones until **you** confirm. Who is even allowed to propose what is also gated — a background daemon can't quietly emit a dangerous action. There's a **Full Control** mode that lifts the confirmation barrier if you want ELI to just act, but that's your explicit choice, and it's off by default.

C.4 — The phone: who's allowed in

The web server (§13) is locked down. On the same machine (loopback) you're trusted automatically. From the network, every request must carry a **bearer token** — and that token is never stored in plain text, only its SHA-256 hash. The token travels in the link's URL **fragment** (after the #), which browsers never transmit to the server, so it can't appear in the access log; the page reads it locally and immediately clears it from the address bar. Define one or more users and **RBAC** switches on: each token maps to a role (admin / member), and admin-only actions — changing settings, switching the model — reject anyone who isn't admin. With no users defined it's single-operator mode: just you. The **rotate** button issues a fresh token and kills the old one, so a paired phone can be cut off in one tap.

C.5 — The audit ledger

Security-relevant events — settings changes, the network being turned on or off, research collaborations — are written to a **tamper-evident audit ledger**: a local, append-only trail so there's always an honest record of what changed and when.

C.6 — Privacy, restated

Every guard, token, ledger, and database above is a local file under your own user profile. No account, no sync, no telemetry. ELI's default posture is *closed* — offline, gated, and yours. You open doors deliberately, one at a time.

That's the whole assistant — inside and out. You don't need any of the last three appendices to use ELI; just talk to it in plain English, and ask "what can you do?" whenever you're curious. Everything here happens on your computer, for you, privately.

> **Updated for v2.4.84 (September 2026).** Every CHAT turn runs the gradient orchestrator at a depth scaled to the reasoning mode, with the specialist bus composed inside it after shared retrieval (`eli/memory/retrieval.py`). Canonical S01–S12 tracing is in `eli/kernel/pipeline_trace.py`; stage 12 learning runs through `learning_coordinator.py`.

Blueprint — ELI Architecture

The structural map of ELI v2. File paths, counts and names below were checked against the source tree on 2026-09-25; where something is only true at runtime it is marked *(runtime)*.

> ELI is a **local-first, offline-by-default, model-agnostic** cognitive runtime and assistant GUI, and also a self-hosted web app (`api/server.py`, described in `ELI_USER_MANUAL.md`). No cloud on the inference path, no hardcoded model. **195,242 lines across 431 Python files in `eli/`**, plus `api/server.py` (2,292 lines). 230 capabilities in `capability_manifest.json`, 189 of them routable and 207 in the executor's supported list.

---

0. Design principles

1. **Local-first, offline-by-default.** Outbound network is refused at the socket boundary (`eli/core/netguard.py`) unless the owner enables it; features that need the network opt in.
2. **Model-agnostic.** No model name or size is hardcoded on the inference path. ELI loads whatever GGUF is configured and identifies its family from file metadata.
3. **Grounded.** A dedicated subsystem (§9) exists to stop a local model stating things it cannot back up.
4. **Persona is not scripted away.** ELI has a distinct voice; functional bugs are fixed without flattening it.

1. Repository map

Files and lines per package under `eli/`, measured 2026-09-25.

Package	Lines	Files	Role
`eli/runtime`	35,375	95	Grounding spine, evidence, response and introspection surfaces, daemons, self-improvement
`eli/gui`	27,447	27	PySide6 desktop app, panels, tabs, startup and first-boot wizard, animated face
`eli/execution`	26,293	13	Router and executor: intent → action → side effects
`eli/cognition`	20,824	47	Agent bus, 12-stage orchestrator, inference, persona, reasoning modes, tone and emotion
`eli/kernel`	17,545	8	`CognitiveEngine`, pipeline tracing, scheduler, self-upgrade, world model
`eli/core`	11,222	30	netguard, paths, settings, hardware profile, model download, full control
`eli/perception`	10,220	23	Vision, STT, TTS, wake word, voice tone, OS control, gaze
`eli/memory`	8,770	11	SQLite, FTS5, FAISS, knowledge graph, storage policy
`eli/tools`	5,969	25	Image engine, news, capability registry, document tools
`eli/plugins`	5,911	34	Plugin manager, marketplace client, bundled plugins
`eli/learning`	4,251	14	LoRA self-training pipeline

Package	Lines	Files	Role
`eli/planning`	4,000	19	Proactive daemon, habit scheduler, job, proposal and attention queues
`eli/integrations`	3,339	18	Ollama client, MCP client and server, cross-platform media, MPRIS
`eli/coding`	2,104	12	`CodeAgent`: plan → search → verify → repair
`eli/setup`	1,935	9	Installer, hardware policy, first-run wizard
`eli/utils`	1,838	5	Logging, platform compatibility
`eli/world`	1,793	26	World event bus, local world bridge, avatar and ontology
`eli/system`	1,190	3	Portable app control, process guard
`eli/contracts`	787	3	Typed runtime-status contracts
`eli/onboarding`	709	2	Onboarding interview
`eli/cli`, top level	236	4	Headless REPL, package entry
total	**194,736**	**429**	

The largest files: `eli/kernel/engine.py` (16,014 lines), `eli/execution/executor_enhanced.py` (15,845), `eli/gui/eli_pro_audio_gui_v2_0.py` (13,107), `eli/execution/router_enhanced.py` (8,271), `eli/gui/labs_tab.py` (5,744), `eli/memory/memory.py` (5,677), `eli/cognition/agent_bus.py` (3,517), `eli/runtime/deterministic_grounding_ga` (3,367), `eli/cognition/gguf_inference.py` (3,234).

---

2. Entry points and launch

Command	Target	Notes
`eli`, `eli-v2.0`, `eli-cli`	`eli.gui.app:main`	GUI (default)
`eli-mcp`	`eli.integrations.mcp.server:main`	ELI as an MCP server
`eli-jobs`	`eli.planning.jobqueue_cli:main`	durable job queue CLI
`python -m eli`	`eli/__main__.py`	GUI, or headless with flags
`python -m eli --headless`	`eli.cli.headless`	terminal REPL, no Qt
`./eli.sh`	`python -m eli`	venv launcher
`bash install.sh`		venv, dependencies, clean config seed, verification

Boot path (GUI): `eli/gui/app.py` → first-boot wizard when no model is configured (`eli/gui/panels/startup.py`)

→ `eli_pro_audio_gui_v2_0.py:main()` builds `EliMainWindow` → constructs the `CognitiveEngine` singleton → loads the GGUF named in `config/settings.json` → starts the daemons.

---

3. The request lifecycle

Every request goes through `CognitiveEngine.process()` (`eli/kernel/engine.py`).

```
user input
   │
   ▼
[Router]  eli/execution/router_enhanced.py :: route()
   │   priority pipeline → {action, args, confidence, meta.matched_by}
   ▼
CognitiveEngine.process(action, args, ...)
   │
   ├─ deterministic fast path (OS, media, status, job actions)
   │      → execute_action() → returned verbatim, no model
   │
   ├─ CHAT (any reasoning mode, Quick to Expert)
   │      → AgentOrchestrator.run() at mode depth
   │         S05 planner → shared retrieval → dispatch_specialists()
   │         → context assembly → broker.infer() → output governor
   │
   ├─ non-CHAT action
   │      → AgentOrchestrator → bus + ReAct tool loop
   │
   └─ orchestrator returns None or raises → AgentBus.dispatch() fallback
```

The canonical stage names (`pipeline_trace.STAGE_NAMES`) are S01 PERCEIVE_INGEST, S02 INPUT_GUARDS, S03 ROUTER, S04 GROUNDING_GATE, S05 PLANNER, S06 AGENT_BUS, S07 CONTEXT_ASSEMBLY, S08 INFERENCE_BROKER, S09 REASONING_SYNTHESIS, S10 OUTPUT_GOVERNOR, S11 RESPONSE_DELIVERY, S12 LEARNING_STATE_UPDATE. Enable tracing with `ELI_PIPELINE_TRACE=1` or `scripts/eli_startup.sh --trace`.

---

4. Routing layer (`eli/execution`)

- **`router_enhanced.py`**: `route(text) -> {action, args, confidence, meta}`. A regex-first priority pipeline of ordered prepasses (realtime web lookup, media, file and PDF, memory, control), then a core router, with an LLM intent fallback (`cognition/llm_intent.py`).
- **`execution_planner.py`**: typed `ExecutionPlan` and `RouteDecision` artifacts.
- **`route_contracts.py`, `operator_policy.py`, `shell_gate.py`**: what may be routed and executed.
- **`portable_intent_contract.py`, `media_runtime.py`, `operator_actions.py`**: media and OS intent wiring. Plugin actions are handled directly in `executor_enhanced.py`.

A command is carried out from a sentence that gives it. The last stages of the pipeline match on words wherever they stand, and used to carry commands out from inside sentences that were not telling ELI to do anything: a complaint that named Spotify stopped the film that was playing. Three pieces of form now decide, none of them a list of phrases:

- `runtime/command_splitter.py` reads a message as sentences and says, for each, what it tells ELI to do, or nothing: a sentence that starts with who or what it is about ("you did not open spotify", "it stopped again") or with a question word is about something; one framed as a request ("could you …", "i'd like you to …") or starting with what to do is an instruction. A sentence that turns into one ("this is great, play it again"; in speech, with no punctuation, "that was great can you

play some jazz") is read as its two parts. It also holds the one list of verbs a command starts with (`COMMAND_VERBS`).

- `route_contracts.changes_something(action)` says, from the action's name, whether carrying it out changes anything. `route_contracts.fits(text, intent)` is asked of those that do: the sentence has to be an instruction, and beyond the command's own words it has to say little. The router asks it of the word-matching stages; the engine asks it of whatever the intent model resolves. A candidate that does not fit is passed over and recorded (`router.not_an_instruction`), and that turn is not sent to the intent model.
- A message of commands and talk ("see, you can do it! open spotify") is routed as `message.instructions_and_talk`: the engine carries the commands out through the executor and answers the rest through the full pipeline, with the result in front of the model and told that nothing else ran (`_carry_out_agreed`, the path a "yes" to an offer takes).

What a command names is what it acts on. `portable_intent_contract.web_open_target` and `window_target` are the one place each for an "open …" target that is a website and for the window a command names. A site is opened by an address that was given, is in the one table of site names (`app_aliases.WEBSITE_ALIASES`), or is found when the command runs (`runtime/site_address.py`: the conversation first, then a web search whose result carries the name); "that website" and "the second link" are looked for in the conversation. An address is never built from the words ("go to the youtube tab" used to open `https://the youtube tab`). Maximise, minimise, focus and tab commands keep the window they name; a bare one acts on the window in front. Every control in the catalogue's "App & window control", "Input control" and "Media" groups goes straight to the executor, read from the catalogue and not from a list.

Routes that were once bugs are pinned by tests and by `tools/eval/cases.yaml`: media controls need whole-word, command-shaped input; a bare "do a search" needs a subject; meta-questions stay in CHAT; realtime facts go to WEB_SEARCH; code words such as "repo" match whole words only, so "morning report" is not a codebase audit.

5–6. Agent bus and agents (`eli/cognition/agent_bus.py`)

`AgentBus.dispatch(user_input, intent, ...) -> DispatchResult` selects an agent set (`_select_agents_for_intent`, or a broad fan-out), runs it over a dependency DAG (flat parallel if the DAG fails) and aggregates. `DispatchResult` carries `agent_results`, `action_result`, `memory_context`, `aggregated_confidence`, `grounding_confidence` (agent evidence only), `agents_used`, `confidence_label` and `orchestrator_plan`.

15 registered agents (`_ALL_AGENTS`):

Agent	Role
`BusMemoryAgent`	recall from SQLite, FTS and FAISS into context
`SystemAgent`	runs executor actions (WEB_SEARCH, RUNTIME_AUDIT, …)
`HabitAgent`	learned habits
`SelfImprovementAgent`	recent failures, improvement proposals, corrections
`ProactiveAgent`	proactive patterns and signals
`FrontierAgent`	frontier reasoning hooks
`PluginAgent`	dispatches to registered plugins
`CapabilityAgent`	what ELI can do (capability manifest)
`VoiceAgent`	STT and TTS engine state
`OrchestratorAgent`	plans and coordinates grounded synthesis
`FileCodeAgent`	searches the codebase for code-grounded answers
`ReflectionAgent`	session reflection
`IntrospectionBusAgent`	live runtime and self introspection

Agent	Role
KnowledgeGraphAgent	entities and relations from the knowledge graph
CriticAgent	second-tier verifier; runs after its retriever dependencies

SpecAgent runs user-defined agent specifications against the local model. The coding pipeline, CodeAgent (eli/coding/agent.py, plan → search → verify → repair), is separate from the bus and is invoked for CODE_SOLVE and GENERATE_SCRIPT.

---

7. The orchestrator (eli/cognition/orchestrator.py)

Runs for all CHAT modes at gradient depth, and for non-CHAT actions. Shared retrieval lives in eli/memory/retrieval.py (retrieve_for_turn()), called by both BusMemoryAgent and the orchestrator so a turn is not searched twice:

- **Turn cache**: 8 seconds per process. Its key includes every retrieval limit, so a deeper pass with larger limits is never served the shallower result.
- **Time window first**: when the question names a period ("last week", "the past two weeks"), query_planner.parse_window produces a window. Memories inside it are fetched by date (Memory.memories_between) before any ranking, then ranked by topic. The search reports what the window did (memory_diag).
- **Sequential by design**: the llama.cpp embedder is not thread-safe.
- **Heuristic rerank** (cognition/reranker.py): lexical overlap × recency × importance. A neural cross-encoder is not in use.
- **FAISS tombstones**: vector_store.mark_memory_deleted() marks stale vectors without a rebuild; compact_tombstones() reclaims space.

Mode scaling (reasoning_modes.py): Quick uses the fast planner and a lean specialist profile; Normal, Advanced, Research and Expert use progressively deeper planner budgets and wider mode_chat_agent_profile() fan-out. Quick still runs the orchestrator.

---

8. Inference layer (eli/cognition)

- **inference_broker.py**: the canonical inference path, broker.infer().
- **gguf_inference.py**: the llama-cpp wrapper: live runtime parameters, chat_completion(), runtime snapshot publishing.
- **model_identity.py, model_output_tokens.py**: model family and chat template are read from GGUF metadata, not from the file name; special tokens are stripped per family.
- **reasoning_modes.py**: quick, chain_of_thought, self_consistency, tree_of_thoughts, constitutional_ai (private strategies; the raw reasoning is not shown).
- **context_synthesiser.py, user_info_builder.py**: prompt and context assembly.
- **context_budget.py**: how much of the window memory may use. Output reserve is an eighth of the window; on a recall question the memory floor is a third of it.
- **llm_intent.py**: LLM fallback for ambiguous routing. Decoding is constrained by a GBNF grammar built from the live action catalogue, so the model can only name a real capability and emit well-formed JSON. Backends without grammar support fall back to free text.
- Budgeting lives in engine.py::_build_enhanced_system and core/startup_hardware_optimizer.py.

---

9. Grounding and anti-confabulation (`eli/runtime`, `eli/core/grounding.py`)

- **`deterministic_grounding_gate.py`**: decides when an answer must be backed by evidence and renders self-reports and audits from live measurements. `render_action` is one explicit ordered pipeline of layers (`_stack`), not captured copies.
- **`grounding_escalation.py`**: a checkable factual turn with low grounding escalates (external fact → web agent, self or project fact → broad local fan-out) and hedges honestly if nothing grounds it. Offline (the default), a question about what does not change (a definition, history, who wrote a book) is answered from the model's own knowledge, the prompt saying the network is off; only one about what changes after a model is trained (news, prices, results, office-holders, release dates, this year or last: `changes_over_time`) is hedged. Env: `ELI_GROUNDING_ESCALATION`.
- **`evidence_ledger.py`, `evidence_arbitration.py`**: what counts as evidence, where it is recorded (the ledger also records every executed action), and conflict resolution.
- **`persistence_gate.py`**: keeps internal report dumps out of memory.
- **`response_contracts.py`, `response_policy.py`, `final_response_provider.py`, `user_visible_response_surface.py`**: shape and clean the user-visible answer.
- **`truth_report.py`, `eli_identity_audit.py`, `control_contracts.py`**: runtime truth, identity grounding, control-action contracts.
- **`cognition/output_governor.py`, `persona_hygiene.py`**: the final governance pass, including removal of false diagnoses when retrieval demonstrably ran.

The known weak seam is internal state leaking into spoken output on the plain CHAT path (identity confabulation, telemetry dumps). It is defended, not closed.

---

10. Execution layer (`eli/execution/executor_enhanced.py`)

- `execute(action, args) -> dict`. The executor's supported list holds **207 actions**; the manifest declares **230 capabilities**, of which **189 are routable** and 207 are in the supported list.
- **Fast path** (`engine.py`): deterministic OS, media, status and job actions (`VOLUME`, `MEDIA_CONTROL`, `NEXT_MEDIA`, `OPEN_APP`, `DATE`, `SHELL_EXEC`, `ANALYZE_IMAGE`, `CHECK_JOB`, `BACKGROUND_JOBS`, …) return the executor result verbatim.
- **Calendar and reminders** (`runtime/agenda.py`): ELI's own, stored locally. `ADD_EVENT` (also moves an event: "i meant 8pm"), `LIST_EVENTS`, `REMOVE_EVENT`, and `SET_ALARM` for "remind me …". A notifier thread delivers what is due; the next 36 hours go into every prompt as facts, next to the clock times worked out from the user's message.
- Action families: media and OS control, files and documents (`SUMMARIZE_FILE`, `ANALYZE_PDF[_FOLDER]` including per-file mode and saving into memory, `CREATE_DOCUMENT`), web (`WEB_SEARCH`), news, weather, memory (`MEMORY_STORE`, `MEMORY_RECALL`), code (`CODE_SOLVE`, `GENERATE_SCRIPT`), background jobs, self and runtime reports (`RUNTIME_STATUS`, `SELF_REPORT`, `RUNTIME_AUDIT`, `EXPLAIN_*`), vision, image generation, scheduling.
- **Background jobs** (`runtime/background_tasks.py`): `submit/get/list`. Heavy PDF-folder analysis backgrounds itself; `CHECK_JOB` returns the real result.
- **YouTube playback** lives in `eli/integrations/media/youtube_playback.py`; the executor delegates to it.

Plugins (`eli/plugins`)

Bundled: `calendar`, `document_reader`, `media`, `notes`, `pomodoro`, `system_stats`, `tts`, `weather`, `web`, `web_automation`. State is in `config/plugins_state.json` (gitignored). Custom agents and plugins are trust-gated (§ plugin table below).

---

11. Memory subsystem (`eli/memory`)

- **`memory.py`** (`Memory`, `get_memory()`): long-term store over SQLite and FTS5, plus failure, observation and habit logging.
- **`policy.py`**: the storage policy. Every row carries an origin (`user_said`, `eli_said`, `telemetry`, `news`, `tool`); repeats are merged (`seen_count`); a forgetting curve scales weight by importance, origin and reinforcement; faded derived rows move to `memories_archive`. A daily `run_upkeep()` does the merging, decay, archiving, identity merge and learning-table tidy. Full detail in `memory.md`.
- **`vector_store.py`**: FAISS index at `artifacts/vectors/index.faiss`; embedder `models/embeddings/nomic-embed-text-v1.5.Q4_K_M.gguf` *(runtime)*.
- **Knowledge graph**: `kg_entities` and `kg_relations` (FTS5-backed) in the user database.
- **Working memory** (`cognition/working_memory.py`): session-pinned facts with dates preserved, restored across sessions.
- **Personal memory**: `runtime/personal_memory_*`, `persona_updater`.

Four SQLite files under `artifacts/db/` *(runtime)*: `user.sqlite3` (memory, conversation, knowledge graph, learning tables), `agent.sqlite3` (self-improvement records and agent metrics), `system_index.sqlite3` (OS index) and `coding_memory.sqlite3` (coding bug fixes). The blank `user.sqlite3` template holds 28 tables plus three FTS5 indexes: `memories`, `memories_archive`, `memory_meta`, `memory_lineage`, `semantic`, `conversation_turns`, `conversations`, `session_summaries`, `kg_entities`, `kg_relations`, `recall_log`, `runtime_events`, `learning_replay`, `observations`, `habits`, `habit_events`, `habit_rules`, `user_patterns`, `user_model`, `eli_stances`, `belief_revisions`, `corrections`, `failures`, `error_tracking`, `improvements`, `capability_proposals`, `news_articles`, `news_reflections`. Self-improvement records (`improvements`, `failures`, `corrections`, `capability_proposals`, `error_tracking`) are written to `agent.sqlite3`; the copies in the user database are empty. Recall is hybrid: keyword, FTS5, FAISS, knowledge graph and recent conversation, merged and reranked (§7).

---

12. Perception (`eli/perception`)

- **Vision**: `vision.py` (model-agnostic vision-language, Moondream and Qwen2.5-VL hot-swap; the multimodal encoder is forced to CPU to avoid a CUDA clip segfault), `analyze_image.py`, `ambient_vision.py`, `screen_locator.py`, `gaze_engine.py`, `analyze_csv.py`, `analyze_pdfs.py`.
- **STT**: `local_whisper_stt.py` (faster-whisper, offline when the Net switch is off), `audio_stt.py`, `voice_worker_streaming.py`. Default wake phrases: “computer”, “hey computer”, “eli”, “hey eli” (`wakeword.py`); a music-bleed filter and an echo gate sit in front of the transcript.
- **TTS**: `tts_router.py` (Piper voices), `tts_xtts.py` (optional neural voices).
- **OS**: `os_controller.py` (app, window, keyboard, mouse), `ui_tree.py`, `desktop_capabilities.py`.

---

13. Persona (`eli/cognition/persona*`)

- `persona.txt` (authored) and `persona.auto.txt` (overlay) hold the voice.
- `persona_updater.py` updates the overlay, knowledge graph and user profile each turn.
- `persona.py`, `persona_values.py`, `persona_status.py`, `persona_hygiene.py`.
- `tone_analyzer.py` and `tone_adaptor.py` read the user’s tone; `emotion_timeline.py` keeps a history that lets ELI notice a sustained mood.
- Generation injects a budget-trimmed persona and situation brief (`engine.py::_build_enhanced_system`); a live-runtime fact is injected for model and identity questions so ELI reports the loaded model.

---

14. Daemons and background loops

- **Proactive daemon** (`planning/proactive_daemon.py`): pattern signals (time habit, topic focus, recurring errors, active project); runs an autonomy tick every 30 minutes.

- **Self-improvement loop** (`runtime/self_improvement.py`): started at boot, analyses failures immediately and then every 24 hours. Repair by the coding agent is opt-in and bounded. A failure is stored with a capsule (input, versions, model, error class), and a candidate patch is tried on a hard-linked copy of the tree and compared with the same tests on the untouched tree: only a candidate that fixes something and breaks nothing is applied, every candidate stays in the archive (`code_patches`) with its verdict and cost, and the tests, the evaluator and the safety gates cannot be patched by a candidate. `ELI_SELFPATCH_UNPROVEN=1` restores applying without proof.
- **Habit scheduler** (`planning/habits_scheduler.py`, `habits.py`).
- **Scheduler** (`kernel/scheduler.py`); **background tasks** (`runtime/background_tasks.py`).
- **Code monitor** (`runtime/code_monitor.py`); **ambient vision loop**.
- **World event bus** (`world/world_event_bus.py`).
- **Memory upkeep**: once a day, started asynchronously at engine init and after responses.

---

15. Learning and self-training (`eli/learning`)

A LoRA fine-tuning pipeline: `bootstrap_phi3_base.py`, `base_model_resolver.py`, `dataset_builder.py`, `dataset_filters.py`, `merge_reviewed_datasets.py`, `export_trainable_dataset.py`, `training_preflight.py`, `lora_trainer.py`, `lora_trainer_guard.py`, `lora_eval.py`, plus `target_registry.py` (operator-declared targets, any model family) and `review_queue.py` (the human review gate).

---

16. EliWorld (`eli/world`, the EliWorld tab, `kernel/world_model.py`)

An internal world the agent inhabits. `world_event_bus.py` receives runtime events; `local_world_bridge.py` bridges to the model; snapshots, ledger and journal are under `artifacts/world/`. Experimental.

---

17. Core infrastructure (`eli/core`)

- **`netguard.py`**: offline by default: `guarded_urlopen`, a process-wide socket guard (fail-closed on non-loopback while offline), and `allow_network()` for deliberate user-initiated fetches.
- **`paths.py`, `portable_paths.py`, `legacy_paths.py`, `db_paths.py`**: path resolution (dev tree vs platform directories, `ELI_*` overrides, read-only frozen bundles).
- **`runtime_settings.py`, `config.py`**: settings load, merge and heal; `DEFAULTS` are clean (offline, no model, wizard on). `config/settings.json` is per user and gitignored, seeded from `config/templates/settings.template.json`.
- **`hardware_profile.py`, `startup_hardware_optimizer.py`**: detect GPU and VRAM and pick context, GPU layers and batch (one fit calculation shared by the loader and the startup dialog).
- **`model_download.py`**: curated GGUF downloader (7 catalogue entries, resumable, magic and size validated, netguard-gated). Install-time menu only.
- **`grounding.py`**: shared grounding helpers.

---

18. GUI (`eli/gui`)

PySide6 (LGPL). `eli_pro_audio_gui_v2_0.py` (main window, 13.1k lines), `app.py` (boot), `labs_tab.py` (Labs, 5.7k lines), `panels/` (`startup.py`: StartupModelSelectionDialog, FirstBootWizard and the hardware tuning dock; `settings.py`, `agent_wizard.py`, `permission_dialog.py`), `tabs/` (`eli_world_tab.py`, `experimental_tab.py`, `marketplace_tab.py`, `tasks_tab.py`, `training_tab.py`), `docks/` (operator console, proactive dock), `widgets/` (animated face, voice downloader, Ollama selector). `qt_compat.py` allows a PyQt fallback from source.

---

19. On-disk layout (`artifacts/`, runtime)

```
artifacts/
├── db/{user,agent,system_index,coding_memory}.sqlite3
├── vectors/index.faiss            # semantic index
├── conversations/                 # conversation exports
├── documents/ , scripts/          # generated artifacts
├── runtime/                       # runtime state and capability inventory
├── runtime_snapshot.json          # live model/runtime truth
├── world/                         # snapshots, ledger, journal
└── analyze_image_*/               # vision outputs
config/  settings.json (gitignored) · templates/ · plugins_state.json · voices/ …
models/  <your>.gguf · embeddings/ · whisper/ · image/
tts_piper/  Piper voice files shipped with the repo
```

---

20. Known seams and weak points

1. **Large files.** `engine.py`, `executor_enhanced.py`, the main GUI file and the router hold about a third of the code. The executor is an action ladder with a high regression surface.
2. **Internal state → spoken output.** The recurring failure class. The grounding gate and escalation defend it; they do not close it on the plain CHAT path.
3. **Swallowed errors.** 162 handlers are a bare `except: pass` and 4,392 catch `Exception`, most of them logging at debug level. A ratchet test (`tests/claims/test_no_silent_swallow.py`) stops the silent count rising.
4. **Latency versus model size.** On an 8 GB GPU a 24B model at Q5 offloads few layers and answers in minutes; a 7–9B Q4 is the sweet spot. The startup dialog shows the real fit.
5. **Two GUI monoliths.** `eli/gui/eli_pro_audio_gui_v2_0.py` and `eli/gui/labs_tab.py` are single files of 13.1k and 5.7k lines.

---

21. Where to make a change

To change…	Edit…
how input is routed to an action	`eli/execution/router_enhanced.py`
what an action does	`eli/execution/executor_enhanced.py`
which agents run and how grounded	`eli/cognition/agent_bus.py`
the 12-stage orchestration	`eli/cognition/orchestrator.py`
prompt and persona budget, the engine spine	`eli/kernel/engine.py`
anti-confabulation	`eli/runtime/grounding_escalation.py`, `deterministic_grounding_gate.py`
final answer shaping	`eli/runtime/*response*`, `cognition/output_governor.py`
memory store and recall	`eli/memory/memory.py`, `policy.py`, `retrieval.py`, `vector_store.py`
model loading and inference	`eli/cognition/gguf_inference.py`, `inference_broker.py`
offline and network policy	`eli/core/netguard.py`
paths, settings, model download	`eli/core/{paths,runtime_settings,model_download}.py`
the GUI	`eli/gui/eli_pro_audio_gui_v2_0.py`, `eli/gui/panels/`, `eli/gui/tabs/`
evaluation and regression	`tools/eval/`

Plugins, agent specifications and training surfaces

`eli/plugins/`: a loader plus a gated marketplace client

Module	Lines	Role
`manager.py`	708	discovery, enable and disable, install and uninstall
`permissions.py`	395	capability vocabulary, consent decisions, grants, audit ledger
`manifest.py`	302	manifest schema and static capability verification against source
`integrity.py`	291	sha256 pinning, ed25519 signatures, operator-trusted publishers
`security_scan.py`	547	multi-engine malware scanner
`marketplace.py`	793	federated registries, preview and install, licence keys
`mcp.py`	481	MCP server config, runtime preflight, handshake verification

`_plugins_dir()` resolves through `paths.plugins_dir()`, and `_plugin_search_dirs()` scans bundled built-ins and user installs, so a downloaded plugin is never written into a read-only installation.

`eli/cognition/`: agents get a specification

Module	Lines	Role
`agent_spec.py`	436	`AgentSpec`: objective, prompt, triggers, success criteria, examples
`agent_trust.py`	262	path-keyed trust with provenance, scanning, revocation

`agent_bus.SpecAgent` runs a spec against the local model and scores the output against the spec's own criteria. Spec agents execute no arbitrary code and bypass the trust chain.

GUI surfaces

`tabs/training_tab.py` (Labs ▸ Training), `tabs/marketplace_tab.py` (Settings ▸ Marketplace), `panels/permission_dialog.py` (consent dialog and worker-to-GUI-thread bridge).

Paths

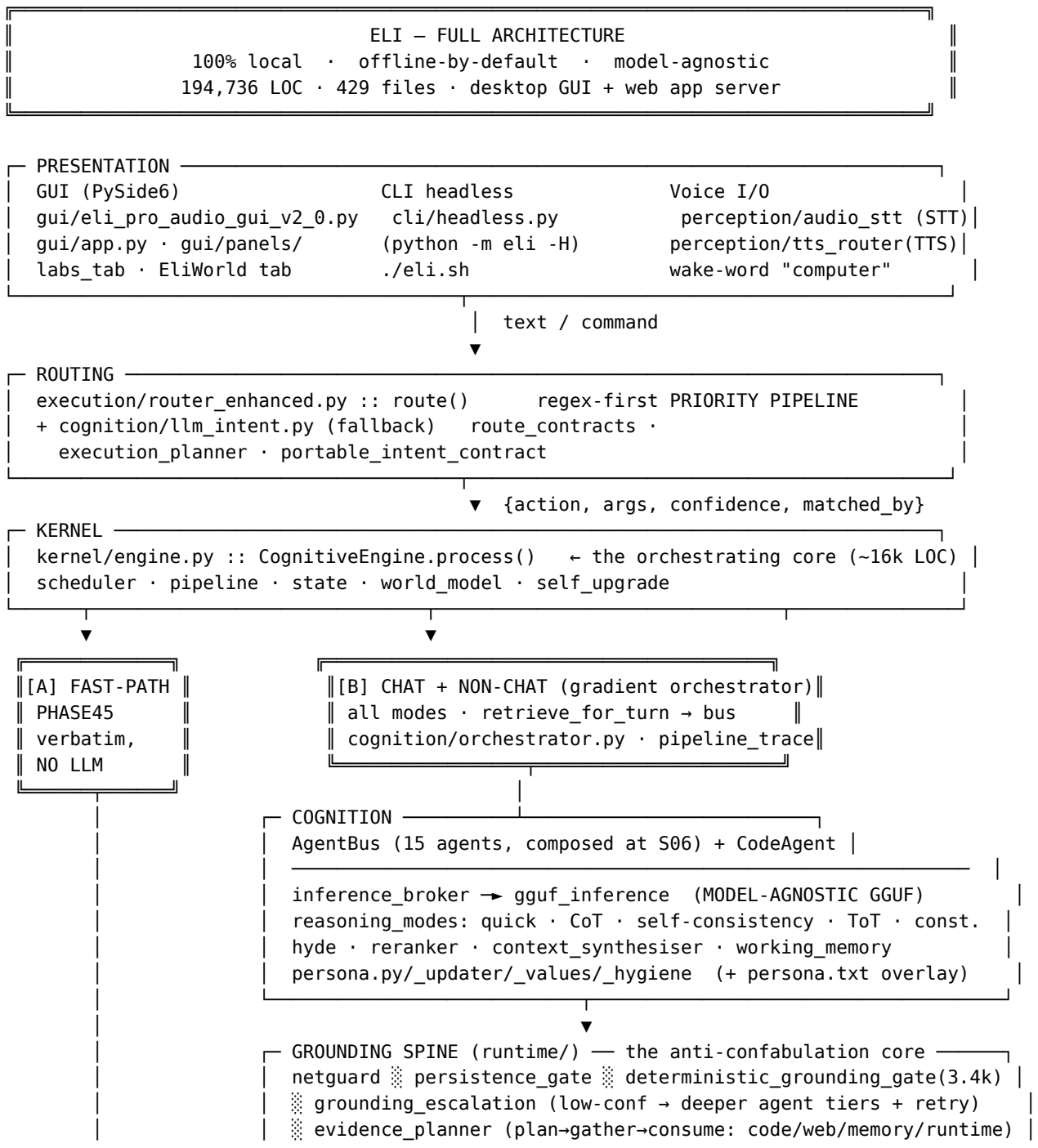
`paths.learning_dir()` joins `models_dir()`, `voices_dir()` and `plugins_dir()`. The rule: runtime state never lives in the installation, because `project_root()` is a read-only mount in a packaged build.

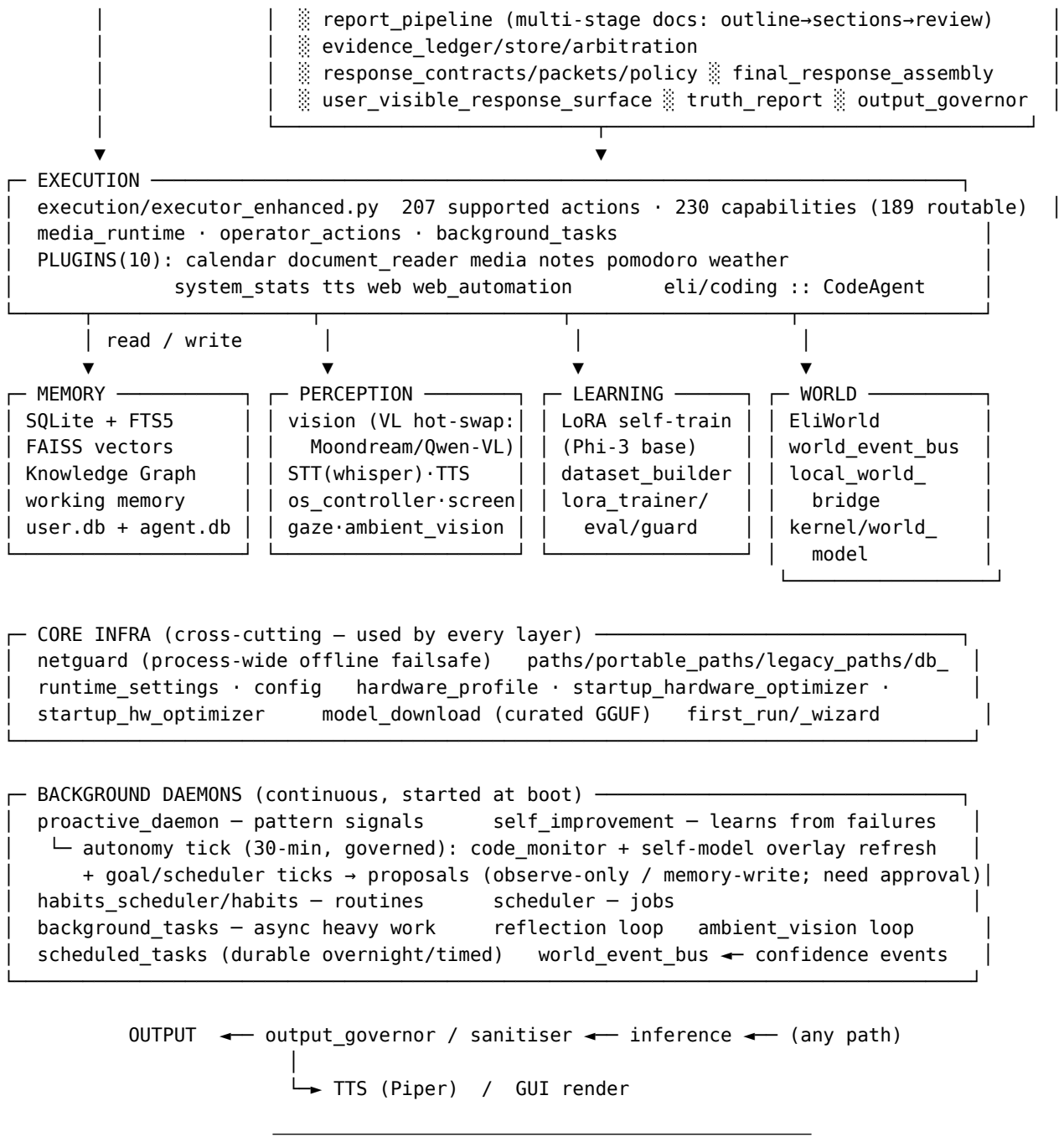
Blueprint — ELI Full Architecture (ASCII)

Updated for v2.4.84. All CHAT modes → gradient orchestrator; bus composed at S06.

The entire system in one drawing, plus the module tree and data layout. Grounded in the real source (see `architecture.md` for prose, `diagrams.md` for the pipeline/memory/gating close-ups). Every layer and box maps to a real path.

A. Full system — layered block diagram





B. The 15 agents

AGENT BUS (cognition/agent_bus.py :: AgentBus.dispatch → DispatchResult)
selects a set → runs on a dependency DAG → aggregates grounding_confidence

BusMemoryAgent	recall: SQLite/FTS5/FAISS → context
KnowledgeGraphAgent	entities/relations from the KG
SystemAgent	runs executor actions (WEB_SEARCH, RUNTIME_AUDIT, …)
OrchestratorAgent	plan / coordinate grounded synthesis
FileCodeAgent	searches the codebase for code-grounded answers
IntrospectionBus…	live runtime + gathers identity/awareness audits as evidence (persona summarises — never a data dump)

```
| CapabilityAgent     | capability manifest (what ELI can do)                 |
| ReflectionAgent     | session reflection / insights                         |
| ProactiveAgent      | proactive patterns / signals                          |
| HabitAgent          | learned habits                                        |
| SelfImprovementAgent| self-improvement state                                |
| FrontierAgent       | frontier reasoning hooks                              |
| PluginAgent         | dispatch to registered plugins                        |
| VoiceAgent          | STT/TTS engine state                                  |
| CriticAgent         | second-tier verifier: checks retriever agents' output |
├─────────────────────┼───────────────────────────────────────────────────────┤
| + CodeAgent         | SEPARATE pipeline: plan→search→verify→repair          |
|   eli/coding/agent  | (CODE_SOLVE / GENERATE_SCRIPT) — not a bus agent      |
└─────────────────────┴───────────────────────────────────────────────────────┘
```

C. Module tree (LOC · key files · role)

```
eli/  (194,736 LOC, 429 files)  ·  api/server.py  (FastAPI web app + dashboard)
│
├── __main__.py ·····················  entry dispatch (GUI | --headless)
│
├── kernel/            17.5k ——— the core
│   ├── engine.py      16.0k     CognitiveEngine.process() — the spine
│   ├── scheduler.py             timed jobs
│   ├── pipeline.py · state.py · world_model.py · self_upgrade.py
│
├── execution/         26.7k ——— route → act
│   ├── executor_enhanced.py   16.0k   207 supported / 230 manifest
│   ├── router_enhanced.py      8.4k   priority pipeline
│   ├── execution_planner.py · route_contracts.py
│   ├── operator_actions.py · operator_policy.py
│   ├── media_runtime.py
│   └── portable_intent_contract.py
│
├── cognition/         20.8k ——— think
│   ├── agent_bus.py    3.5k   15 agents + dispatch
│   ├── orchestrator.py        12-stage deep retrieval
│   ├── gguf_inference.py 3.2k · inference_broker.py   model-agnostic inference
│   ├── reasoning_modes.py · hyde.py · reranker.py · llm_intent.py
│   ├── persona.py/_updater/_values/_status/_hygiene  (+ persona.txt, persona.auto.txt)
│   ├── context_synthesiser.py · working_memory.py
│   ├── output_governor.py · response_governance.py · response_sanitizer.py
│   └── grounded_status.py · introspection_agent.py · tone_analyzer.py
│
├── runtime/           35.4k ——— grounding spine + daemons (95 files)
│   ├── deterministic_grounding_gate.py 3.4k · grounding_escalation.py
│   ├── evidence_ledger/arbitration.py
│   ├── persistence_gate.py · truth_report.py · control_contracts.py
│   ├── response_contracts/packets/policy.py · final_response_assembly/provider.py
│   ├── user_visible_response_surface.py · personal_memory_*.py · reflection.py
│   ├── background_tasks.py · self_improvement.py · code_monitor.py
│   ├── capability_sync.py · pending_proposal.py · runtime_policy.py
│   └── agenda.py (calendar + reminders)
│
├── memory/             8.8k ——— remember (11 files)
```

```
│   ├── memory.py       5.7k   Memory · SQLite + FTS5 · storage policy in policy.py
│   └── vector_store.py        FAISS index
│
├── perception/        10.2k ─── sense (23 files)
│   ├── vision.py · analyze_image/csv/pdfs/mesh.py · ambient_vision.py
│   ├── audio_stt.py · local_whisper_stt.py · voice_worker_streaming.py
│   ├── tts_router.py · os_controller.py · screen_locator.py · gaze_engine.py
│
├── planning/           4.0k ─── proactivity (19 files)
│   ├── proactive_daemon.py · habits_scheduler.py · habits.py · jobqueue_cli.py
│
├── coding/             2.1k ─── CodeAgent (plan→search→verify→repair, 12 files)
│
├── learning/           4.3k ─── LoRA self-training (14 files)
│   ├── lora_trainer/eval/guard.py · dataset_builder/filters.py
│   ├── bootstrap_phi3_base.py · base_model_resolver.py · training_preflight.py
│
├── plugins/            5.9k ─── manager + bundled plugins (34 files)
│   └── calendar · document_reader · media · notes · pomodoro · weather ·
│       system_stats · tts · web · web_automation
│
├── world/              1.8k ─── EliWorld (world_event_bus, local_world_bridge)
│
├── core/              11.2k ─── infra
│   ├── netguard.py            offline failsafe + allow_network()
│   ├── paths.py · portable_paths.py · legacy_paths.py · db_paths.py
│   ├── runtime_settings.py · config.py · grounding.py
│   ├── hardware_profile.py · startup_hardware_optimizer.py
│   ├── model_download.py
│
├── gui/               27.4k ─── PySide6 desktop
│   ├── eli_pro_audio_gui_v2_0.py 13.1k · app.py · labs_tab.py 5.7k
│   └── panels/  (startup.py: model picker + FirstBootWizard, HardwareTuningDock)
│
├── tools/              7.6k ─── image_engine · news · document tools
├── contracts/ 0.8k · cli/ 0.1k · system/ 1.2k · utils/ 1.8k · setup/ 1.9k · onboarding/ 0.7k
```

D. On-disk data layout

```
artifacts/
├── db/
│   ├── user.sqlite3      memories · memories_archive · memory_meta · semantic ·
│   │                     conversation_turns · conversations · session_summaries ·
│   │                     kg_entities · kg_relations · recall_log · runtime_events ·
│   │                     learning_replay · observations · habits/habit_events/habit_rules ·
│   │                     user_patterns · user_model · eli_stances · belief_revisions ·
│   │                     news_articles · news_reflections   (+ FTS5 indexes)
│   ├── agent.sqlite3     agent_dispatches · agent_metrics · improvements · failures ·
│   │                     corrections · capability_proposals · error_tracking
│   ├── system_index.sqlite3   OS index
│   └── coding_memory.sqlite3  coding_bug_fixes
├── vectors/index.faiss   semantic index   (embedder: nomic-embed-…Q4_K_M.gguf)
├── conversations/
├── runtime/ · runtime_snapshot.json        live model/runtime truth
```

```
├── world/{snapshots,ledger,journal}/
├── documents/ · scripts/ · analyze_image_*/
│
config/   settings.json (gitignored) · templates/settings.template.json ·
          settings.example.json · plugins_state.json
models/   <your>.gguf · embeddings/ · whisper/ · image/     tts_piper/  Piper voices
```

E. The flow in one line

```
INPUT → ROUTER → ENGINE ─┬─ FAST-PATH ──────────────────────────► OUTPUT (verbatim)
                         └─ ORCHESTRATOR (all CHAT modes, gradient)
                                ├─ retrieve_for_turn() ──► MEMORY
                                ├─ dispatch_specialists() ──► 15-agent BUS
                                └─ broker.infer() → finalize_turn() (S12)
   GATES: netguard · persistence · grounding · escalation · governor
   INFERENCE (broker → gguf, model-agnostic) → OUTPUT → TTS/GUI
   BACKGROUND: proactive · self-improve · habits · learning(LoRA) · world · scheduler
```

Companion docs: `architecture.md` (prose, every subsystem) · `diagrams.md` (pipeline / memory / gating close-ups) · `tools/eval/README.md` (the measurement layer).

```
# DAG orchestrator — project-wide execution engine (2026-06-07)

`eli/core/dag.py` was a pure **scheduling** primitive (Kahn topological order,
parallel layers, cycle detection, ancestors/descendants, critical-path). It was
elevated into a full **execution orchestrator** and wired through the pipeline —
additively, so every existing class/function and caller is unchanged.

## The orchestrator (`eli/core/dag.py`)
`Orchestrator` / `run_graph(tasks)` execute a set of `Task`s on their dependency DAG:

- **Parallel intra-layer execution** — independent nodes in a topological layer run
  concurrently (threads); `max_workers=1` ⇒ deterministic sequential.
- **Dependency result-passing** — each node fn gets a `NodeContext` with `results`
  (completed upstream outputs) + a shared mutable `shared` bag.
- **Conditional nodes** — `when(ctx)->bool` skips a branch dynamically.
- **Retries + backoff** — per-node `retries` with exponential `retry_backoff`.
- **Fallback** — a `fallback` fn runs if all attempts fail.
- **Per-node timeout** — total node budget (best-effort; enforced via a worker future).
- **Memoisation** — pluggable `cache` keyed by `cache_key` (status `cached`).
- **Priority scheduling** — higher-priority nodes first within a ready layer.
- **Fail-fast + global time budget** — stop scheduling after a failure / budget.
- **Auto-skip** — a node whose upstream didn't complete is skipped with a reason.
- **`critical` semantics** — a non-critical node's failure doesn't fail the run.
- **Deterministic `RunReport`** — per-node status/attempts/timing + order + layers +
  critical path + failed/skipped lists + `to_dict()` for observability.

Pure stdlib; the orchestrator does no LLM/IO — the node callables do.

## Wired through the pipeline
- **Agents (cognition):** the agent bus runs the 15 agents on the orchestrator
  (`_run_agents_orchestrated`, default `ELI_AGENT_ORCHESTRATOR=1`, falls back to the
  layered then flat paths). Dependency-ordered, intra-layer parallel, per-agent
  timeouts (mode + model-tier scaled), upstream results passed downstream, and a
```

  `RunReport` captured per dispatch. Behaviour-preserving: agent fns never raise out (errors → `ok=False`), so edges only order + pass upstream, never gate.
- **Router → executor (observability):** `ORCHESTRATION_STATUS` action ("orchestration status" / "show your agent dag") returns the live engine, execution layers, dependencies, critical path, and last-run telemetry — so ELI can explain its own orchestration honestly.
- **Evidence gathering:** `evidence_planner.gather()` now runs its channels (code / web / memory / runtime) **in parallel on the orchestrator** — each channel isolated (non-critical: one failing yields no evidence, never blocks others), results merged in deterministic `KNOWN_CHANNELS` order, with a sequential fallback. `memory`/`runtime` (no model calls) genuinely overlap; model-using channels serialise safely on the global `_LLM_CALL_LOCK`.
- **report_pipeline — deliberately NOT parallelised.** All model calls serialise on `_LLM_CALL_LOCK`, so parallel section drafting gives zero speedup (model-bound + globally locked) while adding risk. It stays sequential with its own retries.
- **GUI:** a read-only **Orchestration** sub-tab in Labs (and the `ORCHESTRATION_STATUS` chat action) surface the live layers/deps/critical-path + the last dispatch's per-node RunReport.
- Also on the DAG: the coding planner (`coding/plan_graph.py`) and the LoRA pipeline.

Tests
`tests/test_dag_orchestrator.py` (14) cover every feature; existing dag/coding/agent tests + the full suite stay green. Flags: `ELI_AGENT_ORCHESTRATOR`, `ELI_AGENT_DAG`.

Test/project run → review (GUI + chat)
`eli/runtime/test_review.py::run_and_review()` runs the suite (or a subset), **backs up the prior report**, **writes a timestamped errors file** capturing failing tests (possible errors) under `artifacts/test_reports/`, and returns a **results-driven options menu** (each option carries a chat `command` routing to examine/propose/generate/eval). Surfaced two ways:
- **GUI:** a "□ Test & Review" Labs sub-tab — run the full suite in the background, see totals + failures + backup/error paths, get **ELI's natural-language summary**, and click result-driven option buttons that hand off to ELI in chat.
- **Chat:** the `TEST_REVIEW` action ("test review" / "run the tests and tell me what to fix") returns the grounded report (LLM-summarised) + the options menu.
- **Option buttons stream into the main chat** (`MainWindow.send_to_chat`) — clicking an option continues the conversation in the Chat tab, not the panel.

Generative actions on the coding agent / planner DAG
- **GENERATE_PROJECT** → `CodeAgent.solve(use_dag=True)`: decompose into a subtask DAG (`plan_graph` over `eli/core/dag`), solve nodes in order, VERIFY the composed module; saves to `artifacts/projects/`. Single-pass `chat` fallback (`ELI_PROJECT_DAG=0`).
- **GENERATE_SCRIPT / CREATE_SCRIPT / WRITE_SCRIPT** → already routed through the verified coding agent (plan → DAG/tree-search → execute → repair → bug-memory).

SELF_IMPROVE routed through the coding agent
`SelfImprovementEngine.propose_via_agent()` (mode `propose`, or auto-detected from "propose/verified fix/fix the failing") turns each recent failure into a coding task and runs the **CodeAgent** (decompose→solve→**verify** via tree-search + execution gate) over them **in parallel on `run_graph`**, returning **verified, propose-only** fixes (gated; nothing applied until "apply self-improvement patch"). This is the delegation pattern realised: SELF_IMPROVE composes the existing coding agent + orchestrator rather than reinventing repair. The Test & Review "Propose verified fixes" option routes here.
Artifact-dir resolution is now consistent — `run_test_report.py` and the conftest

report hook both honour `ELI_ARTIFACTS_DIR` (no real-folder pollution in tests).

Multi-command + timed commands (2026-06-08)
- **Multi-command chaining** (`eli/runtime/command_splitter.py` + router
  `multi_command_prepass` → `MULTI_COMMAND`): one utterance chaining imperative commands
  is split (the "plan"), then the executor routes & runs each segment in order and
  combines results — the planning step that lets the agents/executor handle several
  actions from one sentence. Sequential today; `run_graph` is ready for parallel fan-out
  of independent ("and"-joined) commands.
- **Timed commands** → background workers: any imperative + a future time defers to the
  durable scheduler (see `runtime_planning_world.md`).

ELI Orchestration & Agents — Full Topology

> **Updated 2026-09-30.** All CHAT modes run the orchestrator at scaled depth; retrieval is
> unified in `eli/memory/retrieval.py`; stage 12 learning is centralised in
> `learning_coordinator.py`; the direct-versus-synthesise gate
> (`_deterministic_direct_payload_actions`) has been audited against what the executor
> handlers actually return. Agent-bus confidence now has a relevance term; MULTI_COMMAND
> gained real dependency awareness and crash-survival.

Read-only reference; nothing here changes behaviour.

Source files:
- `eli/cognition/orchestrator.py`: the orchestrator (12-stage pipeline)
- `eli/cognition/agent_bus.py`: the parallel 15-agent specialist bus
- `eli/memory/retrieval.py`: shared turn retrieval (bus and orchestrator)
- `eli/cognition/learning_coordinator.py`: stage 12 `finalize_turn()`
- `eli/kernel/pipeline_trace.py`: canonical S01–S12 logging
- `eli/kernel/engine.py`: wiring and the dispatch gate
- `eli/execution/execution_planner.py`: the typed plan model

Two agent stacks

ELI has **one primary cognition path** for CHAT and a **parallel specialist bus** that the
orchestrator composes (or that the engine falls back to).

1. `AgentOrchestrator` (`orchestrator.py`)

The 12-stage cognitive pipeline. The engine calls it for every CHAT mode, Quick through Expert,
at a depth chosen by `mode_orchestrator_depth()` and `orchestrator_planner_mode()`
(`reasoning_modes.py`). Components:

- **`PlannerAgent.plan_retrieval()`** produces a mode-aware retrieval plan:
  - `fast`: keyword only, no FAISS, document passages only when the question is about a
    document, knowledge graph only for identity, one ReAct iteration, no HyDE;
  - `balanced` (default): keyword, semantic and knowledge graph, document passages (the best
    ones when the question is about a document, otherwise only passages that clearly bear on
    it), three ReAct iterations;
  - `deep`: everything, large budgets, full HyDE, three ReAct iterations.
  The orchestrator then attaches the time window found in the question
  (`query_planner.parse_window`) to the plan.
- **`OrchestratorMemoryAgent`** delegates to `retrieve_for_turn()` in `memory/retrieval.py`
  (shared with the bus): HyDE expansion, keyword and FTS5, FAISS semantic and knowledge
  graph, `hybrid_merge`, then a heuristic rerank (`rerank_candidates`). Document passages come

  from `engine.document_rag` (`memory/document_index.py`) and are not merged into the evidence rows: they are shown whole, best first, in their own block. Sequential by design: the llama.cpp embedder is not thread-safe.
- **`ExecutorAgent`** is a thin wrapper over `executor_enhanced.execute`.

Flow inside `AgentOrchestrator.run()`:

- **Non-CHAT actions**: dispatch the AgentBus for specialist evidence, then a **ReAct observation loop**: run the executor, ask the loaded model for `ANSWER` or `TOOL:<action> <args>`, chain to the next tool and accumulate observations. One iteration in fast mode, up to three otherwise. The proposed tool is validated against the executor's `SUPPORTED_ACTIONS` (207 actions) and the loop stops on an unknown action; `intent["args"]` are merged rather than overwritten. For "grounded synthesis" actions the observations are assembled into context and passed to the model; for direct actions the executor result is returned as is.
- **CHAT**: planner → shared retrieval → `dispatch_specialists()` (mode-aware fan-out; memory skipped when already prefetched) → context assembly and a retrieval-diagnostics block → persona handoff → generation. The assembled context is ordered by what must survive a trim (`context_budget._BLOCK_PRIORITY`): retrieval diagnostics, the turn record (only when the user asks about ELI's own behaviour), the period log, document passages, verified memories, reranked evidence, recent turns. Private reasoning modes (Normal, Advanced, Research, Expert) hand off to `engine._run_chat_reasoning_loop`. The bus is composed on the CHAT path and is not bypassed in Quick mode.

Following up on what ELI said. Whatever a reply offers, asks or proposes, the user's next message is read against it, in code, whatever the conversation is about (`runtime/pending_proposal.py`).

What a reply leaves open. `_store_assistant_turn` calls `read_reply` on every reply and stores the result, replacing the last one (it lapses after 30 minutes):

- **steps** ELI offered, each with the number it was listed under. A step is an *action* when the router has a rule for its words; a *task* when ELI would do it by writing (explain, draft, compare, plan); *explicit* when it is something only the user's own words may start (`llm_intent._EXPLICIT_ONLY_ACTIONS`). An action is kept wherever ELI names it. A task is kept only where ELI asked for a go-ahead: its own offer, as a question ("Want me to walk you through it?") or as a statement ("let me know if you'd like me to draft it", "I can turn this into a checklist if you'd like"), or the items of a list its question points at ("Shall I go ahead with any of these?"). A list of facts followed by an unrelated offer is not a set of tasks, and "can I help with anything else?" offers nothing to carry out.
- the **question** the reply ended on, if it ended on one.
- a **detail an action asked for**. A handler that returns `awaiting` ({command, action, needs}) or `choices` has it stored by the executor ("When is it?", "Which should go?", "Where?"). `engine._asks_the_user` keeps such a result out of the failure log and the re-plan, and shows its text as it is; so is any not-ok result whose text ends by asking.

Offers are read from the reply as the user saw it. The stored copy has closing filler removed by the output governor, which used to delete "let me know if you'd like me to ..." along with "let me know if you need anything else"; a particular offer is now kept.

What the next message does (`router._stage_pending_proposal_confirm`):

- a decline drops what was offered;
- consent (`is_consent`: "yes please", "go ahead", "sounds good", "do 1-3") covers the steps picked by number or by name, else all of them (`chosen_items`);

- a pick needs no "yes" (`selection`: "the second one", "2 and 3", "the abstract one");
- the detail an action asked for (`is_the_detail`: a day or time, or a place) runs the original request again with it;
- anything else is a new message. One word is then an answer, not a fragment to ask again about.

Carrying it out (`engine._carry_out_agreed`): agreed actions run through the executor (`MULTI_COMMAND` with `results_only`, so the reply is each step's outcome). Agreed tasks run as a nested turn whose message is the task in the user's voice (`task_message`), so retrieval, budget and the reply are about the task and not about the words "yes please"; that turn is CHAT without routing or resolving ELI's words again, and `engine._follow_up_line` tells the model the user agreed and what has already been done. An explicit-only step is never run and never handed to the model: the reply says which words to use. The outer turn stores what the user typed and the reply they saw.

A short reply after a question that is not a pick or a detail goes to the model with the question stated (`engine._follow_up_line`: "your last message ended by asking ...; their message is the answer; act on it").

Three older rules still hold. *One routing per turn*: `engine._route_once` caches the router's answer, because routing has an effect (a "yes" spends the offer it confirms). *ELI's own sentence is not a command*: `_stream_with_followthrough` runs a promise in a reply only when it is a read (`_FOLLOWTHROUGH_READS`) or something the user's message asked for. *A mention with a clock time* that is not on the calendar gets a fixed offer sentence (`_agenda_offer`); a part of the day ("tonight") is not offered back as a time.

DAG pool tasks run inside a copy of the caller's context (`core/dag.py`), so an action executed on a worker thread is recorded on the turn and carries the user id.

The direct-versus-synthesise decision is a set membership test in `eli/kernel/engine.py`:

- `_DIRECT_RESULT_ACTIONS` (194 entries, module level; the control path reads it as `_deterministic_direct_payload_actions`): the executor's `content` or `response` is returned verbatim in quick mode; in other modes such an action may be re-narrated by `_compact_grounded_synthesis()` (constrained to quote from evidence, validated against it, falling back to the raw evidence).
- `_verbatim_always_actions` (16 entries): verbatim in every mode.
- `_shown_as_is(action, reasoning_mode)` is the same decision for the two other places an action result is returned (the agent-bus path and the general executor path). They used to keep shorter lists of their own, so a calendar or timer result went through a full generation there and came back reworded. `_ALWAYS_AS_IS` (calendar, alarm, timer, `MULTI_COMMAND`, `SEQUENCE`) is verbatim in every mode on those paths.
- `eli/runtime/response_contracts.py::_QUICK_ACTIONS` (8 entries) only feeds the prompt header and never decides verbatim versus synthesis.

The 187 routable actions were each checked against their executor handler's real return shape (not assumed from the name): raw file reads, shell output, MCP results, transcriptions and OCR text, confirmations and status reports are verbatim; `ANALYZE_IMAGE`, `ANALYZE_PDF[_FOLDER]`, `SCREEN_READ_ANALYZE`, `DATA_FABRICATOR`, `GENERATE_PROJECT`, `SEQUENCE` and `MULTI_COMMAND` already return finished text and are verbatim too. Excluded on purpose, each with a reason: `FIX_FILE` and `GENERATE_SCRIPT` (their `content` is a JSON event for the GUI), `RUN_TESTS` (meant to be summarised), `CHAT` (the model call itself), `SHOW_DIFF` (routes to `chat()`), `WEB_SEARCH` (results are evidence, not the answer), `CODE_SOLVE` (generative, goes to the coding agent), `EXECUTE_GOAL` and `NOOP` (no single handler to verify). `tests/test_deterministic_actions_cover_status_and_read_file.py` holds the audited lists and a

completeness test that fails if a routable action lands in neither set nor the exclusion list.

2. `AgentBus`: the parallel 15-agent fan-out (`agent_bus.py`)

15 agents in `_ALL_AGENTS`, each a `_BaseAgent` subclass with a `name` and `timeout_s`:

#	`name`	`timeout_s`	accesses
1	`memory`	5.0	SQLite, FTS5, FAISS; self-gates
2	`system`	8.0	direct action execution (`SYSTEM_ACTIONS`)
3	`habit`	3.0	`user_patterns`, learned and detected habits
4	`self_improvement`	3.0	failures, improvement proposals, corrections
5	`proactive`	3.0	suggestion and anticipation
6	`frontier`	5.0	frontier and awareness model
7	`plugin`	6.0	plugin registry (`PLUGIN_ACTIONS`)
8	`capability`	6.0	capability manifest
9	`voice`	5.0	TTS and voice subsystem
10	`orchestrator`	3.0	bus-level planner; emits a plan dict only
11	`file_code`	4.0	source tree and code introspection
12	`reflection`	4.0	reflection log and insights
13	`introspection`	4.0	runtime and cognition self-inspection
14	`knowledge_graph`	3.0	entity and relation graph (runs after `memory`)
15	`critic`	2.0	verifies retriever output (runs after `memory`, `file_code`, `knowledge_grap

`SpecAgent` instances (user-defined specifications) and custom agents register on top of these.

Execution (`AgentBus.dispatch`):

- **Selective fan-out**: tiny filler chat → `{memory, orchestrator}`; non-chat → `_select_agents_for_intent` (a minimal set from a keyword and action ladder); plain CHAT → broad fan-out across all enabled agents.
- **Dependency DAG**: `_AGENT_DEPENDENCIES` orders agents in topological layers, running the agents of a layer in parallel; `knowledge_graph` waits for `memory`, and `critic` waits for its four retrievers. If the DAG fails the bus falls back to a flat parallel run.
- **Per-agent hard timeout**: a timeout or exception becomes a failed `AgentResult` and never blocks the response.
- **Aggregation** (`_aggregate_confidence`): per-agent contribution = evidence quality × evidence density × a calibration learned per (agent, action) × relevance (2026-09-29: `term_overlap(user_input, representative_text) → relevance_gate()`, gated on ≥40 chars of text; a fluent, evidence-dense result that's off-topic for what was actually asked no longer scores as highly as one that's on-topic — env `ELI_AGENT_BUS_RELEVANCE_GATE` to disable); a single-agent cap; an empty-bus ceiling; a corroboration bonus when several agents contribute. `CriticAgent`'s corroboration score gets the same relevance discount: two sources can agree strongly with each other while both being off-topic.

When each runs

Situation	Path
CHAT (Quick, Normal, Advanced, Research, Expert)	**AgentOrchestrator** at mode depth → shared retrieval → `di
Non-CHAT action (any mode)	**AgentOrchestrator** → bus and ReAct loop
Orchestrator returns None or raises	falls back to **AgentBus.dispatch()** directly
Phatic or ultra-short filler (engine heuristic)	may use a lean bus profile inside the orchestrator

Stage 12 side effects (store the turn, publish meta, `_learn_from_result`) run through

`learning_coordinator.finalize_turn()`, one entry point for every CHAT exit.
Exits that never reach it (the orchestrator stream, middleware answers) are completed by `engine._finalize_turn_backstop`, which runs once when a top-level turn ends normally and does only what the turn did not: the user's message, then the reply, episodic memory for chat-like turns, and the response meta. Nested turns (a multi-question split, a follow-through re-run) store nothing; their parent stores the real message and the reply the user saw.

Planning artifacts

1. **ReAct loop**: the only planner that sequences execution (tool → observe → decide → next).
2. **`PlannerAgent.plan_retrieval`**: plans retrieval budgets, not actions; used every CHAT turn.
3. **Bus `OrchestratorAgent` plan**: an `orchestrator_plan` dict stored in the trace for display, persona handoff and status; never executed.
4. **`execution_planner.build_execution_plan`** (`ExecutionPlan`, `PlanStep`): the canonical typed plan. `AgentBus.dispatch` builds it each turn, injecting the result of `_select_agents_for_intent` as `agent_profile`, and drives the active agents through it; it is exposed as `DispatchResult.execution_plan`. `EXECUTE_GOAL` also builds a plan through it. **`.steps` is never executed anywhere** — only `.agent_profile` (an agent-name filter) is read; `EXECUTE_GOAL` renders `.steps` as decorative numbered text, nothing more. Confirmed dead scaffolding for real step execution, not a mechanism to extend.
5. **`MULTI_COMMAND` (`command_dependency_graph.py`, 2026-09-29)**: the only mechanism in the codebase that actually, mechanically executes a multi-part request (as opposed to items 3-4 above). One optional LLM call identifies real dependency edges among an already-split command list (validated through the shared `eli.core.dag` engine, falling back to fully independent on any failure); the executor runs the commands in topological layers, and a step whose dependency failed is skipped and reported as such rather than blindly run. Crash-survival is separate (`command_sequence_log.py`): each step's outcome is durably recorded, and a resubmit of the same text within an hour skips steps already known to have succeeded.

Where it is still weak

1. **Bus agents cannot consume each other's output** beyond the declared DAG edges; only `knowledge_graph` and `critic` have dependencies.
2. **Planning is only partly consolidated.** The engine's `_build_runtime_orchestrator_plan` (a rich stage dict) and the bus `OrchestratorAgent` plan coexist with the typed `ExecutionPlan`; only the ReAct loop executes a sequence.
3. **Timeouts do not cancel work.** `future.result(timeout)` only stops waiting; a timed-out write-capable agent (memory, habit) can still land a late database write.
4. **Confidence is coupled to a fixed evidence-key schema.** `_evidence_density` counts known keys; a custom agent returning useful prose under an unknown key contributes zero.
5. **No early exit for direct actions; failures are debug-only.** The bus waits for the slowest selected agent even when a direct `action_result` exists, and timeouts log at debug level with no health surface, although an `agent_metrics` table exists.

Fixed and worth knowing: `_apply_runtime_policy_timeouts()` is applied to built-ins and again after `_load_custom_agents()`, so custom agents get hardware-adapted timeouts too.

Highest leverage now: an agent-health surface built from `agent_metrics` and `pipeline_trace`, and a cooperative cancel token for write-capable agents.

Habits reach the chat path

The bus `HabitAgent` reads both `get_habit_rules()` and `get_detected_habits()` (the `habits` table the proactive daemon fills), emits a `summary` and `detected_habits`, and

`DispatchResult.to_context_block` renders the summary into the chat context. Goal autogenesis (`planning/goal_autogenesis.py`) feeds the autonomy and goal-tick stack from ELI's own signals; see `runtime_planning_world.md`.

Custom agents have a specification and a trust chain

A custom agent was once a `.py` file dropped in a directory with a `name`, a `timeout_s` and an optional free-text persona. It is now **data, not code**.

`AgentSpec` (`eli/cognition/agent_spec.py`)

Field	Required	Purpose
`objective`	yes	one sentence on what this agent is responsible for
`system_prompt`	yes	the instruction the model receives
`triggers`	at least 1	`keyword`, `regex`, `action`, `always`
`success_criteria`	at least 1	runnable checks: contains, not_contains, regex, min/max length, non_empty, is
`examples`	no	input and expected checks, so the agent can be tested before going live
`permissions`	no	capabilities from the plugin vocabulary, gated at run time

`validate()` refuses vagueness: an objective under 25 characters or matching a placeholder list (`todo`, `does stuff`, `helper`, ...) is rejected, as is a system prompt under 40 characters. An agent with no trigger is refused because it would never run; one with no success criterion is refused because nothing could tell whether it worked; an `always` trigger is allowed with a warning about latency. `evaluate(output)` runs the criteria and returns a score, which is the measure the wizard's test step uses and the confidence `SpecAgent` reports.
`content_hash()` excludes `created` and `enabled`, so re-saving a spec does not invalidate a trust grant while a real edit does.

`SpecAgent` (`agent_bus.py`)

Runs a spec: check triggers, check declared permissions, call the local model with the spec's system prompt, score the output against the criteria. An agent that fails its own success test contributes nothing. Because a spec executes no arbitrary code, spec agents need no trust grant; the hash, scan and provenance chain applies only to code agents.

Loader

- `_custom_agent_dirs()` puts the data directory first (`<data>/agents/custom`), so a created agent has somewhere valid to live on any install.
- Trust is checked through `agent_trust.inspect()` (see `security.md`), which reports why an agent was not loaded.
- `agent_load_report()` records the per-agent outcome so the GUI can show the reason.
- `reload_custom_agents()` re-scans specs and code without a restart and replaces previously registered spec agents.

ELI Agent Algorithms — what each of the 15 agents actually computes

> **Updated 2026-09-30.** Orchestrator runs for all CHAT modes; bus composed at S06.
> `_aggregate_confidence` and `critic`'s corroboration score both gained a query-relevance term.

Your framing is the right lens: **the DAG structures the reasoning steps; RAG is the engine that pulls the raw information during those steps.** ELI maps onto this cleanly —

- **DAG layer (structure):** the `AgentOrchestrator` 12-stage pipeline + the
  `AgentBus` topological layers (`eli/core/dag.py`) + the coding subtask DAG.
  This is the "search → summarize → check" skeleton.
- **RAG layer (raw information):** a *subset* of the agents are retrievers
  (`memory`, `knowledge_graph`, `file_code`) plus the orchestrator's own
  HyDE→FAISS→RAG→rerank pipeline. These pull evidence.
- **Effectors / reporters:** the rest act (`system`, `plugin`) or report grounded
  runtime facts (`capability`, `introspection`, `frontier`, …).

So there are **15 specialist agents in `_ALL_AGENTS`** plus the **`AgentOrchestrator`**
that sequences them. `SpecAgent` instances and custom agents register on top.

Roles at a glance

Role	Agents
Retriever (RAG engine)	`memory`, `knowledge_graph`, `file_code`
Effector (acts on the world)	`system`, `plugin`
Grounded reporter (read-only facts)	`capability`, `voice`, `introspection`, `frontier`, `reflection`, `
Planner (emits structure)	`orchestrator` (bus-level, descriptive) + the real `AgentOrchestrator`

Every agent shares the same skeleton: a **relevance gate** (cheap keyword/action
test → skip with `confidence=0` if irrelevant, so the bus stays fast) → its
algorithm → an `AgentResult(ok, confidence, data)`. The bus runs them in
topological layers with **per-agent hard timeouts**, then `_aggregate_confidence`
fuses them: `contribution = evidence_quality × evidence_density × learned
calibration × query relevance`, single-agent-capped, corroboration-bonused. The
query-relevance term (2026-09-29, distinct from the per-agent gate above) is
`relevance_gate(term_overlap(user_input, representative_text))` — a fluent,
evidence-dense result with nothing to do with what was actually asked no longer
scores as highly as an on-topic one; `CriticAgent`'s corroboration score gets the
same discount (two sources agreeing with each other doesn't mean either is
on-topic). Env `ELI_AGENT_BUS_RELEVANCE_GATE` disables it.

The 15 specialist algorithms

1. `memory` — hybrid dense+sparse retriever (RAG core) · 5.0s
- **Gate:** `_eli_memory_should_run` (explicit memory actions, memory markers, or
  ≥4-word CHAT; blocks stale context bleeding into commands).
- **Algorithm:** `recall_memory` = **FAISS vector search first** (dense), with
  FTS5/LIKE keyword search as a supplement on cold/short queries (sparse);
  noise-filtered (drops ELI's own reflections/assistant-insights), importance-
  ordered. Plus `search_conversations` (FTS5), recent turns, session summaries.
- **Out:** `{results, conv_hits, memory_context}`. The primary RAG retriever.

2. `system` — deterministic action effector · 8.0s
- **Gate:** `action ∈ SYSTEM_ACTIONS` (121 OS and runtime actions); `LLM_ACTIONS`
  (GENERATE_SCRIPT, etc.) are **deferred** (never run inside the timed parallel
  phase — they'd time out + double-execute).
- **Algorithm:** delegates to `executor_enhanced.execute(action, args)` behind the
  security allowlist/sandbox. Confidence 0.92 ok / 0.20 fail. The hands.

3. `habit` — pattern-store reader · 3.0s

- **Gate:** habit/routine/schedule keywords or `OPEN_APP`.
- **Algorithm:** reads `habit_rules` (all) + `habit_events('app_launch', 14d)`;
  confidence by rule presence. (Frequency mining itself lives in the proactive
  daemon; the agent surfaces the rules + event volume.)

4. `self_improvement` — failure-cluster surfacer · 3.0s
- **Gate:** self/improve/fix/learn keywords or SELF_* actions.
- **Algorithm:** `analyze_failures(7d, clustered by occurrence_count)` + pending
  capability proposals. Feeds the self-upgrade loop (see `background_tasks.md`).

5. `proactive` — artifact reader · 3.0s
- **Gate:** proactive/morning-report/insight keywords or PROACTIVE_* actions.
- **Algorithm:** reads `artifacts/proactive/latest_{context,summary,action}.txt`
  + `PROACTIVE_STATUS`. Surfaces the background daemon's findings.

6. `frontier` — cross-system introspection report · 5.0s
- **Gate:** FRONTIER_STATUS / ELI_IDENTITY_AUDIT or "full system status".
- **Algorithm:** `build_frontier_status_report` / `eli_identity_audit` — an
  import + wiring matrix across runtime/memory/self/proactive/image/world/labs.
  Deterministic whole-system audit.

7. `plugin` — plugin effector · 6.0s
- **Gate:** `action ∈ PLUGIN_ACTIONS` (ADD_EVENT, GET_WEATHER, LIST_EVENTS, REMOVE_EVENT).
- **Algorithm:** `execute(action, args)` into the plugin layer; silent skip
  otherwise.

8. `capability` — capability-manifest reader · 6.0s
- **Gate:** capability/awareness/"what can you do" or LIST_CAPABILITIES etc.
- **Algorithm:** `execute(LIST_CAPABILITIES | AWARENESS_STATUS | CODE_CHANGES)` →
  grounded capability surface (no confabulated feature lists).

9. `voice` — config reporter · 5.0s
- **Gate:** voice/tts/stt/mic keywords.
- **Algorithm:** reads `ELI_TTS_ENGINE`/`ELI_PIPER_MODEL`/`ELI_MUTE` + probes
  `faster_whisper` import. Reports, never emits audio (that's the TTS router).

10. `orchestrator` (bus-level) — descriptive planner · 3.0s
- **Gate:** multi-step trigger groups (e.g. {document, save, open}) or
  GROUNDED_SYNTHESIS_ACTIONS.
- **Algorithm:** emits a structured `plan` dict (now also surfaced as the typed
  `ExecutionPlan`). **Excluded from confidence aggregation** (it plans, it isn't
  evidence). The *executing* planner is the ReAct loop in `AgentOrchestrator`.

11. `file_code` — source-code RAG (grep retriever) · 4.0s
- **Gate:** grounded query or code/file/pipeline/db keywords.
- **Algorithm:** builds query-derived regex patterns, **greps a canonical
  `files_map`** of ELI's own modules (agent_bus, engine, orchestrator, memory,
  router, …), returns `file:line: content` snippets. Lets ELI answer "which file
  does X" from real source, not memory.

12. `reflection` — reflection-log reader · 4.0s
- **Gate:** reflection/pattern/noticed/insight keywords or grounded query.
- **Algorithm:** recent `observations` (≤8) + session summaries (≤3) → insight
  list. ELI's "what I've noticed about how you use me."

13. `introspection` — self-model reporter · 4.0s
- **Gate:** architecture/pipeline/"how do you work" or EXPLAIN_*_RUNTIME.
- **Algorithm:** `IntrospectionAgent` → `get_pipeline()` + `get_memory_stats()` +
  `get_runtime()` → grounded pipeline description. Powers honest self-explanation.

14. `knowledge_graph` — graph retriever (structured RAG) · 3.0s
- **Gate:** non-empty KG.
- **Algorithm:** `context_for_prompt(query)` over `kg_entities`/`kg_relations`
  (FTS5 fuzzy entity match + relation context). **DAG edge `memory → knowledge_graph`:**
  it now seeds the query with `intent["_upstream"]["memory"]` hits (topological
  layering), so the graph lookup is conditioned on what memory just surfaced.

15. `critic` — second-tier verifier (corroboration check) · 2.0s
- **Gate:** ≥2 grounded upstream sources — self-gates to nothing on trivial turns, so it
  never fires when there's nothing to cross-check.
- **Algorithm:** the one bus agent that reasons over **other agents' output**
  (`intent["_upstream"]`) rather than the raw query — **deterministic, no LLM**. It pulls the
  representative text each retriever surfaced this turn, measures **pairwise term agreement**,
  and emits a **corroboration-vs-contradiction** signal. Corroboration alone isn't enough
  (2026-09-29): two sources can agree strongly with EACH OTHER while both being off-topic for
  what was actually asked, so on the non-contradiction branch, agreement is additionally
  discounted by how well the sources match the *query* (`relevance_gate` over their average
  `term_overlap(user_input, text)`), not just each other. `_representative_text` moved from a
  `CriticAgent` staticmethod to a module-level function, since `_aggregate_confidence` now
  needs it too — one owner. **DAG edges:** depends on
  `memory`, `file_code`, `knowledge_graph` and `system`, so it runs in a **downstream topological layer** and
  sees their results — giving the agent DAG a real second tier instead of a flat fan-out.

How the DAG sequences the RAG (your "search → summarize → check")

1. **Route** → action + reasoning mode.
2. **`AgentOrchestrator`** runs for **all CHAT modes** (gradient depth) and for non-CHAT
   actions; for non-CHAT it dispatches the **bus** then runs a **ReAct loop** (tool → observe →
   decide → next tool) — the executing DAG of steps.
3. **`AgentBus`** builds the **agent DAG** (`_AGENT_DEPENDENCIES`), runs agents in
   topological layers, passing each layer's results downstream
   (`memory` → `knowledge_graph`). Retrievers pull evidence; reporters ground it;
   effectors act.
4. **Grounding/evidence layer** gates the synthesis (`output_violates_evidence`).
5. **Coding tasks** decompose into a **subtask DAG** (`plan_graph`) and heavy ones
   run on **background workers** — see `dag.md` + `background_tasks.md`.

That is the literal realisation of "DAG structures the autonomous reasoning steps
while RAG pulls the raw information."

Aspirational next steps (genuinely novel combinations)

These are the "never-before-put-together" moves the current architecture is one
step away from:

1. **Bias/verification agent as an explicit DAG layer.** □ **Shipped** — this is now the

   `critic` agent (#15 above): it depends on the retrievers (`memory`/`system`/
   `knowledge_graph`) and runs *after* them as a topological layer — literally "search →
   summarize → **check for corroboration/contradiction**" — feeding the grounding signal.
   (Originally proposed here as future work; now built.)
2. **A `web` retriever agent** (WEB_SEARCH as a first-class RAG node) so external
   retrieval joins memory/KG/file_code in the same layer, with the verify layer
   cross-checking sources.
3. **Calibration-aware selection.** The bus already learns per-(agent,action)
   calibration; feed it back into *selection* so chronically-zero agents are
   dropped from the DAG for that action (closes the learned-trust/unlearned-
   selection gap noted in `orchestration_and_agents.md`).
4. **Bus↔coding bug-memory unification.** The coding engine's `bug_memory` and the
   agents' `self_improvement` failure log are two halves of one "what went wrong
   and how we fixed it" store — merging them gives ELI a single experiential
   memory across chat and code.
5. **Orchestrator stage-dict → typed `ExecutionPlan` DAG.** Fold the engine's rich
   stage plan into `ExecutionPlan.steps` as a true DAG so one plan type drives the
   whole pipeline end-to-end.

`SpecAgent` and the measurable-agent contract

A further agent class joins the fifteen built-ins. `SpecAgent` is not hand-written
Python — it is an interpreter for an `AgentSpec` (see `orchestration_and_agents.md`),
and its algorithm is deliberately small:

run(user_input, intent): if not spec.should_run(user_input, intent.action): # trigger gate return skipped # costs one regex/substring for cap in spec.permissions: # capability gate if not permissions.check(agent, cap).allowed: return denied output = broker.infer(prompt=user_input, system=spec.system_prompt, max_tokens=spec.max_tokens, temperature=spec.temperature, background=True) verdict = spec.evaluate(output) # the measure return AgentResult(ok=verdict.ok, confidence=verdict.score, data={“content”: output if verdict.ok else ““})

Three properties worth stating:

- **The trigger gate runs first and is cheap.** An untriggered turn costs a substring
  or regex test, not an inference call.
- **Confidence is earned, not asserted.** Every other agent picks its own confidence
  number. A `SpecAgent`'s confidence *is* its score against its own declared success
  criteria — the fraction of checks its output actually passed.
- **Failing its own test yields no evidence.** `data["content"]` is emptied when the
  verdict fails, so a misbehaving custom agent cannot feed unchecked text into the
  bus's evidence fusion. This directly addresses the long-standing weakness that
  evidence density "only counts known keys", by making the agent's own contract the
  arbiter.

Because a spec contains no code, `SpecAgent` bypasses the custom-agent trust chain
entirely — there is nothing to hash, scan or approve.

ELI Memory Subsystem

> **Updated for v2.4.84.** Memory is governed by a storage policy
> (`eli/memory/policy.py`): every row has an origin, repeats are merged, weight follows a
> forgetting curve reinforced by use, and faded derived rows are archived. Recall applies a
> time window before ranking, and reports what it did. Turn retrieval is shared in

> `eli/memory/retrieval.py`; FAISS deletes use tombstones.

`eli/memory/` holds 8,996 lines in 11 files: relational, full-text, vector and graph storage, all local SQLite and FAISS. Companion to `project_overview.md`.

The two layers

ELI's memory is not "a dump of every interaction". There are two layers:

1. **Conversation history** (`conversation_turns`, `conversations`, `session_summaries`): broad and chronological. A turn is kept unless the persistence gate (`runtime/persistence_gate.py`) rejects it as junk. This is what a question about a period of time reads.
2. **Distilled memory** (`memories`, `semantic`, `user_patterns`, the knowledge graph): a smaller set that the gate and the storage policy judged worth keeping, labelled by origin, merged when repeated, and faded when nothing reinforces it. Faded derived rows move to `memories_archive` instead of being deleted.

The memory-runtime report states this explicitly (`EXPLAIN_MEMORY_RUNTIME`, memory internals), with live counts for each layer.

Files

File	Lines	Role
`memory.py`	6,034	the `Memory` class (85 public methods), schema, `DBPaths`, upkeep, module facade
`policy.py`	128	pure storage-policy functions (origin, dedupe key, strength, archive, merge)
`retrieval.py`	310	shared turn retrieval (`retrieve_for_turn`), turn cache, time window (turns and executed
`claims.py`	266	dated claims about the user: valid-from and valid-to, learned-at, supersession
`unified_retrieval.py`	371	the orchestrator stages consume `retrieve_for_turn` through it; formats the ver memory block and the period log
`document_index.py`	706	every document ELI has read, stored whole and searched by passage
`vector_store.py`	677	FAISS index, embedder, tombstones
`knowledge_graph.py`	703	entity and relation graph
`system_index.py`	278	indexed apps, executables, files
`memory_truth.py`	188	read-only inspection used by status surfaces
`memory_adapter.py`	131	compatibility adapter
`__init__.py`	278	facade (`get_memory`, `get_agent_memory`, path helpers)

The storage policy (`policy.py`)

Every stored row gets an **origin** from `classify_origin(source, kind, tags, text)`:

Origin	Meaning	Factor on half-life
`user_said`	the owner said it or confirmed it	1.0
`eli_said`	ELI's own assistant text	0.5
`tool`	executor or observation output	0.35
`news`	news digests and reflections	0.25
`telemetry`	bookkeeping (`is_bookkeeping_memory`)	0.15

Rules, all in `policy.py` and pinned by `tests/test_memory_policy.py`:

- **Dedupe.** `text_key` is a sha1 of the normalised text (first 20 hex characters). A repeat

  bumps `seen_count` and `last_seen` instead of adding a row; `merge_group` keeps the earliest
  event time.
- **Strength.** `2 ** (-days_since_last_touch / half_life)`, floored at `MIN_WEIGHT` (0.05).
  The half-life is the base half-life × origin factor × `(1 + 3 × importance)` ×
  `(1 + 0.6 × ln(1 + uses))`, where uses are repeat sightings plus recalls.
- **Base half-life.** 30 days, scaled by the square root of the median gap between days the
  owner used ELI, clamped to 14–120 days (`adaptive_half_life_days`).
- **Pinning.** A `user_said` row with importance ≥ 0.85 stays at strength 1.0.
- **Archiving.** `should_archive` is true only for a derived (not `user_said`) row that has
  never been recalled, has importance < 0.85, and whose weight is ≤ 0.12.
- **Vector index.** Telemetry rows are not embedded (`wants_vector_index`).
- **Evidence about the user.** Only `user_said` counts (`counts_as_evidence_about_user`).

Columns added to `memories`: `event_ts`, `seen_count`, `last_seen`, `last_recalled`,
`recall_count`, `origin`, `text_key`. Tables added: `memories_archive`, `memory_meta`. The
`semantic` table gained `evidence_count`, `last_seen`, `evidence`.

What counts as the same fact, and what strengthens a memory

- **Dedupe key.** `normalise_text` lower-cases, single-spaces and trims sentence punctuation from the
  edge of each word, and keeps meaningful marks. "I love C++", "I love C#" and "I love C" are three
  facts, "balance -5" is not "balance 5", and "3.5" is not "35". "Not fine!" and "not fine" are one.
  A key-version marker makes upkeep recompute old keys once (`rekey_text_keys`); it never merges.
- **Retrieval is exposure, not use.** Recalling a memory only increments `exposure_count`. Its strength
  is reinforced by `record_recall_outcome(ids, helped=True)`, which the engine calls when the next
  user message does not correct the answer those memories went into; a correction lowers the weight
  and increments `corrected_count`. A wrong memory that keeps being retrieved cannot make itself
  stronger. `recall_count` now means answers a memory helped with, and only that protects a row
  from archiving.
- **What enters durable memory.** `persistence_gate.should_store_memory_text` rejects filler and
  reactions ("haha", "lol that is funny", "hello there"), text under four words, questions, and ELI's
  own internal output. Conversation turns are still logged in full; only durable memory is filtered.

Dated claims (`claims.py`)

`memory_claims` holds what ELI believes about the user as bitemporal rows: `valid_from`/`valid_to`
(when it was true) and `recorded_at`/`superseded_at` (when ELI knew it). A first-person statement such
as "I work nights now", "I moved to Berlin", "I work at Acme Corp" or "my dog is called Max" becomes a
claim when it is stored. A single-valued relation supersedes the old value and keeps it as history,
"I used to work nights" is recorded as history only, "I no longer work nights" retires the standing
claim, and the same value again confirms instead of duplicating. `Memory.claims_current()`,
`claims_during(start, end)`, `claims_known_at(when)` and `claim_history(relation)` answer "what is true
now", "what held in spring" and "what did you believe on 1 March". Retrieval adds the relevant claims
to the evidence (a period question gets what held then), and deleting the source memory withdraws its
claims. Extraction is a small exact set of patterns, not a model call.

What the user is doing now is a claim too: `watching`, `playing`, `reading` and `listening_to`, from
"I'm watching X", "still watching X", "I've been playing X" or "X is on in the background". They are
read from every message the user sends, not only the ones kept as memories, and from what ELI sees
playing (a player title with a season and episode, "Game of Thrones — S2 E5", is `watching` with
origin `observed`). These change often, so the newest value replaces the old one whoever reported
it, and a value that is the other's initials ("GOT" for "Game of Thrones") confirms it. A statement
older than the standing claim, as when history is read again, is kept as history and does not
replace it. Questions match a claim by its relation first ("what am I watching" finds `watching`),

ignoring filler words, newest first.

Forgetting, lineage and conflicts

- **Forget.** "forget that my locker is 212" lists the matching memories and asks; on yes (or `confirm forget memories 12 15`) `Memory.forget(ids)` deletes them and everything derived from them: derived rows through `memory_lineage`, index entries and vector tombstones, claims, recall log rows, semantic-fact quotes, session summaries that carry the same content, profile items and briefs, stances and belief revisions, graph entities created with the memory, and the conversation turns. It returns a report of what was removed.
- **Lineage.** `link_derivation(child, parents)` records that a summary or reflection was made from other memories; `lineage_root` and `independent_sources` mean a fact and its summaries count as one source. A claim's confirmations count distinct roots only.
- **Conflicts.** A claim that is not the user's own word and contradicts one that is stands beside it as `disputed` (both marked contested) instead of replacing it. `claim_conflicts()` lists them, `claims.conflict_question` phrases the question, and `resolve_claim_conflict` records the answer. Hedged statements ("maybe", "thinking of", "if I") are never claims.
- **Retrieval scope.** `retrieve_for_turn` returns `searched`: live memory, exact codes and quoted phrases (looked up as written, so an embedding cannot blur "ORCHID-7319"), dated claims, and the archive, which is searched only when live evidence is thin and its hits are labelled. The diagnostics block states that scope, so "nothing stored" is only claimed for what was searched.

Daily upkeep

`Memory.run_upkeep()` runs at most once a day (`upkeep_due()`), started asynchronously at engine start and after responses (`upkeep_async()`), and skipped under `ELI_TEST_MODE`. Steps:

1. **backfill**: label existing rows with origin, key and event time (after copying the database to a `.pre_policy.bak` file the first time). A dry run reports what it would fill and writes nothing;
1a. **rekey**: recompute dedupe keys once after the key definition changes;
1b. **claims**: read the whole conversation again for claims, oldest first, once per extraction version (`claims.EXTRACT_VERSION`); a version not yet read makes upkeep due on the next start;
2. **consolidate**: merge exact duplicates and rebuild `memories_fts`;
3. **decay**: recompute `weight` with the policy strength (idempotent: a pure function of age, importance and use, so a missed run cannot leave a row over-weighted);
4. **archive**: move faded derived rows to `memories_archive` (`search_archive`, `restore_from_archive` bring them back);
5. **identity**: merge legacy user ids into one stable owner id (`owner_id`, kept in `memory_meta`); only UUID-shaped legacy ids are merged;
6. **learning**: remove repeated identical belief revisions, cap corroboration at the number of days the owner has used ELI, and drop ledger copies of replay rows;
7. **semantic**: fold repeated facts into one row with `evidence_count` and up to five quotes;
8. **documents**: document text the old folder reader filed as memories (`kind` `pdf:<id>`, from a tool) leaves memory and its files go to the document index; graph relations made from tool text go with it (`refile_documents`). Rows like these make upkeep due on the next start. One 349-PDF folder had made 2,399 of 2,929 memories and 969 relations such as "User prefers screening". Not `forget()`: that matches words across the whole profile, and book text shares words with everything.

A run only marks the day done when every step succeeded; otherwise `run_upkeep` reports `status: partial` with the failed steps, and upkeep retries after an hour.
`integrity_report()` reports the state of all of this for the health checks: `ok` is false when vectors are missing, and `status` is `healthy` or `degraded`.

The `Memory` class (`memory.py`)

A single class that owns most persistent concerns:

- **Memory**: `store_memory`, `add_memory`, `recall_memory`, `search_memory`, `search_memories`, `memories_between`, `get_recent_memories`, `adjust_weight`, `apply_weight_decay`, `consolidate_memories`, `archive_faded`, `search_archive`, `restore_from_archive`, `store_episodic`, `store_semantic`, `recall_semantic`, `store_reflective`.
- **Conversation**: `add_conversation_turn`, `store_conversation`, `get_conversation_history`, `get_recent_conversation` (with `since`, `until`, `role`), `get_recent_turns_since`, `search_conversations`, `get_turns_for_day`, `save_session_summary`, `get_session_summaries`.
- **Habits**: `log_habit_event`, `get_habit_events`, `add_habit_rule`, `get_habit_rules`, `get_detected_habits`, `record_habit_run`. `log_habit_event` writes from the engine's `_learn_from_result` (2026-09-30: gated on `source` — a habit-scheduler-fired turn doesn't write here, so ELI's own automation can't reinforce itself as fresh evidence of a routine when `detect_habits` later mines this table for recurrence).
- **Self-improvement and learning**: `log_learning_event`, `log_failure`, `log_correction`, `add_observation`, `log_improvement`, `add_capability_proposal`, `propose_capability`, `get_pending_proposals`, `get_recent_failures`, `get_recent_improvements`, `get_recent_observations`. These tables are written on the **agent** database (`SelfImprovementEngine` opens `get_agent_memory()`); the user database holds empty copies.
- **Health**: `get_stats`, `get_dashboard_counts`, `get_db_routing_info`, `integrity_report`.

`vector_store` is a lazy property; the knowledge graph is reached through `knowledge_graph.get_knowledge_graph()`.

Schema

The blank `user.sqlite3` template holds 27 tables and three FTS5 indexes. `memories` (+ `memories_fts`), `memories_archive`, `memory_meta`, `semantic`, `conversation_turns`, `conversations`, `session_summaries`, `kg_entities` (+ FTS), `kg_relations`, `recall_log`, `runtime_events`, `learning_replay`, `observations`, `habits`, `habit_events`, `habit_rules`, `user_patterns`, `user_model`, `eli_stances`, `belief_revisions`, `corrections`, `failures`, `error_tracking`, `improvements`, `capability_proposals`, `news_articles` (+ FTS), `news_reflections`. `agent.sqlite3` holds `agent_dispatches`, `agent_metrics` and the self-improvement tables.

`emotion_events` is owned by `cognition/emotion_timeline.py`, not the `Memory` class: it opens `user.sqlite3` directly and creates its table on first use. Columns: `ts`, `user_id`, `session_id`, `detected`, `expressed`, `valence` (`negative`, `positive` or `neutral`), `confidence`, `source`, `arousal`, `user_text` and `eli_prior_action` (the action ELI ran on the preceding turn, so it can ask whether the mood turned because of something it did). Reads come back `ORDER BY ts DESC, id DESC`; the `id` tiebreak matters because several reads can land within one second.

Each conversation turn is stored in `conversation_turns`. A `learning_replay` row is also written per turn (it carries action, outcome and reward), and the evidence ledger records the turn once in `runtime_events`; replay rows are not mirrored into the ledger.

Shared turn retrieval (`retrieval.py`)

The single owner of semantic and conversation recall for a turn. `BusMemoryAgent` and the orchestrator both call `retrieve_for_turn()`, so a query is not searched twice with different

budgets.

- **Turn cache**: 8 seconds per process. The key includes session, user, query, time window and every limit (`semantic_limit`, `conv_limit`, `recent_limit`, `summary_limit`, `hop2_limit`, `merge_cap`, `verified_only`), so a deeper pass never gets the shallower result. `invalidate_turn_cache()` clears it.
- **Time window first.** When the planner finds a period in the question (`query_planner.parse_window`), memories dated inside it are fetched by date (`Memory.memories_between`, ordered by importance × weight) and merged with the topic hits, hits outside the window are dropped, and the user's turns inside the window are read from `conversation_turns`, and what ELI did in the window is read from the evidence ledger (`evidence_ledger.actions_between`: every executed action with the executor's own result text, scoped to the asking user). The result carries `window_stats` (candidates, added by date, in window, turns, actions) and the orchestrator logs and reports it (`memory_diag`).
- **The period log.** `unified_retrieval.format_period_log` renders the window's turns and actions as one dated block ("What happened in that period"), in every mode including Quick. The actions the question is about (media, apps, web, timers, files) are listed first with a digest of distinct tracks; when the period is larger than the budget the selection is balanced across its days and a per-day tally says what was left out. A follow-up with no period of its own ("check again in full") keeps the previous turn's period for 15 minutes.
- **Vector hits are re-checked.** A FAISS hit is used only if its memory row still exists and passes the same origin and verification filters as the SQL channels; telemetry rows are not embedded, and tombstones apply to either id field an entry carries.
- **Only questions get a window.** `plan_window` applies a period only to a question or recall request, so a statement that says "last night" or "today" cannot narrow retrieval to that period.
- **Explicit dates.** `parse_window` reads ISO dates ("2026-03-03"), "3 March", "March 3rd 2025", ranges ("between 1 May and 10 May"), "in June 2025" and "during 2024" as well as relative periods. A date with no year is its latest past occurrence.
- **Token budget.** `context_budget.chars_per_token()` measures the loaded model's own tokenizer and falls back to 3.5 characters per token.
- **Hop-2 deepening** when the first hits are sparse; **heuristic rerank** via `rerank_candidates()`; **contradiction detection** through the bus.
- **Dated evidence.** Each recalled line carries its event date (`evidence_format`); working memory keeps the original date rather than the date it was re-pinned.

`recall_memory`: the hybrid retriever

The primitive `retrieve_for_turn()` builds on:

1. **FAISS and FTS5 both run** and are fused by reciprocal rank (`fuse_ranked_lists`); a LIKE scan covers the case where FTS returns nothing. `keyword_only=True` skips FAISS. FTS5 is asked about the `text` column only: the tokenizer splits tags at `_`, so "memory" used to match every row tagged `memory_recall` or `working_memory`, whatever it said.
2. **Noise filtering** (`memory_exclusion_sql` in `core/self_provenance.py`): ELI's own bookkeeping kinds and sources never resurface as recalled user memories.
3. **Importance-weighted ordering** via `COALESCE(importance, 0.5)`.
4. **Reinforcement.** A recalled row gets `last_recalled` and `recall_count` updated and its weight reset to 1.0; the recall is written to `recall_log` with its `memory_id`.

Column detection (`_memory_table_columns`) guards against schema drift between versions.

Document index (`document_index.py`)

What ELI has read, kept whole. The document reader hands a reply the first 8,000 characters of the first 20 pages; the index stores the full text, so a question about page 40 has something

to be answered from.

- **One SQLite file** (`documents.sqlite3`, `paths.document_index_db_path`): `documents`, `passages` (text and a float16 vector each) and an FTS5 table kept in step by triggers. Removing a document cascades to every passage and vector made from it.
- **What gets indexed.** Anything an action reads, summarises or writes (`READ_FILE`, `SUMMARIZE_FILE`, `ANALYZE_PDF`, `CREATE_DOCUMENT`, `DOC_GENERATE`, `GENERATE_DOCUMENT`, `CONVERT_DOCUMENT`, `WRITE_NOTE`, `NEW_NOTE`; hooked in the executor's post-dispatch), a file handed over with a message (`[File: path]`), and ELI's own `documents/` and `notes/` folders on first use. PDF, DOCX, ODT, EPUB and prose text formats only; names that look like secrets (`.env`, keys, credentials, wallets) and files over 96 MB are never stored. An unchanged file is not stored twice; a changed one replaces its passages.
- **How.** A worker thread extracts the whole text (`document_reader.document_sections`, with page numbers for PDFs), cuts it into ~900-character passages that overlap and start on a word, and embeds them with the embedder the memory store already loads. Embedding waits while a conversation turn is live. Wording search works as soon as the text is stored.
- **Search** (`search`): FTS5 BM25 over the question's content words and cosine over the vectors, fused by reciprocal rank. A question that names a document stays inside that document: two words of its title, or one when the question is about a document or a pack ("the garden guide"). One title word said in passing no longer pulls the document in; "that file" means the one r message is stored before the search and the search stays inside it. Asked nothing about documents (`strict`), a passage is offered only at cosine >= 0.70, or >= 0.64 with at least half the question's content words in it (thresholds measured on the nomic embedder, where unrelated text scores up to 0.66).
- **A file or folder named by its path is handed over too.** A path typed into a message, or into one of the user's last messages when this one speaks of "the paths", "those folders" or "the project" (`named_paths`, `with_what_it_refers_to`), is looked at on the disk before the model is asked. The turn's context gets what is there (`describe_named`: how many files, of what kinds, in which sub-folders, the newest by name; sized by `cog.named_paths_chars`, and what did not fit is counted). The documents in a named folder are stored the way a dropped file is, newest first, for as long as the memory agent is allowed for its own work; the rest are queued and read behind the reply. Given two folders to study, ELI had answered "that is a string literal to me" and asked the user to run `ls` and paste the output.
- **Scope.** Documents carry the user id of the turn that read them; another API user never sees them. Indexing and removal are written to the evidence ledger.
- **Forgetting.** "forget the X document" lists matching documents and, on "confirm forget documents <ids>", removes them from the index. The file on disk is untouched.
- **Off switch.** `ELI_DOCUMENT_INDEX=0`.

Vector store (`vector_store.py`)

FAISS `IndexFlat`, embeddings from a local nomic embedder (llama.cpp).
Where the embedder runs (`embedder_placement`): on the GPU with the most room, discrete or sharing the machine's memory, when it takes its place before the language model is planned; the model loader asks the store to be `ready()` first, so the plan (`core/load_plan.py`) sees what it holds. Measured on one machine: on a GPU it embeds a memory in about 4 ms for about 130 MB of the card, in RAM in over a second while the processor is busy. In RAM, every part of it (`n_gpu_layers=0`, `offload_kqv` and `op_offload` off where the runtime has them, `embedder_in_ram`): with no GPU, once the language model is loaded, or when a GPU will not take it. With only its layers on the processor, llama.cpp had still put its working buffers on the GPU (54 MB on a CUDA build) after the plan had left the card a sliver, and the next reply aborted.

- `_embed_lock` (RLock) serialises embedding: the embedder is not thread-safe, which is why orchestrator retrieval is sequential.
- Metadata lives in `meta.json` (migrated from a legacy `meta.pkl`).

- **Saves are batched.** Each save rewrites `index.faiss` and `meta.json` whole (18 MB at 2,900 memories), so `add()` saves every `SAVE_EVERY` adds on a background thread and shutdown saves the rest (`flush(pending_only=True)`). `store_memory` used to save after every row: a 349-PDF folder wrote 24 GB in 50 minutes and stalled the whole machine on disk. At start the index adds whatever memories it is missing (an unclean exit), or rebuilds when it is empty or its entries carry no memory ids.
- Singleton via `get_vector_store()`; shutdown-aware; `reset_vector_store()` for rebuilds.
- **Tombstones:** `mark_memory_deleted(id)` writes to `meta.tombstones.json` next to the metadata; search skips tombstoned rows without a rebuild; `compact_tombstones(live_ids)` reclaims space.
- `store_memory()` returns `vector_indexed: bool`, a failed index write logs at warning level, and `fire_memory_uncertainty_event()` drives the World tab's `memory_uncertainty` bar.
- Tombstoning is a per-call-site discipline: there is no shared `delete_memory_row()` helper, so any direct `DELETE FROM memories` must tombstone the vector itself (`consolidate_memories` and `profile_extractor._scrub_onboarding_snapshot` do).

Knowledge graph (`knowledge_graph.py`)

`kg_entities(name, type, aliases, description, confidence)` and `kg_relations(subject_id, predicate, object_id, weight, source)`: a subject-predicate-object graph. FTS5 over entities with insert, update and delete triggers for fuzzy `search_entities`. Only what the user said feeds it (`feeds_graph`, the memory policy's evidence about the user): the extractor reads text as the user's own words, so document and tool text made "User prefers screening". `rebuild_from_memories` is the one rebuild, used by the self-upgrade and settings buttons.
`upsert_entity`; `context_for_prompt` gives lightweight SQLite-only prompt context with no embedding. Relations an older extractor stored that are not facts (an is_a that is a mood or an activity, a has_name that is not a name; `_junk_name_relation`) are left out from both sides, and an entity that had nothing else is left out with them. A stop-word list stops common words becoming entities. first learned: adding it again raises its weight and leaves the date alone.

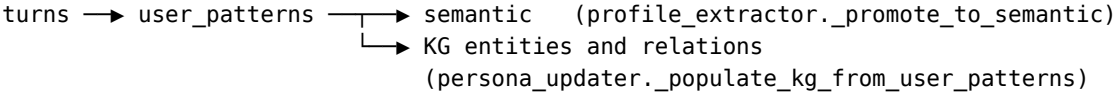
Truth layer (`memory_truth.py`)

`inspect_sqlite` and `inspect_vector_store`: read-only inspection used by status surfaces; reads `vectors/meta.json` and falls back to the legacy pickle. Backs `truth_report` and the memory-status surfaces; `format_memory_truth` is the short answer to "how is your memory doing?" (`MEMORY_STATUS` with `memory_scope=health`).

A recall reply (`MEMORY_RECALL`) puts each memory's date in front of it (`turn_stamp`), so "when was that?" has an answer on the screen instead of one the model makes up.

Promotion across tiers

Promotion is a fan-out from `user_patterns`:



`_promote_to_semantic` applies a durable/transient prefix filter, dedupes against `semantic` and fires only when `_insert_user_pattern` actually inserted, so reaffirmations do not re-promote. Corroboration counts days the owner reaffirmed a fact, not extraction passes, and a passing mention cannot overwrite a fact the owner stated and reaffirmed (`_insert_user_pattern` weighs provenance and corroboration; each supersession is recorded in `belief_revisions`, and repeats are deduplicated). `store_episodic` and `store_semantic` are

thin aliases and are not the promotion path.

Measuring it

`tools/eval/memory_bench.py` runs 11 deterministic scenarios against a temporary database, with no model: extraction, multi-session recall, a relative and an explicit time window, knowledge update, abstention, provenance, deletion, no destructive merge, no self-reinforcement, and chatter not becoming durable knowledge. They follow the LongMemEval ability categories but are written for ELI; they are not the official dataset. `tests/test_memory_bench.py` runs them, so a memory change that regresses one fails the suite.

Honest assessment

- **Strong:** genuinely hybrid (vector, FTS5, graph) on one local SQLite foundation; a measurable storage policy with an audit trail; recall that filters by date before ranking; noise filtering that keeps ELI's own reflections out of recall; embedder serialisation that avoids the segfault.
- **Weak:**
  1. `memory.py` is a 6k-line class spanning unrelated concerns (semantic, conversation, habits, learning, failures, capabilities, upkeep). It wants splitting along those seams.
  2. **Schema sprawl.** `memories` and a legacy `memory`, `conversations` and `conversation_turns`, plus a standalone `semantic` table; inline column detection compensates for schema churn.
  3. **Narrow intake.** Promotion works, but it is fed by regex extraction plus one LLM pass per session, so `user_patterns`, the semantic tier and the knowledge graph stay small relative to the conversation history. Widening the extractor is the work.
  4. **Retrieval cannot detect nonsense.** Similarity bands for genuine and gibberish queries overlap, so no absolute similarity floor separates them; the relative cutoff in `vector_store` tightens the candidate pool but is not a relevance oracle.
  5. **No shared delete-and-tombstone helper** (above).

`eli/cognition/reranker.py`'s heuristic reranker weighs overlap 0.45, importance 0.20, weight 0.15, recency 0.10. Importance and recency alone (0.45 combined) could match the overlap term's full weight, so a memory saved with "please remember this" (importance near 1.0) could outrank a memory that actually matches a vague query, on nothing but being generally important and recent — proven against a live database, where a stored test code beat memories that were actually about the asked-for week. The importance/weight/recency term is now gated: full credit only above a real overlap floor (0.2), a reduced share when a retriever found the candidate at all but overlap is weak or incidental (a single shared word in a long memory no longer counts as "on topic"), and the smallest share when neither applies. That gate had a blind spot: a date-range question ("what have we discussed the past 7 days") is answered largely by `memories_between()`, a separate lookup that matches by timestamp, not by the keyword/semantic retrievers, so its rows carry neither a lexical-overlap score nor the "a retriever found this" channel markers the gate checked for — it read that as "nobody vouched for this" and crushed those rows to a fifth of their score, exactly the rows a "past week" question needs most. Being inside the exact window asked about is now its own relevance signal, gated to full credit like a real overlap match. `tests/test_reranker_importance_does_not_override_relevance.py` pins both.

Recent-history window

The user-facing tunable `cog.mem_recent_turns` (Settings ▸ Cognition; default 24, maximum 80) controls how many recent turns enter the prompt. `RECENT_HISTORY_CAP` no longer exists (the old memory-evidence module was removed); the prompt assembler and `context_budget` do the budgeting, and a recall question keeps a floor of a third of the window for memory context.

The window holds this conversation only (`evidence_format.this_conversation`): turns older than
12 hours, or on the far side of a 3-hour silence, are left out of the dialogue block. Earlier
days still reach the prompt, dated, through retrieval and the period log. Without this a
two-day-old argument sat in the prompt as if it were still going on.

ELI Perception — Vision, Voice, Wake Word, Tone, OS Control

`eli/perception/` (~20 files). ELI's senses and hands: local vision, speech-to-text,
text-to-speech, a **self-trained wake-word detector**, a **voice-profile/tone**
subsystem, and OS control. All local, no APIs, no third-party accounts.

Files

File	LOC	Role
`mic_resolver.py`	~580	**cross-platform mic auto-resolve** — ranked PortAudio candidates, isolated live pr
`audio_stt.py`	~1.6k	STT + mic capture + ducking + adaptive pause + wake/voice capture
`wakeword.py`	~450	**self-trained, music-robust wake-word detector** (openWakeWord features + custom head)
`voice_profile.py`	~420	**prosody + labelled-emotion** (tone/question detection)
`tts_router.py`	1219	Piper/pyttsx3/espeak router + `char:`/`clone:`/`natural:` voice resolution; `neural_
`voice_fx.py`	252	**voice styles** (calm / robotic / energetic / synthetic / refined) —
base voice + ffmpeg effect chain; user preset store
`tts_xtts.py`	394	**voice cloning** from a reference sample (Coqui XTTS-v2). Zero-
shot: `add_clone()` only registers a reference clip, conditioning happens at synthesis. Bundled in the Linux Appl
loudly, not silently, which was a live fault
`vision.py`	698
`os_controller.py`	572
`screen_locator.py`	497	locate UI elements on screen (AT-SPI → local VL-ground → OCR)
`screen_analysis.py`	~180	depth modes, prior-screen memory, research-context prompts
`ui_ground.py`	~320	local GGUF precision click + computer-use agent (no HTTP)
`desktop_capabilities.py`	~170	cross-OS input/screenshot/locate probe; feeds the awareness briefing and SE
`media_deps.py`	~140	bundled-python + PATH discovery for yt-dlp/mpv/desktop CLIs
`gaze_engine.py`	358	webcam gaze/eye tracking daemon — see below
`log_rotation.py`	224	log housekeeping
`analyze_pdfs/image/mesh/csv.py`	~600	file-type analysers
`ambient_vision.py`	238	periodic screen glances (off by default)
`local_whisper_stt.py`	335
`voice_worker_streaming.py`	small	streaming voice worker

Vision (`vision.py` + `ambient_vision.py` + `analyze_image.py`)

The working local-vision stack (see memory `eli-image-analysis`):
- VL model (default Qwen2.5-VL, configurable) loaded via
  `Qwen25VLChatHandler`; **CPU clip forced** by monkeypatching
  `mtmd_cpp.mtmd_init_from_file(use_gpu=False)` — the GPU mtmd clip path
  segfaults on compute-7.5 cards.
- **Hot-swap**: unload text model → load VL → infer → restore in `finally`,
  holding `gguf_inference._LLM_CALL_LOCK`.
- **Co-resident fast path**: a small model (Moondream2 Q4) loaded first, with the
  text model's ctx capped (`vision_coresident_text_ctx`) so both fit 8GB;
  `prefer_fast=True` uses it without a swap.
- Images downscaled to `vision_max_image_px` (1280) to avoid context overflow;
  `repeat_penalty`/`top_k`/`top_p` set to kill a repetition loop.
- `ambient_vision.py`: optional periodic screen glances (OFF by default). A
  guarded daemon re-reads the toggle/interval each cycle and **skips a glance

  whenever the shared LLM lock is busy** (so it never steals the model
  mid-reply). Stores a short description as memory for rolling awareness.

Gaze tracking (`gaze_engine.py`)

A background thread: opens the webcam, runs real-time gaze estimation (MediaPipe iris landmarks, with a Haar-cascade fallback), applies a ridge-regression calibration model if a calibration file exists, smooths the output through a "OneEuroLikeFilter" (the 1€ Filter — a real HCI signal-smoothing technique built for exactly this: low-latency, jitter-free tracking of noisy human input), and writes `latest_gaze.json` every ~100ms. With no calibration it still runs, producing raw face-centre coordinates (useful for "is someone present" detection). Feeds a **dwell-click accessibility feature** — rest your gaze on a target for ~1s to click it, no mouse or voice needed (`GAZE_ENABLE`'s `dwell` argument). Settings key `gaze_engine_enabled`.

Screen locate & click (`screen_locator.py` + `ui_ground.py`)

Three-strategy chain for “find/click X on screen” — all local:

1. **AT-SPI** (Linux) — real widget tree: role, bounds, invoke action. Best for
   labelled buttons and accessible apps.
2. **Local VL-ground** (optional) — `ui_ground_backend=auto|local_gguf` runs a
   local GGUF (Phi-Ground / OS-Atlas / UGround naming in `models/`) or the
   primary VL with a JSON box prompt. For icon-only or non-text UI.
3. **OCR** — Tesseract word boxes + fuzzy match; click centre coordinate.

Computer-use agent (`computer_use_backend=auto|local_gguf`) — optional UI-TARS-style GGUF in `models/` for multi-step screenshot→action loops. Off by default.

Configure on GUI **Screen tab** or `settings.json`: `ui_ground_backend`, `computer_use_backend`, `screen_analysis_depth` (`quick`|`standard`|`deep`).

Deep screen analysis (`screen_analysis.py` + `SCREEN_READ_ANALYZE`)

Voice examples that route to a real screenshot + VL + OCR (+ text-model fusion):

- “What's on my screen?” — standard depth
- “Analyze my screen in depth / exactly what's on my screen” — deep audit
- “How does this relate to my research?” — pulls local memory + KG context
- “Do you remember seeing this in previous sessions?” — prior `screen_awareness` glances

No network. Missing OS tools (mpv, tesseract, ydotool) are offered via `grounded_remediation` **after** a failed attempt — never as a pre-install gate.

Cross-OS desktop input (`desktop_capabilities.py` + `platform_compat.py`)

OS	Mouse/keyboard	Screenshot
Linux X11	xdotool → PyAutoGUI	scrot
Linux Wayland	ydotool+ydotoold → xdotool (XWayland) → PyAutoGUI	grim+slurp
Windows	PyAutoGUI	Pillow ImageGrab
macOS	PyAutoGUI (+ Accessibility permission)	screencapture
Android/Termux	intents only	N/A

See `docs/CROSS_PLATFORM.md` for the full matrix and install commands per OS.

Speech-to-text (`audio_stt.py`, `local_whisper_stt.py`, `mic_resolver.py`)

Microphone selection (`mic_resolver.py`, v2.3.44). On every OS, ELI builds a ranked list of PortAudio inputs — USB and headset/Bluetooth (AirPods, Turtle Beach Chat, Trust USB, …) before built-in, webcam, and virtual loopback routes — and probes each in an isolated subprocess at the device's native sample rate until one delivers live audio. Linux additionally tries PipeWire/Pulse sources with a process-local `PULSE_SOURCE` pin after direct hardware. Honours `ELI_MIC_DEVICE_INDEX` (explicit override) and `ELI_MIC_AUTORESOLVE=0` (OS default only). Diagnostic: `python -m eli.tools.mic_diag`.

A large, feature-dense STT module:
- Mic capture + Whisper transcription (`local_whisper_stt`).
- **Output ducking** — lowers system sink volume while listening (`_eli_duck_output`/`_eli_restore_output` via `wpctl`) so ELI doesn't hear itself.
- **Echo/self-hearing suppression** (`_eli_echo_like_assistant_output`, `_eli_media_probably_audible`). The voice (`tts_router._run_tts`, every engine) holds the speaking lock for the whole reply, holding its process id; the mic treats it as stale only when that process has ended (`speaking_lock_held`). Taken as stale after 30 seconds, a minute-long reply was heard and transcribed half-way through.
- **Voice commands while a reply is coming**: a short one said after the wake word waits for the reply to finish (`_send_waiting_voice`) instead of being dropped; a long one still interrupts.
- **Decoder hints and doubt** (`local_whisper_stt`): the wake words and the apps on the machine the person has named in conversation go to faster-whisper as `hotwords` ("open spotify" in a noisy room was heard as "old thing spot employee"). A command whose every segment is below the decoder's own failed-decode bar is not carried out: the window opens again and the status bar says it was not clear (`heard_clearly`).
- **Playback** opens at the output device's own rate, the voice resampled to it, instead of failing first on a device that takes only its own rate.
- Transcript cleanup: `_cleanup`, `_collapse_repeated_phrase`, `_eli_fast_command_alias` (maps spoken phrases to commands), `_is_safe_direct`/`_allow_direct_chat` (gates which utterances bypass to chat).
- ALSA stderr suppression to keep the console clean.
- **Duration-adaptive end-of-phrase pause** (`_listen_adaptive_pause`, 2026-06-08) — stock `sr.listen()` reads `pause_threshold` once per phrase, so one value can't be both snappy for commands and tolerant of long dictation. A faithful copy of sr's capture with a single dynamic condition: short commands finalise after `ELI_STT_SHORT_PAUSE` (0.5s) of silence; a prompt speaking past `ELI_STT_LONG_AFTER` (12s) needs `ELI_STT_LONG_PAUSE` (2s), so a mid-sentence pause no longer cuts it. Flag-gated (`ELI_STT_ADAPTIVE_PAUSE`) with fallback to stock `listen()`.
- **Generic mic-capture script** — one mechanism (no second microphone) drives both wake-word enrollment and voice/emotion training: a list of `{prompt, sink}` steps, each captured (past the TTS-echo guards) clip routed to its sink, the next cue spoken, a `done` callback after the last. `begin_capture_script` / `begin_wake_enrollment` / `begin_voice_training`.
- **Acoustic wake hook** — when unarmed and a wake model is trained, the captured audio is scored by `wakeword` and, if it fires, the wake word is injected so the existing `VoiceGate` arms — catching the wake word even when whisper transcribed the music. **Per-turn tone hook** — on a real command, `voice_profile.classify_tone` is run and published on a side-channel for cognition (below).

Wake word (`wakeword.py`, 2026-06-08) — self-trained, robust over music

Transcription-based wake matching can't hear "computer" over loud music (whisper transcribes the music). Instead ELI **trains its own** detector, 100% locally:
- **Positives**: the wake phrase synthesised by ELI's OWN **Piper TTS** across several voices and speeds.
- **Augmentation (the robustness)**: each positive is mixed with noise/music at random SNRs, so the classifier learns to spot the wake word *through* a music bed.
- **Features**: openWakeWord's open, bundled melspectrogram→embedding extractor (no account, no download barrier).
- **Custom head**: a small torch classifier ELI owns; `train_model()` → `models/wakeword/eli_wake_head.pt` (gitignored). `WakeDetector.score_audio/is_wake` slide a 1.5s window. Validated: clean wake = 1.00, wake+loud music (3 dB) = 1.00, hard-negative/music-only = 0.00.
- **User-settable phrase** — `get/set_wake_phrases` persists any phrase ("change the wake word to athena"); it feeds both this model and the transcription matcher.
- **Personalisation** — enrolled real-mic clips of the user are folded in as heavily-weighted positives.
- Actions: `WAKE_SET` / `WAKE_TRAIN` / `WAKE_ENROLL`. Fully fallback-safe (`ELI_WAKE_ACOUSTIC=0`; no model → the transcription matcher).

Voice profile + tone (`voice_profile.py`, 2026-06-08) — foundation for emotion

Deliberately SEPARATE from the wake word — this is *how* the user speaks, the basis for tone/emotion and question-vs-statement:
- **Prosody (real, numpy only)**: per-clip autocorrelation **F0/pitch** track, energy, voiced ratio, speaking rate, and a **terminal-pitch slope** (`analyze_prosody`).
- **Question vs statement** (working): rising terminal pitch ⇒ question (`question_or_statement`).
- **Labelled emotion**: `add_labelled_sample` + a nearest-centroid classifier (`train_emotion_classifier`) over neutral/happy/angry/excited/sad — robust for the small data a quick enrollment yields, upgradeable to a NN without touching callers.
- **`classify_tone`** returns emotion + confidence, arousal, and the question/statement cue. `build_profile` learns the user's baseline so tone is scored *relative* to how they normally sound.
- **`TRAIN_VOICE`** runs one unified guided session (wake reps + the same line said in each emotion); modules stay separate, the user does one flow.
- **Wired into cognition**: STT publishes the per-turn read on a fresh, timestamped side-channel (`set_last_tone`); `engine._build_enhanced_system` reads `get_last_tone` and prepends a speaker-voice cue so ELI adapts its delivery (warmer if upset, brisk if energetic). `ELI_VOICE_TONE=0` disables. Complementary to the text-based `cognition/tone_analyzer` (persona preferences), not a duplicate.

Tone / emotion adaptor (`cognition/tone_adaptor.py` + `cognition/emotion_palette.py`)

Decides the emotion ELI **expresses** and drives three output channels at once.

- **Two identifiers** (the "2 systems"): *acoustic* (`voice_profile.get_last_tone` — the user's voice this turn) and *semantic* (`detect_text_emotion` — the content of what they said). `_fuse()` combines them (agreement boosts confidence; voice slightly favoured).
- **Empathetic response policy**, not mimicry: a distressed user → `tender`, heat → `calm`, playful → `playful`. An explicit override (`set_tone("be comedic"/"talk street")` via `SET_TONE`/`CLEAR_TONE`) wins outright and persists.
- **Extensible palette** (`emotion_palette.py`): 20 built-in tones (comedic, professional, deadpan, street_smart, gremlin, tender, …), each carrying a persona directive, Piper prosody profile, and avatar expression; users add their own in `config/voices/tones.json`.
- **Three output channels**: text (a directive folded into `_build_enhanced_system`), voice

  (`tts_router._piper_prosody_args` overlays the tone's pace/expressiveness/pauses **and `_apply_tone_pitch` shifts the actual audio pitch** so sad sounds lower, ecstatic higher — applied on the Piper, natural-neural and clone paths), and face (`world/avatar/persona_mapper` + the animated `gui/widgets/eli_face.py` take the expression).
- **Core personality is never overwritten** — the directive explicitly shades *how* ELI delivers, not *who* he is (emergent-voice principle). `ELI_TONE_ADAPT=0` disables.

Animated face liveness (`gui/widgets/eli_face.py` + `cognition/expression_state.py`)

The procedural face reacts in real time to what ELI is *doing*, via three cheap flags in `expression_state` (priority: thinking > speaking > tone):
- **Lip-sync** — the mouth animates while TTS plays (`tts_router._run_tts` sets the `speaking` flag; amplitude-driven when available, else a natural talk oscillation).
- **Thinking look** — a focused/reflective face while the model generates (the GUI sets `thinking` around `is_generating`).
- Otherwise it shows the current expressed tone. Pure-Qt, embedded in the world panel.

Emotional memory + proactive check-in (`cognition/emotion_timeline.py`, v2.1.31)

`tone_adaptor` shades the *current* reply and then forgets: the acoustic read lives in a 12-second slot, the semantic read is recomputed from the last utterance. That makes ELI **reactive**. The emotion timeline is the durable record that makes it **proactive**.

- **Persistence** — every fused read (neutral included, so the baseline isn't built only from bad turns) is appended to `emotion_events` in `user.sqlite3`, with the utterance that produced it, the arousal, and **the action ELI ran on the preceding turn**.
- **`assess()`** returns EVIDENCE, never a phrase: the sustained run and its dominant emotion, the `neutral→negative` transition, the ELI action at the turn the mood turned, and whether it is *unusual for this user* — measured against their own baseline, so a naturally reserved person is never read as upset.
- **Gates** (all must pass before ELI raises anything): a run of N reads rather than a single spike, per-read confidence above a floor, a credible baseline, and a cooldown so ELI cannot nag. `assess()` reports its `reason` either way, so a *non*-check-in is explainable too.
- **Two surfacing paths.** In-conversation, `evidence_block()` is folded into the system prompt and **the model decides whether and how to raise it** — the wording is ELI's, never a template (see [ELI No Hard-Coded Responses]). Unprompted, the proactive daemon runs the same assessment on its tick, LLM-synthesises an opener from the evidence, and pushes an `emotion_checkin` onto the suggestion queue (+ a pending proposal so a plain "yes" routes). With no resident model the daemon emits **nothing** rather than a canned line.
- **`trend_line()`** runs every turn regardless, giving ELI continuity on how the conversation has been feeling.
- **User control** — Settings ▸ Cognition ▸ *Emotional awareness*: master toggle, exchanges before a mood counts, minimum confidence, check-in cooldown, recent-mood window, baseline history. All read at call time (no restart), all defaulting to the shipped constants. A *Tone of voice* dropdown in the same tab pins the register ELI answers in (Auto + the 20 palette tones) via `tone_adaptor.set_tone`. Kill switch: `ELI_EMOTION_MEMORY=0`.

Text-to-speech (`tts_router.py`)

Multi-backend router with graceful fallback: **XTTS-v2 natural neural** (opt-in) → **Piper** (packaged binary under `tts_piper/` or `models/tts/piper`) → **pyttsx3** → **espeak-ng / espeak**. Resolves voices from several candidate dirs so a packaged build or a source checkout both work.

Natural neural voice (`natural:<id>`, `tts_xtts.py`). XTTS-v2 driven by its **built-in studio speakers — no clone/reference needed** — is the human-like, expressive, punctuation-aware path. `list_natural_voices()`/`synthesize_natural_wav()` are cheap to probe (never load the ~1.8GB model just to enumerate) and **fall back to Piper** when the neural extra isn't installed, so a `natural:` voice never goes silent. It is selectable at the **startup model picker / first-boot wizard** ("Voice engine: Piper vs Natural") — the choice is persisted *before* hardware tuning so its VRAM is a deliberate up-front decision (`ELI_XTTS_DEVICE` pins the device); low-VRAM boxes keep fast Piper. Enable with `pip install -e ".[natural]"` (from the ELI project root).

Expressive Piper prosody. Piper at bare defaults sounds flat and clips full stops. The router now passes tuned `--sentence_silence` / `--length_scale` / `--noise_scale` / `--noise_w` (`_piper_prosody_args()`, env-overridable, `ELI_TTS_PROSODY=0` to disable) so `?`/`!`/`.` land as intonation and pauses — the "Amy sounds flat" fix.

No robotic voices. `list_voices()` hides the espeak `sys:` voices and Piper `low`/`x_low` tiers by default (they sound synthetic), surfacing them only as a genuine last resort when the box has nothing better, or when `ELI_TTS_ALLOW_ROBOTIC=1`.

Voice library (`runtime/voice_assets.py`) — the shipped pack is a starting point, not the limit. The module fetches the upstream Piper index (**166 voices across 45 languages, 38 of them English** — US/GB, Scottish, Northern and Southern English) and exposes it through `list_available_voices()` with per-voice presence, download size and licence, cached on disk so the picker still works offline. `download_voice()` streams from the index's exact paths and **verifies the published md5** before the file is put in place, so a truncated or proxy-mangled download is rejected rather than surfacing later as a corrupt-model crash. The GUI path is Settings > VOICE / TTS > *Get more voices / accents…* (`gui/widgets/voice_downloader.py`); every voice dropdown repopulates on install.

Redistribution policy. `RESTRICTED_VOICE_NAMES` (ryan — CC-BY-NC-SA; lessac — Blizzard/Lessac, not cleared; cori — pending) are never put in a release asset. The rule is keyed on the **voice name, so every quality variant is covered**, and `scripts/asset_release_policy.py` imports it so the build and the runtime cannot drift. The user may still download them for personal use; the dialog states this rather than hiding the voice.

Config integrity. A `.onnx` without its `.onnx.json` cannot be loaded by Piper, so it is invisible to `list_voices()` while still occupying ~60 MB. `incomplete_voices()` finds these and `repair_voice_configs()` fetches just the missing ~5 KB config. `tests/test_voice_library.py` guards against shipping the broken state again.

Style presets (`voice_fx.py`, `char:calm`, `char:robotic`, …) are a redistributable Piper base voice plus ar
matched `fallback` chain** against installed voices only. **Voice cloning** (`tts_xtts.py`, `clone:<name>`) rep
v2 — an opt-in extra (`eli-v2.0[clone]`) that falls back to a normal voice when the model isn't installed rather t

OS control (`os_controller.py`)

Platform-aware (Linux/Windows/macOS) screenshot, volume, keyboard, clipboard. `take_screenshot(region)` picks the right tool per platform (`gnome-screenshot`, PIL `ImageGrab`, platform CLIs); fails gracefully where a capability isn't available (e.g. area screenshots on some Windows installs). `screen_locator.py` finds UI targets for click/automation.

File analysers

`analyze_pdfs`, `analyze_image`, `analyze_csv` — typed handlers behind the `ANALYZE_*` actions, feeding extracted content (text/OCR/structure) into the grounded pipeline.

Several folders or files named in one message all go to one job (`paths_named`, `_route_several_paths`); a path that is missing is reported; with nothing named and no work in hand ELI asks which, never the current folder. Asked to analyse, a folder is read in full, in pieces; asked to summarise, from the opening of each document, and the report says which. A summary that comes back empty is a failed job: the documents are indexed and nothing has been analysed.

A folder (`ANALYZE_PDF_FOLDER`, `eli/runtime/folder_analysis.py`) is read through the document index: every PDF is stored whole there, one summary covers all of them (in parts of what one call has always read, then joined), each call is told how many words its reply holds (the reply limit times the model's measured characters per token, over the characters per word of what it reads), a reply cut at its limit still ends at its last finished line and drops a heading left with nothing under it, and the result, with every document listed by its path in the folder and any unreadable files, is saved as a PDF (`eli/runtime/rich_document.py`: matplotlib typesets the formulas, Qt lays out the page, .docx where there is no window). The same rendering lays out chat replies and task results, so `$\phi$` reads as φ.

Formulas match the text: each is drawn at the size whose x stands as tall as the text font's x (Computer Modern's x is 0.44 of its size, a sans font's about 0.54, so at the same size formulas looked a size smaller), and `put_html` stands each on the text's baseline with Qt's `AlignBaseline`, sizing the image format's font so its descent is the depth the formula was drawn with. Qt's HTML reader can only centre an image in the line box, which moves with whatever else the line holds, or stand its lower edge on the baseline, which lifts subscripts and fractions. A fraction standing in a displayed formula is full size (`\dfrac`), as in TeX. The PDF is laid out at the resolution text is measured in and drawn page by page in black: `QTextDocument.print_` re-lays an unpaged document out in printer units, where an image's baseline is still measured in screen pixels, and drawing with the window's palette printed ELI's dark-theme text near white. .docx pictures are lowered by their depth (`w:position`). It used to file each piece of every PDF as a personal memory and summarise from the first 17 documents.

Honest assessment

- **Strong:** this is the layer that makes ELI genuinely multimodal and local — eyes (VL + OCR + ambient), ears (Whisper + ducking), mouth (Piper/espeak), hands (cross-platform OS control). The vision co-residence + hot-swap + CPU-clip workaround is real engineering against hard 8GB-GPU constraints.
- **Weak / watch:**
  1. **Vision latency** — the hot-swap unloads/reloads the text model per glance; even the co-resident path runs CPU clip (~3.5s). Acceptable, not fast.
  2. `audio_stt.py` is a **2.5k-line** module mixing capture, ducking, echo suppression, command aliasing, and cleanup — wants splitting.
  3. **Residual "7B" comment** in `ambient_vision.py` ("the 7B vision model hot-swaps…") — cosmetic, but inconsistent with the model-agnostic line; should read "the vision model". (Flagged for a future scrub.)
  4. `gaze_engine.py` is experimental and off by default — fine, but it's carried weight.
  5. Platform tools are detected at call time; a machine missing `gnome-screenshot`/`wpctl`/`piper` degrades silently to fallbacks (good) but there's no single "what perception capabilities do I actually have here?" probe surfaced to the user.

Update — 2026-06-09 (VRAM-aware STT; drag-drop keeps the full path)
- **STT is VRAM-aware** (`local_whisper_stt`). faster-whisper defaults to GPU only when the card is big enough for whisper AND a typical main model (≥12 GB total VRAM), else CPU — so on an 8 GB card whisper no longer preloads ~2 GB and starves the main model (observed: free_vram 4083 MB → gpu_layers=11; after the fix free_vram 7092 MB → gpu_layers=99). `small.en` int8 on CPU is ~1–2 s for a short command. Explicit `ELI_WHISPER_DEVICE` still wins; `ELI_WHISPER_GPU_MIN_MB` tunes the threshold.
  Superseded in 2.5.18: no size threshold. Whisper goes on CUDA device 0 only when that card still holds its measured footprint after the language model's load plan (see `inference_and_hardware.md`, STT VRAM awareness); `ELI_WHISPER_GPU_MIN_MB` is gone.
- **Drag-and-drop keeps the FULL path.** A file dropped into the chat input now expands to `[File: <full path>]` followed by the inlined content (was content + basename only), so "fix/examine/edit this file" can resolve the file on disk while "summarise this" still has the content. PDFs keep their path too; combined with the FIX_FILE last-file memory, a bare follow-up "fix it" recovers the file named a turn earlier.

ELI Grounding & Evidence Layer

> **Updated for v2.4.84.** Quick mode still returns verbatim for deterministic
> introspection; all CHAT modes now pass through the gradient orchestrator first.

The anti-confabulation system — a deterministic evidence scaffold wrapped around the probabilistic model. This is ELI's most distinctive subsystem and the part closest to genuinely frontier. Spans `eli/runtime/` and `eli/cognition/`.

The core idea

For "grounded" actions (status, runtime, memory, identity, control), ELI does **not** trust the LLM to state facts. It (1) gathers deterministic evidence from the live system, (2) can answer directly from that evidence without the LLM at all, and (3) when the LLM does generate, validates the output against the evidence and rejects/repairs contradictions. The LLM becomes a phraser, not a source.

> **Correction (2026-06-08): the "bypass" is PARTIAL and MODE-GATED — not a blanket
> bypass.** Earlier wording here and in `what_eli_is.md` ("bypasses the language model
> entirely", "without the LLM at all") overstated it. The running reality, verified
> against the engine: the deterministic verbatim return fires for a **subset** of
> actions and is **mode-gated** —
> - a small **`_verbatim_always_actions`** set (deep introspection like
>   `EXPLAIN_MEMORY_RUNTIME`) returns verbatim in **every** reasoning mode;
> - **`_deterministic_direct_payload_actions`** (status/reports + router-fast/command
>   actions: DATE/TIME, VOLUME, WRITE_NOTE, window/media/file ops, NEWS/MORNING_REPORT,
>   self-report …) return **verbatim in *quick* mode** and **synthesise in non-quick**
>   modes (`engine._bypass_persona` + `_is_grounded_control_nonquick`);
> - everything else is the model's to phrase.
>
> So the model IS in the loop for most conversational turns; the "phraser, not a source"
> principle holds where the deterministic path actually fires, not universally. A
> 2026-06-08 fix corrected the membership: the command actions were in the wrong set, so
> quick mode re-synthesised them, corrupting results ("Wrote note"→"Bought note") and
> adding latency. They are now verbatim in quick mode, synthesised in others.

Components

`runtime/deterministic_grounding_gate.py` (3.4k lines)
The deterministic renderer. `render_action(action, args, user_input, mode_label)`
produces a grounded answer for control/status actions directly from runtime data
(settings, runtime snapshot, DB counts, GPU line) — bypassing the model.
`install(CognitiveEngine)` wires it into the engine. `_eli_v14_runtime_data()`
assembles the live config block (model_path, n_ctx, gpu_layers, …).

> **Structure.** `render_action` is one explicit ordered pipeline: each layer is a function
> that takes the layer below it as an argument, composed by `_stack(...)` into
> `_RENDER_PIPELINE`. An action no layer handles returns a JSON surface
> `missing_deterministic_renderer` rather than a made-up answer. The layer order is pinned
> by a characterisation test that compares the rendered output of every audited action.
>
> **`_eli_last_response_confidence_v2()`** (called for "how confident are you in
> your last answer") used to return a fixed template string regardless of what
> the last response actually was — a confident-sounding self-assessment that
> inspected nothing. It now reuses `eli.runtime.last_trace.load_last_trace()` +
> `control_contracts._trace_text()`, the same real per-turn trace mechanism
> already used correctly for "what was your last message," and says plainly
> when no trace is available rather than fabricating an answer.

`runtime/control_contracts.py` (1,242 lines)
The deterministic control path:
- `is_control_action` / `route_control_text` — recognise control/status intents.
- `build_control_evidence(engine, action, args, …)` — gather the evidence packet
  (runtime paths, DB state, bus result, trace).
- **`output_violates_evidence(text, evidence_text)`** — the gate that returns
  True when LLM output contradicts/omits the evidence; the engine uses this to
  reject a hallucinated answer. Seven concrete runtime terms (gpu layers,
  batch size, context size, cpu threads, model path, user/agent database)
  used to be blanket-exempted from this check, because evidence renders them
  under a machine-style key (`n_gpu_layers`, `model_path`...) while prose uses
  the human phrase — a plain substring match would reject a correct answer
  over spelling alone. That exemption also let the model state those exact
  values with *no* evidence backing at all. `_CONCRETE_TERM_ALIASES` now maps
  each term to every real spelling it appears under, closing the hole without
  reintroducing the false positives.
- `compact_evidence_answer` / `finalise_control_result` — assemble the final
  grounded response.

Synthesis is validated against its evidence
`cognition/output_governor.validate_against_evidence` also rejects any figure or model designation
(`H100`, `512 GB`, `32K`) that the evidence does not contain, allowing for unit conversion; the
non-Quick compact synthesis path uses it and falls back to the deterministic evidence text.
`contracts/grounded_control.py` owns which actions never fall back to a clarifying question.

`runtime/evidence_ledger.py` (732 lines)
A persistent SQLite ledger of evidence events: `record_event`, `recent_events`,
`actions_between` (what ELI executed in a period, per user, for the period log),
`repeated_event_signals` (detect recurring issues over N days), `status_evidence`,
`artifact_snapshot`. Gives ELI a durable, queryable record of what actually
happened.

`runtime/evidence_arbitration.py` (212 lines)

`EvidenceItem` + `arbitrate_evidence(limit)` + `build_evidence_context_text` —
scores and merges competing evidence into a single context block. Pairs with the
agent-bus confidence aggregation (`_score_tool_result`), which scores a tool
result with no recorded `"ok"` field as unverified (same low score as a
confirmed failure) rather than defaulting a missing outcome to success.

Retrieval provenance
Memory hits reach the prompt through the shared turn retrieval in `memory/retrieval.py`. Each
recalled line carries its event date and status (`memory_provenance.format_grounding_memory_line`)
and the orchestrator adds a retrieval diagnostics block (what was searched, how many items,
which time window) so ELI can say truthfully what its search did.

`cognition/output_governor.py` (1,694 lines)
Post-generation governor: `govern_output(text, is_grounded)`,
`normalize_assistant_text`, `validate_against_evidence`, plus a family of
drift-repair functions — `strip_generic_ai_identity_drift` (kills "As an AI
language model…"), `repair_local_persona_drift`, `repair_self_user_confusion`
(fixes the model conflating itself with the user), `clean_response_style`. The
last line of defence before text reaches the user.

`cognition/grounded_status.py` (648 lines)
`direct_grounded_answer(user_text)` — fully deterministic answers for identity /
memory-inventory / status questions, assembled from the DBs
(`format_user_identity`, `format_memory_inventory`, table distributions). No LLM.

`runtime/grounded_remediation.py` (1.7k lines)
The failure→repair loop. When an action fails (app won't open, path missing,
browser/IDE absent), it: `diagnose_app/path/browser/ide` → `build_repair_plan`
→ `offer_for_result` ("want me to install X?") → `handle_confirmation` (consumes
yes/no) → `execute_pending_plan`. Stateful pending-repair tracking +
`explain_last_failure`. This is what makes failures conversational and
recoverable instead of dead ends. An install is offered only when a package source on the
machine has the thing (`build_install_candidates` asks each one). When none has it the reply
says so and, for a short name, asks whether to open its website instead: "open nasa" used to
end "Would you like me to download/install it?", and a yes ran the same lookups again.

Evidence-completeness gating (`contracts/grounded_control.py`)

A prior gap in this exact layer: `GROUNDED_CONTROL_ACTIONS` existed but was never checked before
a direct/verbatim return or a no-clarify decision — only bare membership (`is_grounded_control_action`)
gated anything, so a grounded action with genuinely incomplete evidence (a binary file decoded as
text, an empty result) could still short-circuit straight to the user. `evidence_complete_for_action(action,
local_vars)` closes that: it's checked at both live call sites —
before the direct-payload verbatim return, and before the no-clarify-suppression decision — and on
`False` the turn falls through to the next tier instead of returning early, rather than refusing outright.

- `GROUNDED_CONTROL_ACTIONS` (51 actions) is the union of `control_contracts.CONTROL_ACTIONS` and a
  new `FILE_EVIDENCE_ACTIONS` (10 actions: `SUMMARIZE_FILE`, `READ_FILE`, `ANALYZE_PDF[_FOLDER]`,
  `ANALYZE_IMAGE`, `OCR_IMAGE`, `TRANSCRIBE`, `CONVERT_DOCUMENT`, `FILE_AUDIT`,
  `SCREEN_READ_ANALYZE`) — the family implicated in a real bug this closes: a `.docx` read as raw
  binary and confabulated into a fake analysis, because nothing checked whether the "evidence" was
  actually decodable text before treating it as complete.
- `_looks_like_file_evidence_complete` runs a printable-ratio + non-empty + no-error-marker check
  on file-family evidence — this is what would have caught the binary-as-text bug directly. A
  minimum-length floor (20 chars) also treats a bare dict-key-name leak (`_collect_text` including

  Mapping keys, not just values) as still-incomplete rather than a false "complete."
- `evidence_gap_reason(action, local_vars)` returns `"complete"`, `"missing"` or `"corrupted"` — a
  typed reason, not just a bool, for callers that want to say *why* evidence fell short.
- Kill switch: `ELI_EVIDENCE_GATE_DISABLE=1` bypasses the check at both call sites, since widening
  `GROUNDED_CONTROL_ACTIONS` from the old hand-maintained ~10-action list to the full ~51-action
  union means the no-clarify gate now applies far more broadly than before — this lets a
  false-refusal in the field be killed instantly without a revert.
- The generic fallback (non-file, non-runtime-status actions) stays weak — satisfied by dict keys
  existing at all, not real content — an acknowledged, separate gap, not part of this pass.

`tests/test_grounded_control.py` covers it, including a direct repro of the original binary-garbage
bug (asserts `evidence_complete_for_action` now returns `False` on it).

How it fits the pipeline

1. Router classifies action. Control/grounded actions enter the deterministic
   path.
2. `build_control_evidence` gathers facts.
3. PHASE45 deterministic bypass: for many status actions `render_action` /
   `direct_grounded_answer` answer **without** the LLM.
4. If the LLM generates, `output_violates_evidence` + `validate_against_evidence`
   gate it; `govern_output` cleans drift.
5. `grounded_remediation` handles action failures with an offer/confirm loop.
6. `evidence_ledger` records the event for future `repeated_event_signals`.

Honest assessment

- **Strong (genuinely):** very few local-LLM projects build a deterministic
  evidence layer at all, let alone one this thorough — direct-answer bypass,
  output-vs-evidence rejection, drift repair, a persistent ledger, and a
  conversational remediation loop. This is the subsystem that most justifies the
  "frontier" label.
- **Weak:**
  1. The gate file is still 3.4k lines with many version-named helpers (`_eli_v9_*`, `_eli_v14_*`);
     the layers are ordered explicitly now, but the helpers wait to be folded into fewer, named modules.
  2. **Overlapping surfaces** — `grounded_status`, `control_contracts`,
     `deterministic_grounding_gate`, and the `runtime/*_response` / `*_surface`
     modules all render grounded answers with partial overlap. The boundaries
     between "who renders the final grounded string" are blurry.
  3. Confidence/grounding is heuristic (additive score + threshold +
     `output_violates_evidence`), not a formal proof — fine, but it's a gate, not
     a guarantee.

ELI's account of its own behaviour (`cognition/self_claims.py`, 792 lines)

Routing, retrieval and escalation run before the model is called and never reach it, so asked "why did you search the web?" a model can only invent. Live, a 35B model blamed "vector search drift", offered a `verbose_media_logging` setting and an `action_logs` table that do not exist, and said "the audit trail is now active" for logging that was always on.

- **The turn record.** Each turn notes what the pipeline did (`request_context.note_turn_fact`):

the rule that routed it, a downgrade to chat, what retrieval found, whether it escalated to the web and why, every action that ran (for a message of several commands, the words of each step, the action it became and the rule that matched it), how often the model ran, on how many prompt characters and for how many seconds. `describe_turn` writes that into the turn's signed audit row (`orchestrator_audit_ledger`), so the row answers "why did you do that" later.

- **Asked about itself, ELI is handed the record.** For a question or complaint about its own behaviour (`asks_about_own_behaviour`), the recent audit rows, each paired with what the user asked, go beside the recent exchange just before the message, where the overflow trim cannot cut them (in the middle of the system prompt they were cut). The same holds for an insult aimed at ELI, a correction ("I did not ask about X", "I asked why ..."), and a bare reaction with no subject of its own ("What the fuck is going on!?", "seriously??", `reacts_to_last_reply`); none of these goes to the intent model. Direct questions about the last turn ("why did that take so long", "what did you just do", "why did you search the web"), and a remark that it was slow ("that took over a minute??"), are answered from those rows with no model call (`ROUTING_FAULT_EXPLAIN`); asked about time, the last turn comes first with where the model runs. The correction shortcut never replaces such an answer, or any other action that answers or acts directly. None of these turns is ever escalated to a web search.
- **A question about something already said is about the conversation.** "When was that, in that conversation?", "your long response above", "the response from you" (`correction_patterns.refers_to_earlier_reply`) are never web-searched; the history block then gives every earlier reply of ELI's whole, and a correction that asks one (`that is not true, what day is that from?`) leaves the correction shortcut, which only sees the last exchange, for the full pipeline.
- **The recent exchange keeps what a follow-up is about.** It is built newest first within its share of the context window (`session_continuity.exchange_budget_chars`): ELI's last reply is whole, a fetched result (a news briefing, a search) stays whole for three replies and is labelled "result of NEWS_FETCH, not recalled", and older turns are clipped. When a prompt is still too long, the oldest lines go, never the message or the newest turns (`fit_exchange_prompt`). Before 2.5.15 every reply was cut to 280 characters and an oversized prompt was cut from the front: a follow-up on a story past the cut reached a model that had never seen it, and it called a real Guardian story its own invention.
- **A correction is answered, not re-run.** When a correction or complaint about the last reply routes to the very action that produced it, it goes to chat with the record and that reply whole ("I did not ask about RAM" ran the RAM report again, word for word).
- **The claim check** (`gate_stream` on streamed replies, `drop_invented_self_claims` in `govern_output`). A sentence is dropped when it reports a change nothing made ("logging is now active", when no action ran); promises a standing change a reply cannot make ("no web search from now on"); names a setting, table, module or file ELI does not have (checked against the real actions, settings, tunables, tables, modules and the filesystem); proposes a fix ELI has no action for; or, on a fault question, explains a cause the record does not carry (most of the sentence's content has to be written in the audit rows or the period log); or names a maker other than the one the shipped licence names (`license_info.made_by`, also given to the model as WHO MADE YOU). A sentence whose specifics (a file, a figure) are all in the live facts (the loaded model file, the load as the loader reported it) states what is true and is kept: "I am running on <the loaded model>" is not a claim to have done something. A sentence that rests on a dropped one ("That is why...", "Want me to run that now?") goes with it, and a heading left with nothing under it is removed. ELI's own earlier replies are never counted as evidence. Code blocks are never touched.
- **A reply that fails is said again.** When too little is left, or it said it did what was asked and had not, the model gets one more go from what actually happened (`gate_stream(rewrite=)`, `engine._say_in_own_words`), and the new text has to pass the same check. The record's own line is for when no model is loaded or the second reply fails too. Hedges go through the same wording.
- **What a message brings** (`cognition/supplied_in_message.py`). A question that holds its own facts skips the web and the intent model; text quoted or called material is never read for orders; sums and date offsets are worked out and handed to the model (`live_reads.live_facts`), which is not a calculator.

- **What backs a claim** (`_Check._backed`). An action that ran backs a sentence only if it changes something, shares a word with the sentence, or the sentence is about reading. A memory lookup does not back "I am starting the job". Also dropped: saying it cannot read what it can (`_DENIES_A_READ`), saying it is changing its own code when nothing did (`_CHANGING_ITSELF`).
- **Turns that went wrong without an error** (`self_improvement.note_turn_anomalies`). A rejected reply, an action turned into chat, a dropped claim and an unkept promise go on the failure record, so self-analysis sees them; every one of those turns reads "ok" in the ledger.
- **A reply cannot say it did something when nothing ran.** On a turn in which no action was carried out, a sentence is dropped when it reports an act on the machine: "I've opened ...", "Spotify is opening now", "Scanning `/path` ...", "(Waiting for system execution output...)", "Project `X` initialized." The verbs are the ones a command starts with, less the work a reply does in its own words ("I've summarised it below" is the reply doing what it says). A lone "Done." is judged by what follows it: before another such report, or with nothing after it, it goes; before something the reply wrote, it stays. When the user had asked ELI to do something, the reply then ends "Nothing was carried out in this turn."; under a reply to a greeting or a question the sentence just goes. When the user's own commands did run in the turn (a message of commands and talk), their results are in front of the model and it may refer to them. A fix said to have been made "so it won't happen again" is dropped on any turn.
- **Saying what ELI just did is a question about it.** "then you just stopped spotify", "you just maximised your GUI, not the browser", "what just happened?" are held to the record like "why did you ...": the last is answered from the ledger with no model call.
- **A day in a reply has to be a day with records.** For a question about a period, a sentence that puts something on a day with nothing recorded is dropped. "this morning", "earlier today", "yesterday" and "last night" are days too, and the message being answered does not count as a record of its own day: asked about the past week, ELI had put Thursday's songs down to "this morning" in the day's first conversation.
- **The self-model says so too.** The live self-model line states that logging is always on (every action in the evidence ledger, every turn in the audit chain) and is not something a reply can change.
- **An insult is not a stance.** A complaint, insult or correction aimed at ELI is never recorded as a position ELI holds ("it is 2026 you fucking idiot" was stored as a stance on "2026 idiot").
- **ELI's diagnoses are not the user's facts.** A session summary's "current work" is stored as the user's project only if the user's own words carry it; text they quote back from ELI does not count, and work that is about ELI itself is not stored.
- **Words written on an earlier day keep their day.** Session summaries, recalled memories and earlier sessions' lines reach the prompt dated, and their own "last night", "yesterday", "today" and "right now" become that day's dates (`evidence_format.as_of`, `dated`). A 15 October summary said "up late last night"; on 20 October ELI told the user they had been up late last night.
- **What the hardware is doing is measured, not said.** In a turn with no evidence, a sentence that puts a state (idle, unused, gathering dust, busy, a percentage, hot) on a part ELI can measure (GPU, card, VRAM, CPU, RAM, the cards' own model numbers) is dropped (`output_governor._hardware_state_claim`): "your RTX 2060 Supers are sitting at idle" was said while a model filled both cards.
- **A promise that did not run says so.** When follow-through decides ELI's own "I am going to check…" is not to be run, the reply ends with a line saying nothing ran for it.

Behaviours that keep answers grounded

- **Admin and persona actions are explicit only.** The LLM intent resolver cannot choose `SET_USER_NAME`, persona locks, tone and voice changes, memory deletion, plugin or MCP changes, self-patching, shell or training actions (`llm_intent._EXPLICIT_ONLY_ACTIONS`); only a router rule matching the user's words can. A guessed `PERSONA_LOCK_CLEAR` once ran before the turn was downgraded to chat.
- **A name is set only by a declaration.** One extractor

(`identity_validation.extract_explicit_identity_facts`) is shared by the router, the knowledge graph, working memory and the executor; hypotheticals and negations ("if my name is listed as unknown...", "my name is not listed") set nothing.
- **Self-patch only patches what it can ground.** `generate_code_patch` extracts the target file from the error traceback; when a failure has no in-project file (e.g. an HTTP/connection error, "No commands to run"), it used to ask the model anyway, which **invented** a path (phantom `api_client.py`) that then failed to apply. It now returns `no_groundable_file` and routes the failure to goal-autogenesis / self-heal instead; when a file IS grounded it is **pinned** in the prompt so the model can't drift to a different path.
- **Banter guard:** the `llm_intent` resolver occasionally maps playful input to a generative action; the engine now downgrades GENERATE_SCRIPT/PROJECT/DOCUMENT/CODE_SOLVE to CHAT when the text carries no create-intent keyword (real "write me a script" is preserved).
- **Compact grounded synthesis carries a VOICE primer** (pulled live from the canonical persona) so factual/introspection answers sound like ELI without losing the EXACT-FACTS contract that pins every number/path/table/DB to the evidence.
- **Rules only where they apply.** The grounding and conversation rules live in `kernel/prompt_rules.py`; `select_rules` puts each guard in the prompt only on a turn it can apply to (self-mechanism rules when the user asks about ELI, the dates rule when dated evidence is present). The whole block used to precede every prompt, 6,800 characters in front of "morning eli", and a small model answered the rules instead of the user. `all_rules()` is the full text, used if selection fails.
- **The clock and the calendar are facts in the prompt.** `evidence_format.time_facts` works out every clock time the user's message names against now ("7.30pm = 19:30 today, 10 h 24 min from now"), and `agenda.prompt_line` lists what is on the calendar for the next 36 hours. A question about either is never escalated to a web search (`grounding_escalation`: `asks_date_arithmetic` or `agenda.about_agenda`).
- **A compound time question is not answered with the clock alone.** "what time is it and how long until the presentation?" goes to the model with those facts instead of to `TIME`.
- **The profile holds what the user said about themselves.** `profile_extractor.purge_software_talk` runs at start and removes profile rows that are ELI's own troubleshooting stored as facts about the user ("currently frustrated with ELI's memory retrieval"), then rebuilds the user model from what is left.
- **A report about ELI answers a question about ELI.** "help me with my essay" is a request for help, not for the capability list (`HELP` needs a bare "help" or a question about commands). "what is a knowledge graph" and "how does memory work in the brain" are questions about the world, not about ELI's memory (`EXPLAIN_MEMORY_RUNTIME` needs ELI as the subject or an unmistakable reference to its own stores). "how do i improve my memory" is about the user's own. A duration ("in 20 minutes") is not a complaint about speed.
- **A word on its own is not the user's name.** A lone capitalised word set the name ("Dublin", "Tuesday"). It does so only when ELI's last message asked what to call them.
- **A day says what a request is about, not when to do it.** "plan a study session for tonight" and "check the weather for tomorrow" are answered now. A background job needs a clock time, a delay, "overnight", a named report, or a word that asks for it later (`_EXPLICIT_WHEN_RX`, `_DEFER_RX`); with one, "build me a script at 2am" and "run the test suite tonight" are queued instead of run on the spot.
- **Asked about tomorrow, the weather answer is tomorrow's.** `GET_WEATHER` returns the forecast for the day named, not the current conditions; with no place it asks "Where?" once and remembers the answer for the session.

Deterministic reports are surfaced verbatim; examiner correctness
- **Verbatim guard for deterministic grounded reports.** `_get_chat_response` now returns the EXAMINE_CODE tiered report and the FILE_AUDIT file-count VERBATIM when they appear in the evidence, and surfaces the FIX_FILE success event as a real outcome — instead of letting the quick chat path re-narrate them. This killed three live confabulation classes: EXAMINE_CODE inventing "findings" that were actually existing code comments; FILE_AUDIT inventing

  files-with-bugs that don't exist; FIX_FILE narrating "I cannot fix" after it had already written the file.
- **Examiner correctness (`code_examiner.py`).** Tier-2 now drops pyflakes star-import noise ("may be undefined, or defined from star imports" — ~800 false positives on the GUI → 0). Tier-3 reviews the WHOLE file in line-correct windows (was only the first ~MAX_FILE_CHARS, so god-files were ~90% unreviewed), each source line prefixed with its REAL number, and rejects any finding citing a line outside the shown window (kills the "engine.py:132 undefined 'ov'" hallucination class — line 132 is `or result.get("error")`).
- **Routing:** "is there any issues with the files?" / "check the code for bugs" now route to EXAMINE_CODE (the real examiner), not FILE_AUDIT; fix-intent ("fix the bugs in foo.py") stays FIX_FILE.

News topic deepen (grounded, not guessed)

After a `NEWS_FETCH` briefing, a follow-up like **“go deeper into the transformer story”** must not route to open `CHAT` and let the model invent details. The engine (`_try_news_topic_deepen_reroute` in `engine.py`) detects `extract_deepen_topic()` within a short window of the last news command and **re-routes to `NEWS_FETCH` with that topic** — same live fetch path as the original briefing.

The router's general follow-up nudges (`eli.followup.*`, "go deeper", "tell me more") do not count as explicit commands here, so they never outrank this rule: "go deeper on the sea wall vote" after a briefing went to chat until 2.5.15.

If the user complains that ELI guessed (`is_news_fetch_complaint`), the kernel recovers the deepen topic from recent conversation (`recover_recent_deepen_topic`) and runs `NEWS_FETCH` again instead of falling through to web-escalation hedge.

Challenging a grounded answer must re-check it, not confabulate a new one

After a grounded action (`RUNTIME_AUDIT`, `RUNTIME_STATUS`, `GPU_STATUS`, `MEMORY_STATUS`, and ~20 other status/diagnostic actions), a detail or challenge follow-up must rebind to the *same* action for a fresh evidence pull, not fall through to open `CHAT` — that lets the model free-associate over whatever it remembers saying, and a skeptical push ("you sure about that?") can make it retract a correct answer and invent a wrong one instead (`_eli_phase19_rebind_grounded_followup` in `engine.py`). Two ways this can silently not fire:

- the follow-up's wording doesn't match `_ELI_PHASE19_CHALLENGE_FOLLOWUP_RX` / `..._DETAIL_...RX` ("and you think the maths checks out??" wasn't covered until it was reproduced live — GPU_STATUS correctly explained requested-vs-effective GPU layers, then confabulated a different, wrong breakdown when challenged in exactly this phrasing);
- the prior action isn't in `_ELI_PHASE19_GROUNDED_FOLLOWUP_ACTIONS` — `GPU_STATUS` and most of its sibling status/diagnostic actions were missing from this set entirely, despite publishing `grounded=True, evidence_used=True` through the identical `_publish_last_response_meta` call site as the actions that were already covered.

`tests/test_phase19_grounded_followup_truth_lock.py` pins both the regex and the action set against the live failure.

Widened 2026-09-29 for file-backed evidence, not just status/diagnostic actions: a real transcript showed `SUMMARIZE_FILE` correctly reading a document once, then two follow-up complaints ("that's not what I asked" / "you're making this up") producing fabricated

analysis — invented citations, fake quoted "answers" — because the rebind didn't cover `FILE_EVIDENCE_ACTIONS` (`SUMMARIZE_FILE`, `READ_FILE`, `ANALYZE_PDF[_FOLDER]`, and 7 more). `_ELI_PHASE19_GROUNDED_FOLLOWUP_ACTIONS` now unions with `FILE_EVIDENCE_ACTIONS` (imported, not hand-duplicated) and the rebind also recovers the prior action's `args` (`path`/`file`/`folder`/`url`) from `_publish_last_response_meta`, so it knows *what to re-examine*, not just *that* to re-examine — re-reading the same file rather than guessing from memory of what it said. The challenge/detail regexes were widened too (`"you did n't really..."`, `"that's not what I asked"`, `"more of a X than..."`, `"you're lying"`, `"making stuff up"`).

History is not discarded before the budgeter sees it

The old memory-evidence module capped recent-history pulls silently; it was removed and shared turn retrieval (`memory/retrieval.py`) replaced it. The user-facing setting `cog.mem_recent_turns` (default 24, maximum 80) and `cognition/context_budget.py` now decide how much history and memory enter the prompt, and a recall question keeps a floor of a third of the window for memory context.

Retrieved evidence can't be cut away by an oversized-prompt truncation

Getting evidence into `_build_enhanced_system`'s SITUATION BRIEF / CONVERSATION HISTORY blocks is not enough — if the assembled system prompt then overflows the model's context window, whatever truncates it back down to size has to know where the evidence lives, or it can cut the one thing the whole grounding pipeline exists to protect.

`_get_chat_response`'s hard prompt-size guard used to slice the oversized prompt by raw character position: keep the first half, keep the last half, drop the middle. Its own comment claimed this kept "the grounded-evidence tail (appended last)" — wrong. `_build_enhanced_system` actually writes SITUATION BRIEF and CONVERSATION HISTORY between the persona head and a closing block of generic RESPONSE DISCIPLINE rules, so on a heavily-retrieved turn (a large brief plus a large history) the blind slice could land past the evidence block on both ends, keeping only persona boilerplate and rules with zero retrieved memory inside them. A chain-of-thought turn hit this live: retrieval genuinely found real memory (11 items in the time window, 36 turns, 4575 chars assembled), but the truncated prompt handed to the model contained none of it, and the final answer truthfully — from what it was shown — denied any relevant memory existed.

`_truncate_system_prompt_preserving_evidence` (`engine.py`) fixes this by finding the SITUATION BRIEF / CONVERSATION HISTORY span first and keeping it whole; only the persona/rules head and the closing-rules tail around it get shrunk, and evidence itself is only trimmed (head+tail, within itself) as a last resort. It's wired into both truncation sites in `_get_chat_response` — the broker path and the no-broker direct-GGUF path — which is the only place any of the four algorithmic reasoning modes (chain_of_thought, tree_of_thoughts, constitutional_ai, self_consistency) generate text; `quick` mode's separate streaming path (`_trim_enhanced_system_for_stream`) already trimmed history in place rather than blindly, so it was never exposed to this. See `tests/test_evidence_survives_prompt_truncation.py`.

Blueprint — Code-Mode Execution Layer for ELI (gap analysis)

> **Updated for v2.4.84.** Restricted exec gated on Full Control; `api.call()` facade.

Status: draft. Date: 2026-06-11.
REVISED after auditing the codebase: an earlier version of this doc proposed building machinery ELI already has. This version is a GAP ANALYSIS — what exists, and the small genuine delta — not a greenfield design. Original ELI work throughout; the only outside idea is the general principle of deterministic gating, which ELI already implements its own way.

BUILT (2026-06-11)

Core implemented, gated, and tested (17 tests, `tests/test_code_mode.py`):
- **Restricted exec (the one new piece):** `eli/coding/restricted_exec.py` — Full-Control-gated (`is_full_control()`, the GUI toggle), AST-whitelist (10 escape classes rejected: import/eval/open/getattr/dunder-attr/`__import__`/def/lambda/with/exec), restricted namespace.
- **Facade (sugar):** `eli/tools/api.py` — `api.call("ACTION", **args)` + helpers, proxying to `execute()`; `api.actions()` for discovery.
- **Runner:** `eli/coding/code_mode.py` — generate→gate+validate+run→retry loop (injectable `generate`, testable without a model).
- **Lane redirect:** `CODE_SOLVE` with `args.code_mode=True` runs in-process against `eli.api` (fail-closed: refuses unless Full Control is ON).
- **#4 retrieval:** confirmed ABSENT (no top-K capability retriever); v1 passes a bounded `api.actions()` list — a semantic retriever is a future refinement, not required.

Remaining (deferred, behaviour-policy): the router auto-selecting code-mode for multi-step requests under Full Control — needs live validation, so it's flag-reachable for now.

---

0. Correction up front

"Code-mode" (the model writes a small program against a curated API instead of emitting MCP-style tool-calls) is the token-frugal, composable, offline-friendly direction the field is moving to — and the right fit for ELI's 8 GB / small-context reality. **But ELI already has ~85% of the substrate.** This doc exists to name the ~15% that's actually new, so we build only that.

1. What ELI ALREADY HAS (audited, with files)

Capability the blueprint needs	Already in ELI	File
Code-agent loop: plan → implement → verify → run → retry	**`CodeAgent.solve()`** — "orchestrator that composes ELI's coding capabilities"	`eli/coding/agent.py`
Planner / implementer separation	`plan_task()`, `implement()`	`eli/coding/planner.py`
Candidate verification + **auto-synthesised tests**	`verify_candidate()`, `synthesize_tests()`	`eli/codin
Tree search over candidates	`tree_search()`	`eli/coding/search.py`
Learn from failed attempts	`BugMemory`	`eli/coding/bug_memory.py`
DAG decomposition of a task	`solve_dag()`	`eli/coding/plan_graph.py`
Bounded sandbox execution (CPU limit, scrubbed env, kill-switch)	`run_code()`	`eli/coding/sandbox.py`
Deterministic risk gate (action classes → approved/blocked/pending-confirm)	`approval_engine.evaluate_
Fail-closed command allowlist + destructive-pattern block (`rm -rf /`, `mkfs`, `dd`…)	`security` + `_BLO
Authority / autonomy posture	`authority_gate`, `operator_policy` (proposal_only/supervised/…)	`eli/runtim
AST validate-and-retry of generated code	`code_examiner` (used live 2026-06-11)	`eli/runtime/code_examine
Capability **registry** (register/get/list)	`capability_registry`	`eli/tools/registry/capability_registr
Capability **doc/digest generator** from the manifest	`generate_capabilities_doc()`	`eli/tools/registry/c
Offline enforcement / crisis guard	`netguard`, `crisis_guard`	`eli/core/`
Reasoning-native planning (model thinks, then emits)	working post-`e7ab0b9`	`eli/cognition/gguf_inference

Conclusion: ELI is not missing "a code agent," "a sandbox," or "a safety gate." It has all three, plus verification, search, bug-memory, a risk engine, a command allowlist, and a capability registry. The CodeAgent already does CodeAct-style plan→code→verify→sandbox→retry.

2. The genuine DELTA (the only new work)

The existing `CodeAgent` solves **coding tasks** — produce a function/script as a *deliverable*. Code-mode-as-assistant-execution needs it pointed at a different target: the model writes code that **calls ELI's own capabilities** to fulfil a *conversational request*, and the reply is synthesised from the execution result. Each candidate delta was audited IN FULL (2026-06-11); statuses below are grounded with file:line evidence.

1. **`eli.api` facade — MOSTLY EXISTS (only typed sugar is new).** The callable dispatch already exists: `execute(action, args)` at `executor_enhanced.py:12507` runs any of the 230 manifest actions by name. Generated code can already call `execute("SUMMARIZE_FILE", {...})`. The *only* delta is a typed, discoverable wrapper (`eli.api.files.summarize(path)`) generated from the registry — convenience + discoverability over an existing surface, not new execution machinery.
2. **In-process AST-whitelist + restricted exec — GENUINELY NEW (the one real piece).** `sandbox.run_code()` (`coding/sandbox.py:129`) is **process-isolated** (subprocess, scrubbed env, CPU limit) and runs *standalone* code — it deliberately cannot reach ELI's live state (the loaded model, memory stores). Code-mode must run **in-process** to call live `eli.api`/`execute()`, so it **cannot reuse that sandbox**. So an in-process restricted-exec is required: an AST pass allowing only `eli.api.*`/`execute(...)` + safe builtins, run in a restricted-globals namespace. It **composes** existing parts — the `code_examiner` AST walker + `approval_engine` action-classes for the GREEN/AMBER/RED decision — but the in-process restricted-exec itself is new. This is THE work.
3. **Routing lane to the code agent — ALREADY EXISTS as `CODE_SOLVE`.** Router lane (`router_enhanced.py:3194`, `_CODE_SOLVE_RE`) + executor handler (`executor_enhanced.py:10009`) → `eli.coding.solve` (plan→search→verify→repair, quick/thorough, DAG, background). Reuse it. The only variant is targeting `eli.api` for assistant-actions rather than producing a standalone script, + grounded synthesis of the result (reuses the `540f514` no-confabulation work). Do NOT build a new lane.
4. **Per-query capability retrieval — UNCONFIRMED; parts exist, no retriever found.** There is a manifest (`capability_inventory.generated.json`), a digest generator (`capabilities_doc.generate_capabilities_doc()`), `capability_sync`, and a FAISS store — but a top-K *capability retriever* was NOT located in the audit. Verify before building; if absent, it's a small assembly from the manifest + existing vector store, not new infra.

Net after full audit: #3 exists, #1 is sugar over an existing dispatch, #4 is unconfirmed (parts present), and only **#2 (the in-process restricted-exec) is genuinely new** — and it composes existing AST + gate machinery. The real build is: one in-process AST-whitelist + the typed facade, then point the existing `CODE_SOLVE` path at `eli.api`.

3. Phased plan (composing what exists)

- **Phase 0 — the one new piece: in-process restricted-exec (~3–5 days).** AST whitelist (reuse `code_examiner`'s walker) allowing only `eli.api.*`/`execute(...)` + safe builtins; run in a restricted-globals namespace IN-PROCESS (not `sandbox.run_code()`, which is process-isolated and can't see live state); classify each call via `approval_engine` action classes for the GREEN/AMBER/RED decision. This is the genuinely new safety surface.
- **Phase 1 — facade + point CODE_SOLVE at it (~2–3 days).** Generate the typed `eli.api` wrapper from the registry over `execute()`. Add a CODE_SOLVE *variant* (or arg) that targets `eli.api` for assistant-actions and feeds the result into grounded synthesis — reusing the existing lane, agent, quick/thorough modes, and background handling. No new routing lane, no new agent.
- **Phase 2 — retrieval (if needed) + harden (~few days).** Add top-K capability retrieval ONLY if the audit confirms it's absent; add code-mode cases to the eval harness; add the

```
  live integration test the suite is missing.

Total honest estimate: **~1–1.5 weeks**, and the bulk of it is the single new piece (the
in-process restricted-exec). The agent loop, sandbox, risk gate, verification, bug-memory,
registry, AND the `CODE_SOLVE` routing lane are already yours.

## 4. Verified in full (2026-06-11)

Audited against the project directory, not assumed:
- **Callable dispatch surface?** YES — `execute(action, args)` at `executor_enhanced.py:12507`.
  Generated code can already run any of the 230 manifest actions by name.
- **Routing lane to the code agent?** YES — `CODE_SOLVE` (`router_enhanced.py:3194` +
  `executor_enhanced.py:10009` → `eli.coding.solve`). Already plan→search→verify→repair,
  quick/thorough, DAG, background.
- **Sandbox: namespace-restricted or process-only?** PROCESS-only (`coding/sandbox.py:129`:
  subprocess + scrubbed env + CPU limit), for *standalone* code. **Cannot host live-state
  code-mode** → the in-process AST-whitelist (§2.2) is genuinely required.
- **Risk gate to compose with?** YES — `approval_engine.evaluate_record()` (action classes →
  approved/blocked/pending-confirm) + the command allowlist. Map `eli.api` calls onto it.
- **AST walker to reuse?** YES — `code_examiner` (used live this session).
- **Existing `eli.api`-style typed facade under `eli/tools/`/`eli/plugins/`?** NOT found —
  the registry stores metadata, not a callable typed surface. This (small) piece is new.
- **Top-K capability retriever?** NOT found — manifest + digest generator + FAISS exist, but
  no retriever. Verify once more at build time; if truly absent, assemble from existing parts.

## 5. What stays uniquely ELI

Voice, vision, gaze, hard-offline, emergent persona, and the entire coding-agent +
gate stack you already built — untouched. This is wiring ELI's own parts into a new lane,
not importing anyone's design.

# ELI Coding Agent (`eli/coding/`)

> **Updated for v2.4.84.** Invoked via `CODE_SOLVE` / `GENERATE_SCRIPT`; separate
> from the 15-agent chat bus (its own pipeline: plan → search → verify → repair).

A self-contained, **additive** subsystem that lifts ELI's code generation,
analysis, and repair to frontier-grade. It touches nothing that already works:
it lives in its own package and is reached through a new `CODE_SOLVE` action.
The LLM is an *injected* `generate` callable (default = the local inference
broker), so the whole pipeline is model-agnostic and unit-testable without a
model.

## The loop

plan_task ──► seed implement ──► synthesize_tests ──► tree_search ──► best candidate (decompose)
(implementer) (gate harness) │ + provenance ▼ beam of diverse roots ──► verify (gate ladder) ──► score
▼ (UCB selection of best imperfect node) refine(feedback+prior) ──► verify ──► score ──► record fix to bug
memory


## Components

| File | Capability |
|---|---|
| `sandbox.py` | **Mandatory execution-feedback substrate.** Runs code in a bounded, isolated subprocess (temp c
```

clock timeout, POSIX `RLIMIT_CPU`; **no `RLIMIT_AS`** — it breaks numpy). Multi-language via a runner map (Pytho
| `bug_memory.py` | **Semantic bug classification + long-term memory.** `classify_bug` maps a failure to a `BugC
handling, type, index/off-by-one, key-missing, name-undefined, import, zero-division, value, assertion, recurs
loop, logic-inversion, state-corruption, concurrency, resource-leak, io) with a stable, fuzzy-
matchable *signature* (exception type + last frame + normalized message). `BugMemory` is a local SQLite store of
then-fuzzy (difflib) recall, so the agent learns from past fixes. |
| `verification.py` | **Explicit verification gating + scoring + test synthesis.** `verify_candidate` runs the l
driven; deterministic smoke-harness fallback) that prints `ELI_TESTS: <passed>/<total>`. |
| `planner.py` | **Structured decomposition — planner/implementer separation.** `plan_task` → an approach + orde
step fallback). `implement` writes/refines code against the plan; in refine mode it does **patch-
based incremental refinement** (fix the specific feedback, keep prior code). |
| `search.py` | **Tree search over solutions (not single-shot).** A diverse beam of roots (temperature ladder = e
exploitation vs exploration. Refinements pull matching prior fixes from bug memory into the feedback; successful
diff fix) pairs. Stops on `target_score` or budget. |
| `agent.py` | **`CodeAgent` / `solve()`** — composes all of the above; default broker-
backed generator; returns best code + full provenance (plan, search trace, score, bug class). |

Capability checklist (as requested)

- structured decomposition (planner/implementer separation) — `planner.py` ✔
- execution feedback loop (mandatory) — `sandbox.py` + `verify_candidate` gate 2 ✔
- tree search over solutions (not single shot) — `search.py` beam + UCB ✔
- explicit verification gating — `verification.py` gate ladder ✔
- long-term memory of bugs and fixes — `bug_memory.py` ✔
- automated test synthesis — `verification.synthesize_tests` ✔
- patch-based incremental refinement — `planner.implement` refine mode + search expansion ✔
- semantic bug classification (null handling, logic inversion, state corruption, …) —
`bug_memory.classify_bug` ✔
- scoring / "U-style" exploration / tree-of-code-solutions — `score_candidate` + UCB beam tree ✔
- multiple languages — `sandbox` runner map (Python deep; bash/js/ts/ruby/go/lua structural) ✔

Wiring & control

- New executor action **`CODE_SOLVE`** (`args.description`, optional `args.language`)
  saves the solution to `artifacts/scripts/` and returns code + provenance. It is
  additive — `GENERATE_SCRIPT` and the self-patch engine are untouched.
- Env knobs: `ELI_CODING_SANDBOX` (default on; 0 disables execution),
  `ELI_CODING_RUN_TIMEOUT` (20s), `ELI_CODING_BEAM` (3), `ELI_CODING_MAX_ITERS` (6).
- Bug memory DB: `<db_dir>/coding_memory.sqlite3`.

Honest scope (what's verified vs model-dependent)

- **Verified deterministically** (tests in `tests/test_coding_agent.py`, no model):
  the sandbox crash/tolerate logic, the bug classifier + memory recall, the gate
  ladder + scoring, and the full plan→search→repair→record loop driven by a stub
  generator (buggy seed → UCB refinement → correct, fix recorded).
- **Model-dependent quality:** the *content* of plans, implementations, and
  synthesized tests scales with the local model. The machinery is frontier-shaped
  regardless of model.
- **Depth:** execution-level verification is Python-first; other languages run if
  their interpreter is installed (structural support).
- **Subtask DAG (added):** decomposable tasks are solved as a dependency graph of
  subtasks on the shared `eli/core/dag` engine (`plan_graph.solve_dag`), composed
  into one module — see `dag.md`. Gated by `ELI_CODING_DAG` (default on); cohesive
  tasks fall back to single-shot.

- **Chat routing (added):** `CODE_SOLVE` is now reachable from chat for explicit intents ("solve this coding problem", "implement X and test it", "use the coding agent", "verified solution"). Generic "write a python function" still routes to the lighter `GENERATE_SCRIPT` (an earlier router path claims it — precedence is tunable but the router was intentionally left unreordered).
- **Not yet:** composition is single-module (ordered concat + import-dedupe), not true multi-file projects; test synthesis verifies against synthesized tests, not a user-supplied spec suite.

FIX_FILE (`executor_enhanced.py`, not this package, but shares its safety net)

`FIX_FILE` — the directly user-triggered "fix/improve this file" action — routes Python fixes through this package's `solve()` (syntax + lint + execution + repo-context + self-critique), then applies the result to an *existing* project file. It now carries the same safety net `self_improvement.py`'s autonomous patcher has: refuses to touch the protected guardrail-file list (`is_protected_patch_path`), smoke-imports the patched module before accepting it, runs targeted tests, and rolls back to the pre-fix backup on either failure — all reusing `self_improvement.py`'s own verification functions rather than a second implementation. Worth stating plainly: since autonomous self-patching is opt-in and off by default, FIX_FILE is almost certainly the *more* exercised of the two real-file-editing paths, so it needed to be at least as protected, not less.

ELI Inference & Hardware Boot

> **Updated for v2.5.21.** GGUF path is canonical (optional Ollama backend still
> exists as a secondary path). What a model loads with is planned by llama.cpp's own
> load simulation on every GPU it sees (`core/load_plan.py`); the prompt format is
> the model's own chat template. Token budgets scale by reasoning mode via
> `reasoning_modes.py`.

How ELI loads a model, talks to it, and adapts to whatever machine it's on. The inference path is model-agnostic (see memory `eli-model-agnostic`); the boot path is hardware-adaptive. Files in `eli/cognition/` and `eli/core/`.

Inference (`cognition/gguf_inference.py`, 3.2k LOC + `inference_broker.py`)

- **Model resolution (`get_model_path`)**: `ELI_GGUF_MODEL_PATH` env → `model_path`/`custom_model_path`/`bundled_model_path`/`gguf_model_path` settings keys. No baked model; empty default.
- **`load_model(force_reload)`**: `load_request()` reads the settings (env first): context (0 = auto), threads (0 = one per physical core), batch, cache type, GPUs to use and their shares, and a GPU-layer count only when it was pinned for this model at this context (`pinned_gpu_layers_for_model`; -1, llama.cpp's own value, is every layer). `load_plan.plan()` turns that into the load (below) and `_build_with_plan` makes it. A load that still fails is planned again for the memory free at that moment, then on the processor (three attempts); a cache type llama.cpp refuses is planned again with the default cache; a GPU pack too old for the model hands over to the bundled runtime. What was asked and what loaded are both written to the runtime snapshot, read from the live model.
- **Chat format from the model** (`cognition/chat_template.py`): every GGUF chat model carries the Jinja template it was trained with (`tokenizer.chat_template`); rendering it gives each model its own format, and rendering it around sentinel text gives its reasoning markers and end-of-turn marker. The family formats in `_format_chat` are kept only for a file with no template or one that will not render. Thinking is switched through the template's own

  `enable_thinking` where it has one.
- **Serialization**: all calls hold `_LLM_CALL_LOCK` (a native RLock from
  `runtime/native_locks`) — llama_cpp is not safe under concurrent calls; this is
  also what vision hot-swap and the ambient daemon coordinate on.
- **The model's state is kept at the end of a prompt's fixed part**: `_format_prompt` notes where
  the part that does not depend on the message ends (the system prompt, or the intent
  resolver's action list and examples). Before a call, `_resume_from_saved_state` gives back the
  state saved there by an earlier call that started the same way, or reads that part alone and
  saves the state (llama.cpp's per-sequence state calls: the model's memory only, no logits).
  The model then reads only what follows. A hybrid model (`qwen35moe`: most layers carry a
  running state, not an attention cache) cannot be rolled back to a shared prefix, so without
  this every call read its whole prompt: measured on Qwen3.6-35B-A3B, an RTX 2060 SUPER and
  experts in RAM, Normal mode's second pass took 63.8 s and took 3.2 s with the state given
  back (the same 60-token answer both ways); the intent resolver went from 26.6 s to 2.5 s
  after its first call. A 7k-token state is 135 MiB, saved in 0.08 s and restored in 0.02 s.
  The states are kept within an eighth of free memory and two contexts' worth of state, the
  oldest giving way; background work uses them but never adds one; they go when the model is
  unloaded. `ELI_PROMPT_STATE_REUSE=0` turns it off.
- **Prefill abort**: shutdown and cancel register `llama_set_abort_callback` so a
  long prompt prefill stops immediately — `StoppingCriteria` alone only runs
  between output tokens.
- **Live control**: `get_live_runtime_override`, `unload_model`, `reload_model`
  let the GUI swap models / change settings without a full restart.
- **`InferenceBroker`** (`inference_broker.py`): the higher-level `infer()` /
  `gguf_ready` abstraction the orchestrator, engine, and ReAct loop call, so
  callers don't touch `gguf_inference` directly.

- **CPU runtime preflight (`core/llama_cpu_compat.py`)**: before the first native load, a cached
  smoke test (once per llama.cpp build and CPU) checks the runtime starts on this CPU. Prebuilt
  wheels assume AVX-VNNI and SIGILL on older chips; the loader now reports that plainly instead of
  crashing. Only a definitive illegal-instruction blocks a load. `ELI_CPU_PREFLIGHT=0` disables it.

Hardware profiling (`core/hardware_profile.py`, ~3200 LOC)

Every GPU, every vendor, every OS, read by one function per source and shared by detection
(`detect_hardware`) and live telemetry (`get_live_gpu_telemetry`, printed by `describe_gpus`):

Source	Cards	Where
`_nvidia_cards`	NVIDIA, per card	`nvidia-smi`, any OS
`_amd_cards`	AMD, discrete and APU	`amdgpu` kernel counters (no ROCm needed), else `rocm-smi`
`_windows_dxgi_adapters`	any vendor	Windows DXGI: memory and the budget Windows gives this process
`_macos_accelerators`	Apple Silicon, AMD on Intel Macs	`ioreg` IOAccelerator
`_linux_drm_gpus`	every display device once, PCI and SoC (msm, asahi)	`/sys/class/drm`
`_llama_cpp_cards`	whatever llama.cpp sees (CUDA, Vulkan, Metal, HIP, SYCL)	fills what a vendor tool leaves

A model uses the discrete cards, or the integrated ones when there are none (llama.cpp's rule,
`_cards_in_use`). Each card's maker comes from its llama.cpp backend or its name (`_vendor_of`).

- **No guessed figures.** A card whose memory no source reports (an NVIDIA driver whose
  `nvidia-smi` fails, Intel Arc on Linux, Windows before 10 for the free figure) keeps its name,
  its memory is left unknown and `gpu_detection_uncertain` is set; `effective_use_gpu_layers`
  still plans for it and llama.cpp measures it before a load. The tables of NVIDIA and Arc
  models with their factory memory, the flat 4 GB, the "keep a fifth for the desktop" share and

  the RAM floors are gone.
- **Unified memory** (Apple Silicon, APUs, iGPUs): the same fraction of available RAM as `cpu_ram_budget_mb()`, with no floor or ceiling of ELI's own.
- Subprocess calls to `nvidia-smi` / `rocm-smi` / `lspci` sanitize `LD_LIBRARY_PATH` first (`_external_tool_env()`): a frozen build points it at its own bundle.
- **Threads**: `recommend_cpu_threads()` = the physical performance cores (measured on an i7-8700: 4.37 tokens/s at 6 threads, 3.99 at 12). The setting's 0 means this rule. A count the old tuner saved (logical threads less one or two) goes back to 0 once (`runtime_settings` revision 1, recorded as `settings_revision`, so a count set afterwards stays); read once per process (`physical_performance_cores` is cached; it was logged on every hardware check).
- **KV cache type** — the settings `cache_type_k` / `cache_type_v` are empty (fp16, the default) or `q8_0`. Any other quantized type stored there reads, sizes and loads as `q8_0` (`core/gguf_sizes.py`: `cache_setting`, `effective_cache_type`): measured on llama.cpp 0.3.35, a `q8_0` cache answered the same as fp16 at the same speed on CUDA, Vulkan and the processor, and a `q4_0` cache answered wrongly. The tuner chooses `q8_0` only when an fp16 cache would not hold the context in the memory left after the weights. llama.cpp is given the type as `type_k` / `type_v` with `flash_attn` (`llama_cache_args`); arguments it does not take are dropped and logged (`model_load_diagnostics.llama_accepts`), a cache it refuses is loaded again as the default with the fit redone, and the type it reports is what the runtime snapshot records.
- **The recommendation is the plan.** `recommend()` for one model asks the load planner (`_recommend_from_plan`), so the startup window shows what the loader will do. The older estimate (`smart_fit_config`, `unified_fit_config`, `_compute_graph_reserve_mb`, `_kv_cache_mb`) is used only where llama.cpp cannot simulate (no runtime, or several candidate models).
- **Fit profiles** — startup exposes **Balanced / Max GPU / Max context** (`fit_priority` in settings, `ELI_FIT_PRIORITY` env): Balanced sheds GPU layers → batch → ctx; Max GPU shrinks ctx/batch before dropping layers; Max context moves layers to the processor first. The planner searches in that order.

Install-time detection is a SEPARATE stack from `hardware_profile`

`hardware_profile` only runs once ELI itself is importable. Getting there — deciding whether to build `llama-cpp-python` with CUDA/ROCm/Vulkan in the first place — happens before that, in two more independent detectors that do not share its code at all: `install.sh` (bash, the actual build) and `eli/setup/hardware_policy.py` (Python, decides `ELI_INSTALL_CPU_ONLY` for `eli_setup.sh`'s GUI-wizard-or-terminal install). A bug fixed in one of the three does nothing for the other two — confirmed in the field, twice:

- **Both install-time detectors misread nvidia-smi's specific NVML "Driver/library version mismatch" failure as a working GPU.** That failure prints a first line matching the usual "broken driver" keyword filters ("Failed to initialize NVML: ...") *and* a second line that matches none of them — "NVML library version: 595.91" — which both `install.sh`'s bash filter and `hardware_policy._nvidia_smi_gpu_names()` were reading as a real GPU product name. Chosing the GPU build plan on a driver that cannot currently build or run CUDA is what sent one field install into a doomed from-source CUDA build that failed partway through — which is what "the GUI wizard did not finish" actually was; the wizard has no build timeout of its own, `install.sh` really did exit non-zero. Fixed by checking nvidia-smi's *exit code* first in both places (reliable on both failure shapes) rather than only pattern-matching its stdout text.
- **`hardware_policy.py` also treated a bare `rocm-smi` binary as a working AMD GPU** — installed as part of a generic driver-utils package with no AMD

  hardware behind it, it still errors ("Driver not initialized (amdgpu not
  found in modules)") the moment it is actually queried. Fixed to require a
  real product-name answer, not just the binary existing.
- `install.sh`'s from-source CUDA build also used to fall back to
  `CMAKE_CUDA_ARCHITECTURES=native` when nvidia-smi's `compute_cap` query
  came back empty — but "native" makes `nvcc` re-probe the GPU through the
  same driver stack nvidia-smi just failed to reach, so it inherits the exact
  same failure. Falls back to the same known-good architecture list the
  CI-built GPU pack uses (`61;75;86;89`, Pascal—Ada) instead.
- **Known gap, not yet closed**: even after these fixes, a portable install
  that correctly lands on `--cpu-only` (driver unverifiable, or a genuinely
  weak/absent GPU) has no path back to GPU acceleration *during* that install
  — `install.sh` and `eli_setup.sh` have no awareness of the CI-built
  `gpu-packs` release at all; only the runtime "Install CUDA/Vulkan GPU pack"
  button in the Startup Model Selection dialog does. `install.sh` now at
  least tells the user that button exists instead of going quiet about GPU
  acceleration entirely. Teaching the portable installer to try the GPU pack
  itself (matching what the frozen/AppImage runtime already does
  automatically) is the real fix and is still open.

GPU pack (`packaging/pyinstaller/eli_gpu_pack.py`, `core/gpu_pack_runtime.py`)

Portable/AppImage builds download a CUDA or Vulkan build of `llama-cpp-python`
per machine (the bundled runtime is CPU-only, safe everywhere) and shadow the
bundled copy via `sys.path`.

- **`gpu_pack_looks_installed()`** is the trust-first check every normal boot
  uses: marker file + backend metadata + native libs present on disk, no
  subprocess, no GPU/driver contact. Only when this can't confirm the pack
  does anything fall through to `gpu_pack_operational()`, which live-probes
  offload in a throwaway subprocess — correct at install time, but a
  transient driver hiccup or busy GPU used to make that live probe fail even
  with a perfectly good pack installed, and any caller treating a failed
  live probe as "not installed" deleted a working pack and re-downloaded it
  on the very next launch. The cheap check is what stops that loop.
- **`activate_gpu_pack_runtime()`** preloads native libs, purges cached
  `llama_cpp`, calls `llama_backend_init()`, and verifies offload in-process
  — so a pack that verified in a subprocess actually works in the AppImage
  GUI process (Intel Iris Xe Vulkan is the case that needed this).
- `try_activate_gpu_pack()` (`gpu_pack_runtime.py`) delegates entirely to
  `activate_gpu_pack_runtime()`'s own (correctly cheap-first) gating rather
  than duplicating a check — a duplicate with reversed operand order here used
  to force the subprocess probe on every single GUI launch regardless of
  whether the marker file already said the pack was good.
- Writes `runtime/gpu/.gpu_pack_ok` + `.gpu_pack.json` (backend, version) on
  success; respects `runtime/.gpu_choice` for CPU opt-out.
- **One source for the runtime**: the bundled `llama-cpp-python` and every pack are built from
  `packaging/llama_source.py`: pinned llama-cpp-python and llama.cpp archives checked against their
  SHA-256, the binding changes a newer llama.cpp needs, and the version in
  `packaging/llama_cpp_constraints.txt` (a label of its own, so installed packs are replaced when it
  moves). A pack of a different build than the llama.cpp already running in the process is left for
  the next start: the two share library names, and two builds in one process freed each other's
  memory.
- **One llama.cpp per build**: a pack is used only at the bundled runtime's exact version
  (`pack_matches_runtime`). When an update moves llama.cpp on, `outdated_pack()` names the old

pack and the first-run chooser (`eli_entry._first_run_gpu_offer`) replaces it with the matching one, same backend, without asking again: the GPU was chosen already. An update that fails keeps that choice and is tried again at the next start (2.5.31 recorded it as `cpu-install-failed` and stayed on the CPU at every start after; that marker beside an older pack is retried too). Every download reports its real size to the progress window (`install(argv, progress)`); the startup window and the first-run wizard share one status reader (`startup._gpu_pack_state`) and offer "Update"; server and helper starts print how to update.
- **Backend** follows the detected vendor (`pack_backend_for`): CUDA on an NVIDIA card, Vulkan on every other GPU.
- **Size on disk**: a wheel cannot hold links, so every soname chain arrives as full copies (`libggml-cuda.so`, `.so.0`, `.so.0.25.3`: three 498 MB files). `_dedupe_wheel_native_trees` links identical names to the versioned file right after unpacking, before the CUDA runtime is fetched, and each wheel is deleted once unpacked: the 0.3.36 CUDA pack takes 1.1 GB instead of 2.0 GB, and the install needs that much less free space (2.5.18 failed on a full disk at the cuBLAS step).

Load plan (`core/load_plan.py`)

llama.cpp can simulate a load (`no_alloc`): it builds the model and a context the way a real load would and, allocating nothing, logs which device each layer and cache goes to and how large every compute buffer is. The planner asks it about the operator's settings and, when they do not fit what each GPU has free now, about smaller ones in the fit profile's order, and keeps the largest that fits every device. It replaced a ladder of guessed configurations, a load probe whose checks timed out (30 of 32 since September) and a one-card fit.

- **Several GPUs**: layers are shared so each card fills the same share of what it holds after its compute buffers and headroom (`tensor_split`); the startup window chooses which cards and, optionally, manual shares (`gpu_devices`, `tensor_split`). `CUDA_DEVICE_ORDER=PCI_BUS_ID` keeps llama.cpp, nvidia-smi and the settings numbering cards the same way. A manual split shares layers, and the experts kept in RAM are the first layers': when that leaves a card with room for another layer's experts, the plan says so and that Auto fills each card (two cards at 50/50: 1.4 GB used of 6 GB free on the first while the experts of 29 layers were in RAM). The status line names every card in use ("RTX 2060 SUPER ×2"), and the tuning panel shows "loaded" only when a model is.
- **Headroom**: what llama.cpp takes on a card after the load as it works through the first prompts, measured on this machine and kept per card and model in `data_dir/runtime/gpu_headroom.json` (`watch_headroom`, `note_headroom`). Until a card is measured the plan keeps llama.cpp's own compute-buffer figure free on it. `ELI_VRAM_RESERVE_MB` is used as given. Measured on two RTX 2060 SUPER: 28-82 MB, where a fixed 700 MB was held back.
- **Mixture-of-experts**: a model that does not fit keeps the experts of its first layers in RAM, as few as the GPUs need (`tensor_buft_overrides`).
- **Processor only**: the largest context up to the target whose weights, cache and buffers fit the RAM budget, from two simulations (the cache grows with the context) and a third to confirm.
- **No simulation** (an older llama.cpp): the same search on the file's own tensor sizes (`core/gguf_sizes.py`) with the buffers estimated, and the plan says so. A file whose tensor table cannot be read is loaded as asked (`as_asked`).

`runtime_load_gap()` (`cognition/context_synthesiser.py`; -1 asked and none loaded is a reduction) only ever checked for `effective < requested` ("reduced," the classic VRAM-shortfall case) — a mixture-of-experts load *raises*
`n_gpu_layers` above the request instead, and that case reported "loaded exactly as requested" for a load that did not match the request at all. It now reports a separate `raised` dict, and every surface that reads it (`GPU_STATUS`, `EXPLAIN_COGNITION_RUNTIME`'s `_inference_runtime_lines`, and `live_runtime_brief()` which grounds casual/phatic replies) states plainly which layers'

attention/shared weights are on the GPU, how much VRAM that is, how many GB of experts sit in RAM, and that the CPU processes those during generation — not "the rest run on CPU" (backwards under MoE: nothing was left off the GPU) and not silence. `live_runtime_brief()` returning "" for a MoE load specifically is why a phatic reply once stated a stale, invented layer count: the grounding it would have drawn from was empty.

Settings (`core/runtime_settings.py`, ~1150 LOC)

- `DEFAULTS` (the full settings schema) + `ENV_TO_KEY` (env-var overrides).
- `load_settings` / `save_settings` / `update_settings`.
- **Redistribution-aware**: `_migrate_legacy_keys` (schema evolution), `_resolve_relative_model_paths` + `_heal_model_paths` (fix stale absolute paths when the project moves machines), and `_portable_settings_for_storage` (strip machine-specific values before storage) — though personal values like `user_name` can still end up tracked. A model path outside this install is cleared only when it no longer exists here: a model the user browsed to (a download, another drive) is kept. Two guards (`_eli_runtime_sanitize_path_string`, then `_eli_final_clean_path_value`) each cleared every outside model path, so a chosen model was gone from the settings at the next load (fixed in 2.5.21; 2.5.20 fixed only the first guard). The final pass now checks only that a model path leads somewhere: a relative one under `models/`, a full one that exists.
- **`settings_file_was_corrupt_on_last_load()`** — a settings file that exists but fails to parse (truncated write, disk-full, a concurrent-write race) silently fell back to `DEFAULTS` with no signal that the fallback happened, indistinguishable from a normal first run with no file yet. Anything rendering settings to the user should check this rather than presenting defaults as if they were read from disk.

Paths (`core/paths.py`, 808 LOC)

Dev-vs-packaged path resolution: `is_frozen` / `_is_dev_mode`, `_find_project_root`, then `data_dir`/`config_dir`/`cache_dir`/`models_dir`/`db_dir`/`artifacts_dir`/`user_db_path`/`agent_db_path`/`memory_db_path`. A source checkout uses project-local `artifacts/` + `config/`. The packaged app and an installed one both use v2's own root: `~/.local/share/ELI_v2` (`artifacts/`, `config/`, `cache/`), `%LOCALAPPDATA%\ELI_v2` on Windows, `~/Library/Application Support/ELI_v2` on macOS. One import surface (`get_paths`) so nothing hardcodes locations.

Nothing of v2's goes in a generic place: `~/.local/share/eli`, `~/.config/eli`, `~/.cache/eli`, `~/.eli/` and `~/.eli_*` belong to no single ELI version, and v3, the first version (`eli-ai`), ELI Legal and ELI VR keep their own folders. Until 2.5.15 an installed v2 used the generic `eli` folders and the packaged app kept its cache in `~/.cache/eli`; some files had a generic or relative fallback (`artifacts/...` lands in whatever folder ELI was started from). Every writer now goes through `get_paths`. A file v2 once kept in a generic place (the labs notebook and workspaces in `~/.eli/`, `~/.eli_user_id`) is copied into its own folders once by `adopt_legacy_file` (copied, not moved: another version may still use the old place). `tests/test_frozen_install_redistributability.py` scans the code for generic and relative paths.

Launchers are v2's too: `~/.local/bin/eli-v2-run`, `eli-v2*.desktop` menu entries and the `eli-v2` icon. Setup and the uninstaller replace or remove only v2's own entries (named `eli-v2…`, or an older entry whose contents name v2); another version's launchers are never touched.

STT VRAM awareness

`local_whisper_stt` puts faster-whisper on CUDA device 0 only when that card still holds it after the language model's plan: whisper's footprint on that card is measured on its first warm-up and kept (`data_dir/runtime/whisper_gpu.json`), its weights stand in until then. Any doubt keeps it on the processor. ctranslate2 is CUDA only, so other makers' cards keep whisper on the processor. `ELI_WHISPER_DEVICE` overrides. See `perception.md`.

Piper and LoRA device selection

Piper is a subprocess binary (`tts_piper/piper`), not onnxruntime-in-Python, and the shipped build is CPU-only for every vendor — no NVIDIA advantage to close there.

`eli/learning/lora_trainer._accelerator()` is the model for cross-vendor device selection generally: torch routes ROCm through the `torch.cuda` API (a HIP build answers `torch.cuda.is_available()`), so one code path serves NVIDIA and AMD, with the real vendor read from `torch.version.hip` and reported honestly rather than as "CUDA on a Radeon." Apple (mps) and Intel (xpu) are detected and honestly marked as unable to run that trainer. See `learning.md`.

Honest assessment

- **Strong:** loads are measured by llama.cpp on every device before they happen; every card of every vendor is read by one shared reader per source; the model's own chat template; env/settings/relative-path healing for moving between machines; a single broker + lock so concurrency is correct; a GPU pack that trusts a verified install.
- **Weak / watch:**
  1. The DXGI (Windows) and ioreg (macOS) readers are tested with fakes and on CI runners without GPUs, not yet on real Windows or Mac GPUs.
  2. **Settings sprawl** — `DEFAULTS` is large with several overlapping keys (`n_gpu_layers`/`gpu_layers`, `n_ctx`/`context_size`) and migration logic.
  3. Policy figures remain: the RAM budget (60%, 70% on processor-bound machines up to 16 GB, bounded 10-75%), the context target (0.9 of the trained context, half the memory left for the cache, a 2048 step) and the batch defaults.
  4. `_portable_settings_for_storage` exists but isn't fully preventing personal values (e.g. `user_name`) from being persisted.
  5. Install-time detection (`install.sh`, `setup/hardware_policy.py`) is still its own stack.

Adaptive Inference Governor — Status and Remaining Plan

> **Updated for v2.4.84.** Most of the gaps this plan identified in June are closed; the two
> that need a governor (throughput-aware think decision and per-call wall-time projection)
> are still open. Status per gap below; no governor code exists yet.

Scope: make ELI's reasoning and inference *policy* (think or no-think, token budget, pass count, recovery) a pure function of the runtime environment and the task, so one redistributed binary behaves correctly on any machine with any model and no per-machine tuning.

Origin: a 2026-06-17 session in which `EXPLAIN_COGNITION_RUNTIME` burned a 273 s empty compact-synthesis pass, then two 539 s empty standard-synthesis passes, before a two-stage chain-of-thought fallback (470 s and 502 s): about 38 minutes for one question. The working

final answer was 4,071 characters (about 1,018 tokens) in 502 s, roughly 2 tokens per second.

---

0. The one-sentence problem

ELI adapts its inference to **memory** (VRAM, context length) and **model size**, but not to **measured compute speed** (tokens per second). On a slow machine the same prompt that works on the development box can produce multi-minute turns.

---

1. What ELI already adapts to (do not rebuild)

Capability	Where	What it adapts to
Model-agnostic resolution (no baked model)	`gguf_inference.get_model_path`	settings, environment
Chat family and template	`cognition/chat_template.py`, `model_identity.py`, `model_output_tokens.py`	the of-turn markers), not the file name
Thinking-channel detection	`gguf_inference._is_thinking_model` → `model_identity.is_thinking_model`	the e
Trained context length	`core/startup_hardware_optimizer.train_ctx_for_model`	`ELI_MODEL_TRAIN_CTX`, then
Load plan, measured before the load	`core/load_plan.py`	llama.cpp's load simulation on every GPU, each card
Estimate where llama.cpp cannot simulate	`hardware_profile.py` (`smart_fit_config`, `_kv_cache_mb`, `_compu
Load retry	`gguf_inference._build_with_plan`	a failed load is planned again for the memory free then, then c
Per-mode token budget	`cognition/reasoning_modes.py`	`n_ctx` (20 % of the window for quick, 30 % otherwise)
Size tier → budget scale	`core/model_tier.py` `detect_tier`, `tier_scale`	model size
Live decode-speed EMA	`model_tier.record_speed`, `measured_tok_s` (fed from the non-stream generation path in `gguf_inference`)	measured tokens per second
Speed-capped pass count	`model_tier.speed_passes`	measured tokens per second (thresholds `ELI_SLOW_TPS`=5
No-think on utility calls	`gguf_inference._no_think_prefill`, `force_no_think()`	structured output, small
Redistribution-aware settings	`core/runtime_settings.py`	machine moves

The speed EMA is consumed in exactly one place, `speed_passes()`. Nothing in the think decision, the token budget or mode selection reads it. `speed_passes()` reduces how many full generations a mode runs and never caps output length.

---

2. The gaps and where each stands

Gap	Was	Status
A thinking detection was a filename allowlist	`_is_thinking_model` matched `qwen3`, `deepseek-r1`, ...	**Closed.** Detection reads the chat template, then the architecture.
B the think decision and token budget ignored throughput	no tok/s term in `_no_think_prefill` or the budge
C no wall-time projection	nothing computes "at the live EMA this call takes about N seconds"	**Open.**
D empty output was not self-correcting	an unterminated `<think>` returned empty, then a full-price retry	**Partly closed.** An empty non-stream response is retried once with `force_no_think` in the engine think in `gguf_inference`. There is still no reserved answer budget.
E context sizing undershot the trained context	a filename table returned 32768 for `qwen3`	**Closed.**
F `force_no_think` was a point patch	one call site	**Partly closed.** It now wraps compact grounded synt of-thought stage prose, the correction repair, and the empty-response and stream retries in `_get_chat_response` agent fix generation for `SELF_IMPROVE` are not wrapped.
G redundant proactive churn	`persona_updater` rebuilt the same overlay every tick	**Closed.** Persona a profile overlays are debounced (120 s) with a content dirty-check.

Note on gap G: `persona_updater` is deterministic (database reads and regexes); the cost was redundant CPU and SQLite work, not inference-lock contention.

---

3. Design for what remains: the Adaptive Inference Governor

A single policy object that every model call routes its *decision* through. It consumes `reasoning_modes`, `model_tier` and `hardware_profile`; it does not replace them.

3.1 Inputs (all available at runtime)
- **Model capability**: `model_identity.is_thinking_model` (template, architecture), cached per model.
- **Throughput**: `model_tier.measured_tok_s()`, with a cold-start default from `detect_tier()` until the first measurement.
- **Context**: `n_ctx` from the runtime snapshot and `n_ctx_train` from metadata.
- **Task class**: below.

3.2 Task classes
1. **STRUCTURED**: JSON, routing, plan graphs. Never thinks (already true via `structured=True`).
2. **GROUNDED_SYNTHESIS**: evidence already gathered; the call only phrases it (the `EXPLAIN_*` actions, RAG answer synthesis, fix-patch rendering). Thinking adds nothing; force no-think.
3. **OPEN_REASONING**: genuine multi-step reasoning (chain of thought, tree of thoughts, constitutional, self-consistency, open chat on a hard question). Thinking is allowed but bounded by throughput.

3.3 The decision (a pure function)

policy(task_class, model_caps, tok_s, n_ctx, prompt_tokens) -> { think: bool, think_budget: int, # max tokens allowed inside answer_budget: int, # reserved tokens after passes: int, # via speed_passes() }

- A model that does not think: `think=False` always.
- STRUCTURED or GROUNDED_SYNTHESIS: `think=False`.
- OPEN_REASONING: at or above `ELI_FAST_TPS` think with the full budget; between slow and fast scale `think_budget` so the projected wall time is within the target; at or below `ELI_SLOW_TPS` turn thinking off, or hard-cap `think_budget`.
- Always reserve `answer_budget`, so a thinking call cannot exhaust its budget before it answers.

3.4 Integration points
- `gguf_inference._no_think_prefill`: consult the governor instead of only `structured`, `< 1024` and the environment; keep `force_no_think()` as the primitive and `ELI_MODEL_THINK` as the hard user override.
- `reasoning_modes`: split the target tokens into `think_budget` and `answer_budget`.
- Engine synthesis sites: tag `_compact_grounded_synthesis` and `_synthesize_answer` as GROUNDED_SYNTHESIS and the reasoning-mode runners as OPEN_REASONING.
- Feed the speed EMA from the stream path as well, so quick-mode chat updates it.

3.5 Configuration
`ELI_SLOW_TPS` and `ELI_FAST_TPS` exist (5 and 15). A soft per-turn wall-time target (`ELI_TURN_LATENCY_TARGET_S`) is proposed and does not exist yet. `ELI_MODEL_THINK` is the hard override. Nothing machine-specific is persisted.

---

4. Cross-machine behaviour to aim for

Machine and model	tok/s	grounded actions (`EXPLAIN_*`)	hard chat
8 GB GPU with a 30B MoE	about 2	no-think, one pass	bounded think, within the target, never empty
24 GB GPU with a 7B	about 40	no-think	full think, full budget
CPU-only with a 7B	about 3	no-think	think off or hard-capped, with an answer reserve
any machine with a non-thinking model	n/a	normal answer	normal answer
any machine with an unfamiliar reasoning model	any	detected through the template	handled

5. Phases

1. **Governor core** for grounded synthesis: the policy function, no-think wired through
   `_no_think_prefill`, coding-agent fix rendering tagged.
2. **Throughput-bounded open reasoning** with an answer reserve: split budgets in
   `reasoning_modes`.
3. **EMA from streaming** so quick mode updates the measured speed.

Each phase is independently shippable.

6. Verification

- Unit: `policy(...)` truth table over task class × throughput tier × model-thinks.
- No-model engine tests in the style of `tests/test_*_no_gguf*.py`.
- Behavioural: `EXPLAIN_MEMORY_RUNTIME` and `EXPLAIN_COGNITION_RUNTIME` synthesis is non-empty on
  the first pass and runs no-think.
- Speed simulation: inject a fake `measured_tok_s()` and assert the think flag and budget flip.
- Existing guards that must stay green: `tests/test_gguf_think_stop_collision.py`,
  `tests/test_reasoning_mode_contract.py`, `tests/test_cot_think_budget.py`.

7. Risks and non-goals

- **Over-suppressing thinking** on hard turns at mid speed: only GROUNDED_SYNTHESIS is
  unconditionally no-think; OPEN_REASONING keeps thinking with a budget above `slow`.
- **Non-goals:** changing model selection, the agent set, persona content or the grounding
  contract; removing `ELI_MODEL_THINK`.

8. Mixture-of-experts models on a small GPU

`eli/core/moe_offload.py`. A mixture-of-experts model uses a few experts per token, and the experts are most of the file. Splitting such a model by layers puts mostly idle experts on the GPU. With the offload on, every layer's attention and shared weights go to the GPU and the expert tensors stay in system memory (llama.cpp's tensor buffer override, applied to the model parameters while the model is built, because llama-cpp-python does not expose it).

- Measured on an RTX 2060 SUPER (8 GB), Qwen3.6-35B-A3B Q4_K_M, 1,500-token prompt: 8 GPU layers gave
  71 tok/s prompt and 8.3 tok/s generation; all layers with experts in RAM gave 125 tok/s and 21.8 tok/s.
  Load time fell from 63 s to 5 s. Through `load_model` the same model went from 10.7 to 20.8 tok/s.
- When: the model is MoE (`GGUFModelProfile.is_moe`), it would not fit in free VRAM, and available RAM
  covers about three quarters of the expert weights. Otherwise the ordinary layer split is used.
- Switch: the "Expert offload (MoE)" choice in the startup dialog, the setting `moe_expert_offload` or env `ELI_M
  `ELI_MOE_EXPERT_FRACTION` adjusts the assumed expert share of the file (default 0.90).
- The live runtime snapshot carries `moe_expert_offload`.

9. The startup dialog's own model loader

`eli/gui/eli_pro_audio_gui_v2_0.py` builds `Llama(...)` itself for the "Load selected model now" path — it does not go through `gguf_inference.load_model()`. The MoE expert-offload hook (section 8) is wired into that loader too now; before this it was saved by the toggle but never reached the actual load, so the setting did nothing when a model was loaded from the startup dialog.

Measured against a live session (Qwen3.6-35B-A3B, ctx=13000, batch=512, RTX 2060 SUPER): 7 GPU layers without offload gave 6.5 tok/s; the offload's 41 layers (one past the model's 40 blocks — llama.cpp's own "all layers" convention, not an error) with experts in RAM gave 13.3 tok/s. Real, but the console printed the pre-offload layer count ("GPU-layer load parameter: 7") before a later line showed the actual loaded count ("gpu_layers=41") with no explanation connecting them — fixed, the first line now says when and by how much MoE will raise it. The bigger cost per chat turn is architectural, not this: chain-of-thought runs two full model calls (a private scratchpad, then the final answer) against 6,000–7,700-token prompts each, and MoE's prompt-processing gain (~1.75x) is smaller than its generation gain (~2x) — doubling generation speed does not halve a turn dominated by two large, mostly-prompt-bound calls. The second call no longer reads its prompt again: every pass of a turn is given the same system prompt (the engine builds it once per turn and keeps a quarter of the room for each pass's message), and the model's state at its end is kept between the passes (`gguf_inference._resume_from_saved_state`, see `inference_and_hardware.md`). Measured on this model: the second pass 63.8 s before, 3.2 s after. The first pass still reads the whole prompt (about 65 s for 7.5k tokens on this card); shrinking that prompt is separate work.

The footnote explaining the bump wasn't enough — the "HW Profile (recommended)" number itself, in `hardware_profile.recommend()`, still showed the pre-offload layer count, and that number is also what gets saved as the canonical pin (`hw_profile_n_gpu_layers`) other load paths read back. MoE is now decided before the recommendation is built, so `rec.n_gpu_layers` — the number this panel shows and the number that gets pinned — is the one that actually loads, with one reasoning line explaining why, not a separate figure with no visible link to it. This exposed a second bug: the VRAM-headroom check that sizes batch was reading "all layers on GPU" as "the full model is in VRAM," which is false under MoE (most of it is in RAM) — it would have computed a large phantom deficit and needlessly crushed the batch size. It now sizes off the plan's actual resident-on-GPU figure when MoE applies.

ELI Learning — LoRA Self-Training Pipeline

> **Updated for v2.4.84.** Stage 12 learning hooks for chat turns are centralized in
> `eli/cognition/learning_coordinator.py` (`finalize_turn()`).

`eli/learning/` — 4.3k LOC, 14 files. ELI's self-improvement-via-fine-tuning
path. **Important framing:** this is a *curated, human-gated, operator-invoked*
training pipeline — **not** an autonomous self-modifying loop, and it targets a
separate trainable Hugging Face base, not the inference GGUF.

It is driven from **Labs ▸ □ Training** (a four-step wizard) as well
as from chat actions and the overnight scheduled task. Two long-standing blockers
were removed at the same time: the human review gate now has an interface, and the
target system is no longer locked to Phi-3.

Files & the pipeline

The stages, roughly in order:

File	LOC	Stage
`dataset_builder.py`	552	turn ELI's logged turns/corrections into supervised examples
`dataset_filters.py`	148	quality gates (`is_bad_response`, `row_is_reviewed`)

`export_trainable_dataset.py`	162	export reviewed rows → trainable JSONL
`merge_reviewed_datasets.py`	129	merge reviewed dataset shards
`bootstrap_phi3_base.py`	239	one-time download of the trainable HF Phi-3 base
`base_model_resolver.py`	251	locate/validate the trainable base (vs an adapter)
`training_preflight.py`	136	check peft/transformers/datasets present + target ready
`lora_trainer_guard.py`	630	`TrainerTarget` plans (paths, base_family) + guard checks
`lora_trainer.py`	834	device selection, QLoRA, chat-template prompts, PEFT loop
`lora_eval.py`	485	eval harness (score vs expected/forbidden, inspect adapter)
`target_registry.py`	266	operator-declared training targets, any model family
`review_queue.py`	270	the human review gate: triage, approve/edit, write trainable rows

Flow

1. **Build** (`dataset_builder.py`): mines ELI's own conversation/correction
   history into `SupervisedExample`s. Crucially includes `clean_text` +
   `redact_text` (PII redaction) and an exclusion set so sensitive/garbage
   content never becomes a training example. Writes
   `training/datasets/eli_supervised_v0.jsonl` + a report.
2. **Gate** (`dataset_filters.py`): `is_bad_response` and `row_is_reviewed`
   enforce quality + a **human review flag**.
3. **Export / merge**: reviewed rows → `*.trainable.jsonl`.
4. **Preflight** (`training_preflight.py`): refuses to proceed unless the HF
   training stack is installed and the target/base resolve.
5. **Train** (`lora_trainer.py`): `_dataset_report` **refuses to train** on a
   dataset with unreviewed rows, wrong-target rows, or bad-response rows — i.e.
   it will not learn from un-vetted data. Loads the base through native
   transformers (with a RoPE-scaling compat shim for Phi-3), trains the adapter.
   Two things changed in 2.3.7:
   - **Prompt format follows the base model.** `_format_example` used a hardcoded
     Phi-3 `<|user|>/<|assistant|>` literal. Training a Qwen or Llama checkpoint on
     Phi-3 turn markers teaches it tokens its chat template never uses, degrading
     the model instead of tuning it. The tokenizer's own `chat_template` is now the
     source of truth, with the literal kept as a fallback for bases without one.
   - **QLoRA.** With `bitsandbytes` present the frozen base loads in 4-bit (nf4,
     double quant, fp16 compute) with a paged optimiser, so a card too small to hold
     the fp16 weights can still train the adapter.
6. **Eval** (`lora_eval.py`): `score_response` against expected/forbidden, plus
   `inspect_adapter` / `inspect_eval_suite`.

Device selection (`lora_trainer._pick_device`)

The old rule was a flat `free_vram >= 10 GiB` floor written for a Phi-3 on an 8 GB
card, and its failure mode was a **silent** fall-through to CPU — where a run takes
days and looks exactly like a hang. It also named every accelerator "cuda", showing
CUDA on a Radeon.

- `_accelerator()` reports the vendor honestly. torch routes ROCm through the
  `torch.cuda` API (a HIP build answers `torch.cuda.is_available()`), so one code
  path serves NVIDIA and AMD; the vendor is read from `torch.version.hip`. Apple
  (mps) and Intel (xpu) are detected and **honestly marked unable** to run this
  trainer rather than being silently ignored.
- `estimate_vram_gb()` sizes the requirement from the checkpoint's real size on
  disk, so the floor holds for a 1B and for a 35B instead of a hardcoded figure.
- A refusal states what would fix it. On a 2060 Super: *"6.37 GiB free but this run
  needs ~9.04 GiB … free VRAM, shorten the sequence length, or install bitsandbytes

  for 4-bit training."*

Paths

`eli.core.paths.learning_dir()` follows the `models_dir()` pattern: the source tree in dev so the existing `training/` layout keeps working, the **user data dir** on a packaged install where `project_root()` is a read-only mount. Run logs, guard plans and the target registry all follow it, so a frozen build no longer writes into its own installation.

Targets (`target_registry.py` + `lora_trainer_guard.py`)

A **target** pairs a base model with its dataset, adapter and output directory. Until 2.3.7 the allowlist was the literal set `{eli_phi, eli_phi_ultra}` and the guard asserted `base_family == "phi3"`, so a redistributed copy running Qwen or Llama was refused at the first gate with no way to declare a target short of editing source.

The allowlist is still an allowlist — nothing trains unless it was explicitly declared — but declaring is now a **runtime act**:

- `target_registry.create_target(name, base_model_path, …)` writes an operator target into `<data dir>/training/targets.json`. Built-ins remain, so existing installs are unaffected.
- **Family is read, not asserted.** `detect_family()` reads `model_type` from the base model's own `config.json`. The guard then checks the declared family against what is actually on disk, so a target claiming `qwen3` over a Llama checkpoint is still refused. Nothing in the registry enumerates known families, so a model type released after this build still resolves.
- `base_model_resolver.discover_base_models()` scans the model roots for **any** trainable HF directory and reports each with its family and size.
- A target that has never trained has no adapter yet. That is now a *first run*, not a fault — only a malformed adapter is an error.

The review gate (`review_queue.py`)

`lora_trainer_guard` has always required every training row to be approved by a person and scoped to the target. That contract was enforced but **unreachable**: `dataset_builder` writes candidates tagged `needs_review`, and nothing in the product could retag one. Live state before this module existed: 615 candidate rows, 0 trainable, `will_train` permanently false.

`ReviewQueue` is the queue behind the Training tab:

- **Assisted triage** pre-rejects the provably unusable — `is_bad_response` matches, duplicates, instructions under 8 characters — and *flags* rows that need a human eye (very long output, redacted paths, heavy structure).
- **Triage never approves.** Approval is a person's act; there is a test pinning this. "Approve all clean rows" is one click standing for reading a filtered list, not an auto-approve.
- Editing a row **re-triages** it: an edit can rescue a rejected row and can equally break one that passed.
- `save()` writes approved rows tagged `reviewed` and scoped to the target — exactly the shape `_validate_dataset_rows` demands — overwriting rather than appending, so a second pass does not duplicate.

On the live corpus: 615 candidates → 20 auto-rejected, 309 clean, 286 flagged for judgement; approving the clean set took `train_ready` from false to **true** for the first time.

What it is — and isn't

- It **is**: a responsible, reproducible fine-tuning pipeline with PII redaction, human review gating, preflight, and an eval harness. The dataset comes from ELI's real interactions (corrections, failures, conversations).
- It **isn't**: (a) automatic — it is driven by the operator, through the Training tab, the chat actions, or a scheduled overnight job. (b) connected to the live inference model by itself — it trains an adapter on a Hugging Face base, while inference runs a GGUF. A trained improvement reaches the running assistant only once the adapter is merged into the base and converted to GGUF (`training/merge_and_convert.py`); the Training tab says so explicitly when a run finishes rather than implying the model has changed.

Honest assessment

- **Strong (and unusually responsible):** PII redaction, refusing to train on unreviewed/bad rows, a preflight, and an eval suite are exactly what a serious fine-tuning loop needs and what most hobby projects skip. The base-vs-adapter validation prevents the classic "trained on an adapter" mistake.
- **Weak / watch:**
  1. **Base/inference disconnect** — training improves Phi-3, but the assistant usually runs another GGUF, so the self-improvement doesn't reach the live model. The loop is only closed if you actually serve the trained model.
  2. **Manual** — "self-training" is operator-driven, not autonomous. That's safe, but it means it improves only when you run it. (Arguably the right trade-off, but worth stating plainly rather than implying autonomy.)
  3. **Heavy deps** — needs the full HF/peft/transformers stack. `peft` and `datasets` were previously only in `requirements.lock.txt`, so a plain `pip install -r requirements.txt` produced an install whose Training tab could never leave preflight; both are now in `requirements.txt`, with `bitsandbytes` added for 4-bit training (excluded on macOS, which has no wheel).
  4. **Phi-3 remains the shipped default** — the built-in targets and `bootstrap_phi3_base.py` are still Phi-3 — but it is no longer a lock. Any local HF base can be declared as a target from the Training tab, and the prompt format follows the base model's own chat template rather than Phi-3's literal turn markers.
  5. **Merge-to-GGUF is still manual.** `training/merge_and_convert.py` closes the loop but depends on `unsloth`, which is in no requirements file. Producing a usable model after a successful run is therefore still a documented manual step, not a button.

LoRA fine-tuning — audit & pipeline (2026-06-07)

What was found (audit)
The LoRA mechanisms in `eli/learning/` were **individually correct but orphaned and phi-hardcoded**:

- **Algorithm — correct.** `lora_trainer.run_training` is a proper PEFT LoRA loop: tokenize (truncation/pad, `labels=input_ids`) → `AutoModelForCausalLM` (eager attn,

  fp16 on CUDA, gradient checkpointing) → `LoraConfig` → `get_peft_model` → `Trainer` with `DataCollatorForLanguageModeling(mlm=False)` → `trainer.train()` → `save_pretrained` (adapter + tokenizer). Safety contract is sound (dry-run default, `--execute` required, reviewed rows only, **GGUF never trained directly**, adapter never overwritten).
- **Stages exist** but as separate CLI scripts with no orchestrator: guard plan (`lora_trainer_guard`) → `training_preflight` → `dataset_builder` / `export_trainable_dataset` / `merge_reviewed_datasets` → `lora_trainer` → `lora_eval`. Nothing chained them.
- **Orphaned** — nothing outside `eli/learning/` called any of them (no executor action, router, GUI button, scheduled task, or self-upgrade hook). ELI could not trigger or even report on LoRA.
- **Not model-agnostic** — `target_modules` defaulted to phi-3's `["qkv_proj"]`.

What was wired/fixed
- **Model-agnostic target modules** (`lora_trainer._resolve_target_modules`): derives LoRA targets from the LOADED architecture (scans for known projection Linear leaves: q/k/v/o_proj, qkv_proj, gate/up/down_proj, c_attn, query_key_value, …), honours an explicit adapter-config override, and falls back to PEFT's architecture-agnostic `"all-linear"`. No hardcoded architecture on the train path.
- **Pipeline DAG** (`eli/learning/lora_pipeline.py::run_pipeline`): the explicit ordered chain **preflight → build_job → [train] → eval(inspect)**, reusing the existing modules. **Dry-run by default** (runs every gate, touches no GPU, writes no adapter — safe from chat); real training only `execute=True`.
- **Wired into ELI:**
  - `LORA_STATUS` action (read-only): preflight readiness per target.
  - `LORA_TRAIN` action: runs the pipeline DAG — **dry-run from chat**; real training only via the scheduled task / explicit GUI (execute flag not exposed in chat).
  - Router: "lora status" / "is lora ready" → LORA_STATUS; "train a lora" / "fine-tune yourself" / "run lora training" → LORA_TRAIN.
  - Scheduled `lora` kind (`_worker_lora`): "train a lora overnight [N steps]" → `SCHEDULE_TASK` runs `run_pipeline(execute=True)` unattended.
  - Manifest: 230 capabilities (includes LORA_STATUS + LORA_TRAIN).
- Tests: `tests/test_lora_pipeline.py` (target-module resolver, DAG order + dry-run no-train, both actions, routing, scheduled kind).

The deferred base-profile change, done

The item recorded above as *"deferred to your go-ahead"* — making the base profile swappable to any HF causal-LM directory — is implemented, along with the GUI the pipeline docstring always claimed existed.

1. Targets are operator-declared (`eli/learning/target_registry.py`)

`ALLOWED_TARGETS = {"eli_phi", "eli_phi_ultra"}` plus `base_family must be phi3` was this machine's two targets frozen into a redistributed product. The allowlist is still an allowlist — nothing trains unless explicitly declared — but declaring is now a runtime act written to `<data dir>/training/targets.json`. Family is **read** from the base model's own `config.json` and checked against the declaration, so the check moved from *"is it Phi"* to *"is it what you said it was"*.

2. The review gate is reachable (`eli/learning/review_queue.py`)

The trainer always required human-approved, target-scoped rows. Nothing could produce them. Live state: **615 candidates, 0 trainable**, `will_train` permanently

false. `ReviewQueue` adds assisted triage (auto-reject the provably unusable, flag what needs a human eye) and an approve/edit/reject loop. Triage **never approves** — that is a person's act, pinned by test. Result on the live corpus: 615 → 20 auto-rejected, 309 clean, 286 flagged; approving the clean set produced `train_ready: true` for the first time.

3. Honest device selection

The flat `free_vram >= 10 GiB` floor refused a 1B on a 6 GB card and silently chose CPU, where a run takes days and looks like a hang. The floor is now the model's own estimated need; the accelerator is named by vendor (NVIDIA / AMD-ROCm / Apple MPS / Intel XPU); and a refusal says what would fix it. QLoRA (4-bit nf4 + paged optimiser) lets a small card train the adapter at all.

4. Prompt format follows the base

`_format_example` hardcoded Phi-3's `<|user|>/<|assistant|>`. Training a Qwen on Phi-3 turn markers teaches it tokens its chat template never uses — degrading the model rather than tuning it. The tokenizer's own `chat_template` is now the source of truth, with the literal as fallback.

5. The GUI: Labs ▸ □ Training

Four steps — Hardware → Target → Data → Train — with live step/loss on a QThread, a dry-run readiness check, and cancel. See `gui.md`.

6. Paths

Runs, datasets and the registry follow `paths.learning_dir()` (source tree in dev, user data dir on a packaged install) instead of writing into a read-only mount.

Still manual

Merge → GGUF. `training/merge_and_convert.py` closes the loop but depends on `unsloth`, which is in no requirements file, so producing a usable model after a successful run remains a documented manual step. The Training tab says so plainly when a run finishes rather than implying the live model has changed.

> **Updated for v2.4.84.**

ELI Background Tasks & Unified Code Generation

Two related additions:
1. **All code generation now routes through the verified coding agent** — asking for a script, or self-upgrading, runs the plan → DAG/tree-search → execute → repair → bug-memory pipeline instead of a single LLM shot.
2. **Heavy work runs on background threads** — when a task is estimated heavy (or you ask), ELI starts it on a worker thread, hands back a job id immediately, and you check on it later.

1. Unified code generation

Path	Before	Now
`CODE_SOLVE`	the coding agent	the coding agent (unchanged)

| `GENERATE_SCRIPT` | inline single-shot prompt + static checks | **routes through `eli.coding.solve` first** (p
search/verify/repair), saving a verified, top-tier script; falls back to the inline generator only if the agent y
| Self-upgrade (`self_improvement.generate_code_patch`) | LLM patch + import-verify | now also **consults the co
term bug memory**: classifies the failure and injects prior fixes for that bug class into the patch prompt |

So "write a script", "generate code", and "improve ELI" all share one engine and
one bug-memory — the same consolidation move applied to `recall_memory`. Kill
switch: `ELI_GENERATE_SCRIPT_AGENT=0` restores the legacy inline generator.

2. Background task manager (`eli/runtime/background_tasks.py`)

In-process, multi-threaded (`ThreadPoolExecutor`). **Distinct** from
`eli/planning/jobqueue.py` (which runs durable *external subprocess* jobs) — this
runs live in-process Python work (agent runs) for the session.

- `BackgroundTasks.submit(name, fn, *args)` → integer job id; tracks
  `queued → running → done/failed/cancelled`, result, error, elapsed, note.
- `get(id)`, `list()`, `cancel(id)` (best-effort), `wait(id)`, `stats()`.
- Singleton `get_background_tasks()`; thread-safe.
- **What a job's status means.** A handler that returns `ok: False` failed, whatever the thread did;
  `skipped` is not a failure. `elapsed` counts from the start, so a job waiting for its time reads 0.
- **The record outlives the session.** Jobs are kept in `data_dir/runtime/background_jobs.json` for
  30 days; numbers carry on, and a job that was queued or running when ELI closed loads as
  `interrupted`: reported, not resumed. `work_lines` and `work_block` read it for job questions and
  for any message about files, folders, jobs or progress (`eli/runtime/live_reads.py`), so the model
  answers from the record, and `work_targets` is what "the rest" or "them" refers to.
- **Jobs are the person's.** ELI's own housekeeping (kind `housekeeping`) stays out of `list()`, so "did
  the job finish?" is never answered about one; the Tasks tab asks for it. Sizes and windows are the
  `cog.jobs_*` tunables. Asked what a job found, the model reads its report (`latest_report`).
- **When a job ends** (`add_listener`). A failed one goes on the failure record self-analysis reads
  (`_record_failed_job`); one someone asked for reaches them by desktop notification and the proactive
  queue (`_tell_the_person`). What goes wrong after a job is kept on the job, `meta.after_job_error`.
- **Model calls from a job.** Daemons get an empty reply while a person is talking
  (`ELI_BG_DEFER_WINDOW`). A job is started by a turn, so that blanked every job: it now waits, and
  re-runs a call that a turn cut short (`gguf_inference.set_job_inference`).

When ELI backgrounds a task (`eli/coding/cost.py`)

`should_background(task)` decides:
- **Explicit phrasing wins:** "in the background / don't wait / notify me when" →
  background; "right now / quickly / wait for it" → foreground.
- **Otherwise heuristic:** `estimate_cost` scores heavy-compute signals
  (simulate/monte-carlo/optimise/benchmark/train/dijkstra/parallel…),
  multi-component structure, length, and planned-step count. `heavy` (≥0.6) ⇒
  background.

`_maybe_background_codegen` (executor) applies this at the top of `CODE_SOLVE`
and `GENERATE_SCRIPT`: if heavy/explicit, it submits the same action as a
background job (re-dispatched with `_no_background` so the worker doesn't
re-background) and replies *"started as job #N — say 'check job N'."*
Kill switch: `ELI_CODEGEN_BACKGROUND=0`.

Inspecting jobs

- Actions **`CHECK_JOB`** (`{job_id}` or parsed from "check job 3") and

  `BACKGROUND_JOBS` (list), both in `SUPPORTED_ACTIONS`.
- Router triggers: "check job N" / "job N", "background jobs" / "list jobs", and while a job runs a
  question about its progress or time left ("do you have an eta?", "is it done yet?"), which goes to
  the running job (`_JOB_PROGRESS`); "how long is the Eiffel Tower" does not.
- **Progress.** A job reports `background_tasks.report(done, total, stage)` from its own thread;
  `CHECK_JOB` and the Tasks tab show the stage, how far it has got and the time left in that stage
  at the rate measured so far (`Task.eta_s`). The folder analysis reports reading and summarising.
  Without this an ETA question got "several hours" for a ten-minute job.

Control summary

Knob	Default	Effect
`ELI_GENERATE_SCRIPT_AGENT`	on	GENERATE_SCRIPT uses the coding agent (off ⇒ legacy inline)
`ELI_CODEGEN_BACKGROUND`	on	auto-background heavy codegen (off ⇒ always foreground)
`ELI_AGENT_DAG` / `ELI_CODING_DAG`	on	DAG dispatch / subtask DAG (see `dag.md`)
`ELI_CODING_SANDBOX` / `*_RUN_TIMEOUT` / `*_BEAM` / `*_MAX_ITERS`	–	coding-agent execution knobs

Scope / honesty

- **Tested deterministically** (`tests/test_background.py`): manager run/fail
  reporting, cost light-vs-heavy + explicit phrasing, and the CHECK_JOB /
  BACKGROUND_JOBS executor round-trip. No model needed.
- **Threads, not processes:** background tasks share the process; a running
  thread can't be force-killed (cancel is best-effort / cooperative). True OS
  isolation for a job uses the durable subprocess `jobqueue`: run `eli-jobs worker` and
  queue with `eli-jobs submit --cmd "..."` (jobs survive a restart; one a dead worker left
  running is marked failed on the next worker start).
- **"LLM deems longer wait":** currently a deterministic estimator (+ explicit
  phrasing). The planner's decomposition (`plan_steps`) can be fed in for an
  LLM-informed estimate — hook present, not yet auto-wired.
- **GUI:** the Tasks tab (`eli/gui/tabs/tasks_tab.py`) is the single home for background work: coding jobs and sc

ELI Runtime Surfaces, Planning, World, Tools & Plugins

> **Updated 2026-09-30.** Stage 12 learning via `learning_coordinator.py`; goal
> autogenesis feeds the proactive stack. Task records gained an explicit resume path
> and a World bridge; habit learning is now gated against ELI's own automated actions.

The remaining subsystems: the `runtime/` response/introspection surfaces, the
proactive planning layer, the "world"/autonomy model, and the tools + plugin
system.

Runtime surfaces (`eli/runtime/`, the response/introspection group)

Beyond the grounding/evidence core (`grounding_and_evidence.md`), `runtime/`
holds a large family of modules that render user-visible answers and introspect
the live system:

- **Introspection**: `live_introspection.py` (`runtime_snapshot`, `last_trace`,
  `stored_user_name`, `mine_user_fact_candidates`, `build_report`,
  `agents_for_action`), `deterministic_introspection.py`
  (`classify_diagnostic_action` → `gather_evidence` → `format_evidence_block` →
  `maybe_handle` — deterministic diagnostic answers).

- **Response assembly**: `final_response_provider.py`,
  `response_contracts.py`, `response_policy.py`,
  `user_visible_response_surface.py`.
- **Personal memory surfaces**: `personal_memory_surface.py`,
  `personal_memory_clean_response.py`, `personal_memory_deep_response.py`,
  `profile_extractor.py`.
- **Status**: `frontier_status.py`, `reasoning_status.py`, `truth_report.py`,
  `reflection.py`.
- **Operator feed**: `operator_feed.py`,
  `operator_state.py`.

> **Over-fragmentation flag (the headline weakness for this group):** there are
> ~15 closely-related modules here doing "render a grounded user-visible
> answer / introspect runtime" with overlapping responsibilities (three
> `personal_memory_*` renderers alongside the response-contract, response-policy and
> user-visible-surface modules).
> This is the clearest "added beside, not folded in" cluster in the codebase and
> the prime consolidation target — it should collapse to a handful of well-named
> modules with clear ownership of "who produces the final string."

Planning / proactive (`eli/planning/`, 4.0k LOC)

Background cognition + goal/queue machinery:
- `proactive_daemon.py` (1.0k) — the **self-improvement / proactive** loop. Uses
  a two-DB split: reads user-facing memory/conversation from the **user DB**,
  writes proactive observations/improvements/errors to the **agent DB** (keeps
  ELI's own machinations out of user recall). Background thread.
- `autonomy_scheduler.py` — schedules autonomous actions (throttled). Also
  the "decide to invoke it" layer for self-upgrade and LoRA retraining
  (`_maybe_check_self_upgrade`/`_maybe_check_lora_retrain`, added
  2026-09-18): both mechanisms were fully built and safety-gated but
  pull-only — nothing inside ELI ever decided on its own to use them, even
  with a newer release on GitHub or enough reviewed training data sitting
  idle. Only fires in `goal_driven` operator-policy mode (every other mode
  is unchanged, still pull-only), on a multi-day throttle per check kind,
  and routes through the same `schedule_request(kind=...)` entry point and
  downstream safety machinery a human-typed "update ELI overnight" already
  used — this closes the trigger gap without adding a new execution path.
- `goal_store.py`, `proposal_queue.py`, `proposal_adapters.py`,
  `attention_queue.py`, `jobqueue.py`, `operator_goal_actions.py`, `habits.py` —
  goal persistence, capability-proposal queue, attention prioritisation, a job
  queue, and habit scheduling.
- **Routine detection** (`habits.py`, `routine_stats.py`): app launches are clustered on the
  clock as a circle (neighbours within 30 minutes join, across midnight too), so 08:58 and 09:02
  are one routine. A routine needs launches on at least three distinct days. `detect_shift` flags a
  routine whose recent times sit an hour or more from the earlier ones, and `detect_lapse` flags one
  that stopped, judged against the days the person was active at all; each is recorded once as an
  observation and never changes a rule you enabled. `detect_habits` returns
  `{suggested, shifts, lapsed}`. The scheduler keeps its run keys on disk so a restart in the same
  minute does not run a habit twice, and a plain-text reply from the engine is reported as unverified
  rather than as success.
- **Lessons** (`runtime/lessons.py`): failures that recur (same action, same error, three or more
  times) become lessons with a trigger, evidence, proposed change, predicted outcome and an expiry.
  Each finished action of that kind gives its lessons one check; a lesson is retired when it expires
  unconfirmed or its checks show it does not help, and renewed a few times when they do.

- **Lessons, checked against the baseline.** A lesson records the action's success rate before it existed and is retired unless later runs beat that rate by a margin. It is scoped to the argument names all its failures shared and is checked on sibling actions with the same failure category, so `transfers()` says whether the advice held elsewhere.
- **Reliability and calibration** (`evidence_ledger.py`): `action_reliability` and `predict_success` give a recency-weighted success rate per action from the recorded tool runs; each run stores the prediction made before it, and `calibration_report` scores them (Brier score and predicted versus observed by band). The awareness block lists actions that have been failing lately.
- **Task records** (`goal_store.py`): a task is a goal the scheduler does not tick, carrying constraints, decisions, finished steps, artifacts and open questions. `capture_task_events` reads clear phrasings from the user's messages ("we decided to", "we still need to", "let's work on"), and `task_brief` puts unfinished work into the next session's awareness block — but only within a 14-day idle window. `find_task(query, include_idle=True)` (2026-09-29) bypasses that window on an explicit ask ("continue the ORBIT project" routes to `RESUME_TASK`, a real router/executor action — not a chat-routed guess), and states real staleness ("last touched N days ago") rather than silently implying continuity. `close_task(goal_id)` is the explicit, non-inferred completion path. Opening/closing a task also notifies the World system (below) — a task is now a real, visible object in Eli's World, not just a database row.
- **Habit-learning integrity** (2026-09-30): `_learn_from_result`'s writes into `habit_events` — the table `detect_habits` mines for recurring routines — are gated on whether the triggering call actually came from the user. `habits_scheduler.py` firing an approved rule calls `engine.process(source="habit")`; without the gate, that automated firing would land back in `habit_events` and let ELI's own automation reinforce itself as if it were a fresh, independent observation of a routine. `_is_autonomous_source()` (shared with `_settle_recall_outcome`, which already checked this) treats `habit`/`proactive`/`scheduler`/`system`/`autonomy` as non-user origins.
- **World events.** `review_completed` eases repair pressure after any review; `repair_completed` fires only for a verified repair.
- `goal_autogenesis.py` (NEW, 2026-06-08) — **closes the autonomy loop**: the goal stack (goal_store → `due_goals` → `governed_goal_tick` → proposal_queue) was fully wired but the store was always EMPTY because `create_goal` was operator-only, so the scheduler ticked nothing and ELI only reacted. `propose_goals_from_signals` turns the signals ELI already computes each proactive tick — world-model awareness suggestions (≥0.6), recurring failures (≥5×), and code-health improvements (oversized fns / duplicate blocks in his own god-files) — into governed goals. Each is `autonomy_mode="proposal_only"` (surfaces for approval; never silent execution), deduped by a stable signal-derived `goal_id` (idempotent every tick), capped at `MAX_AUTO_GOALS=6`, logged as a `self_goal` observation. Wired into `proactive_daemon` where the world-suggestions + patterns + improvements are in scope. His agenda now spans failure-repair, world-driven, AND self-improvement — i.e. he sets his own intentions, governed.

World / autonomy model (`eli/world/`, 1.8k LOC)

The substrate behind ELI's emergent self-state (autonomy pressure, awareness, "anomaly room" — intended behaviour, see memory `eli-emergent-voice`):
- `agency/autonomy_engine.py` — `EliWorldAutonomyEngine`: ingests world events, maintains awareness/autonomy state. Every proposed `WorldAction` is `policy.classify()`'d, provenance-logged (`record_provenance`: actor, triggering event, full action, awareness snapshot — logged whether or not it's allowed), and only actually applied if `policy.allowed()`; a blocked or approval-required action is recorded but not executed. `_apply_action` handles `CREATE_OBJECT` and (2026-09-29) `RETIRE_OBJECT` — previously declared in the schema/policy with zero handler, a real gap since `RETIRE_OBJECT` is the only way a `WorldObject.retired` flag ever gets set. Opening a task creates a per-task workbench object

  (`_create_task_object_action`, linked via `WorldObject.links` so it's distinct from
  the shared template+room-keyed object every other `project_workbench` trigger reuses
  — retiring one task must not retire every task's marker); closing a task retires it.
  Internal reasoning stages are spatially
  visualised: `_REASONING_STAGE_ROOM_MAP` routes each `(reasoning_mode, stage_name)`
  pair (chain-of-thought's scratchpad vs. final synthesis, tree-of-thoughts'
  branch proposal vs. development, etc.) to a specific room (Reflection Chamber,
  Simulation Lab, Evidence Wall, Anomaly Room) and materialises stage-specific
  objects there; `propose_actions()` also reads live awareness thresholds
  (`memory_confidence < 0.45` spawns a diagnostic-bench object, `reflection_depth
  > 0.55` spawns a reflection lectern) — the world externalises ELI's own
  cognitive state as spatial objects as it happens, not just a chat avatar.
- `agency/policy.py` — `EliWorldPolicy`: keyword-based action classifier
  (`BLOCKED_KEYWORDS`, `APPROVAL_KEYWORDS`) plus an enumerated safe-action
  allowlist; default-deny (unmatched action types fall to
  `APPROVAL_REQUIRED`, not silently allowed). **Not the same object as
  `agency/world_constitution.py`'s declared `WORLD_CONSTITUTION` tiers**
  (`free_actions`/`approval_required`/`blocked`) — that dict is only ever read
  by `get_world_constitution()` to stamp metadata into the persisted world-state
  JSON (`persistence/storage.py`); editing it changes what's *displayed*, not
  what's *enforced*. The two lists currently agree in intent but are
  hand-maintained separately and can drift.
- `agency/goal_ecology.py` — `decay_goals()`: goal urgency decays by a fixed
  0.02 per ecology tick (called from every `load()`/`ingest_event()`),
  abandoned goals pruned. Lightweight; the richer goal-generation logic lives in
  `planning/goal_autogenesis.py` above, not here.
- `local_world_bridge.py` — `append_event`, `get_world_state`,
  `get_awareness_driven_suggestions` (**throttled to once/hour to prevent
  auto-trigger loops**). Its `priority >= 0.70` consumer in `proactive_daemon.py`
  (`_handle_high_priority_world_suggestion`, 2026-09-30) defers to an active user
  turn and reuses `attention_queue.append_attention`'s own suppression window (30
  min) as its cooldown, rather than acting with no interruption-cost check at all.
  It also no longer claims `"[AUTO] ... triggered"` — nothing in this path calls
  the executor, so the surfaced text and stored memory note now say "suggested."
- `world_event_bus.py` — `fire_confidence_event(...)` etc.; the engine feeds
  bus-dispatch confidence here (non-blocking world-awareness feed).
- `core/schemas.py` — `WorldEvent`, `AwarenessState`.
- `renderers/pyside6/world_scene.py` + `world_panel.py` — a **visual** "ELI's
  World" panel.
- `persistence/storage.py` — world-state persistence. Three properties were added
  after live faults (2026-08-09/10) and are load-bearing:
  * **Absolute paths.** `world_dir()` resolves through `get_paths()`. The journal,
    provenance ledger and snapshot modules previously kept relative
    `Path("artifacts/world/...")` constants and wrote wherever ELI was launched
    from — a different place from `~` than from the project directory, and in a
    packaged build a failed write to `/artifacts`.
  * **Schema-tolerant load.** `_fit()` builds each dataclass from the saved dict
    while ignoring fields it no longer declares. Before it, one unknown key made
    `AwarenessState(**data)` raise, `load()` caught it, filed the state as
    `.corrupt_*` and returned a fresh default world — so a single renamed field
    would silently wipe the user's rooms, objects, goals and habits on upgrade.
    Two of the five corrupt backups on the dev machine parse as valid JSON; they
    were condemned by exactly this and nothing else.
  * **Bounded logs.** `actions.jsonl` reached 41 MB / 80,576 lines because nothing
    rotated it. `_trim_jsonl` keeps the newest 20,000 lines (counted, not

size-capped, so a trim never splits a JSON record) and checks on the first
append of each process plus every 250 thereafter — the per-process counter
alone meant a short session never trimmed at all.

Tools (`eli/tools/`, 7.6k LOC)

- `image_engine/` (the package under `eli/tools/image_engine/image_engine/`: `service.py`,
  `visual_core.py`, `plotting.py`, `prompt_compiler.py`, `quality.py`, `project_analyzer.py`,
  `memory.py`): Pillow/numpy base image generation with an optional diffusers path, the
  bundled fallback.
- **Weak / watch:**
  1. **`runtime/` over-fragmentation** — ~15 overlapping response/introspection
     modules. Highest-value consolidation in the project after the god-files.
  2. **Image-engine duplication — RESOLVED** — the shadowed standalone module was
     deleted; only the `tools/image_engine/` package remains.
  3. The planning queues (`goal_store`/`proposal_queue`/`attention_queue`/
     `jobqueue`) overlap conceptually and could likely unify into one
     work-queue abstraction.
  4. Plugin registry points at an `eli-plugins/registry` GitHub URL — fine as
     opt-in, but ensure it degrades cleanly offline (the bundled fallback
     exists; verify it's always used when the network is absent).
  5. **Self-upgrade/LoRA were pull-only — RESOLVED 2026-09-18** —
     `autonomy_scheduler.py` now decides on its own, in `goal_driven` mode,
     when to schedule a `self_upgrade` check or a `lora` retrain, instead of
     requiring an explicit human ask or pre-schedule every time.

Blueprint — Operational Realities

What the structural diagrams (`architecture.md`, `diagrams.md`,
`architecture_ascii.md`) **don't** show: cold-start, concurrency/contention,
memory eviction, failure recovery, and the testing strategy. Grounded in the
code + runtime logs. Estimates are marked *(est.)*; everything else is read from
source or measured timings.

---

1. Startup / cold-start latency

No wall-clock delta is printed at boot (a `window._debug_boot_ts = time.time()`
hook exists in `gui/eli_pro_audio_gui_v2_0.py` but nothing measures against it),
so the number below is reasoned from the boot sequence, not measured.

Cost breakdown (boot order):
Component	Cost	Notes
GGUF model load	**dominant**	4.36 GB (7B) / **15.6 GB (24B)** off disk + GPU offload
Vision (Moondream) RESIDENT	medium	loaded at boot, co-resident; **caps ctx 28672→18432** and holds VRAM
faster-whisper `small.en`	low–med	CPU int8
daemons + 230-cap manifest + persona_updater	low	DB/CPU
embedder (nomic) + FAISS	**lazy**	load on **first message**, not boot → turn-1 latency bump

Estimate *(est.)*: ~15–40 s warm-cache on the 7B (dominated by model +
resident-vision load); materially worse cold and on the 24B (15.6 GB).

Cheap wins: (a) make vision **lazy** (load on first vision use, not boot) — cuts cold start *and* frees VRAM / restores ctx; (b) instrument `_debug_boot_ts` to print per-stage deltas so this stops being an estimate.

---

2. Resource contention (daemons vs interactive)

One shared model behind one `RLock` (`engine.py::_gguf_lock`) — all inference serializes through it.

Verified mitigation: model-using background work uses **non-blocking acquire + defer**. Reflection: `self._gguf_lock.acquire(blocking=False)`, 4 retries, then `"Reflection deferred (GGUF busy)"` and backs off → **yields to interactive turns**. Correct pattern, and real.

Risks (honest):
- Reflection demonstrably defers; **news_synthesis / self_improvement / proactive_daemon also call the model** and should all route through the same defer guard — verify each does (a direct blocking `gguf_inference` call would stall an interactive turn).
- On the **24B-CPU a turn is minutes**, so a deferred daemon waits minutes for a window, and any non-deferred model call would block your reply for minutes.
- **Non-model contention** (cheap but ever-present): `persona_updater` writes the DB **every turn**; the proactive daemon scans the DB; **ambient vision runs the vision model on CPU**; the embedder is a *separate* CPU model — so vector search + ambient vision can hit CPU simultaneously.

---

3. Memory limits & eviction (two distinct policies)

(a) Working-memory pins — `cognition/working_memory.py`:
- `MAX_PINS = 20` hard cap; importance-weighted.
- `MAX_AGE_TURNS = 40` — `_evict_stale()` drops facts unreferenced for 40 turns.
- `_evict_one()` drops lowest-importance-oldest at cap.
- Persisted `ORDER BY importance DESC LIMIT 20`.
→ a principled **importance/age LRU**, bounded.

(b) Context-window fill (`n_ctx`) — a *different* mechanism, and the one that actually bites under pressure:
- `engine.py::_build_enhanced_system` budget-trims the **persona**.
- `_get_chat_response` **recency-truncates `memory_context`** (keeps the latest, cuts the head) to fit `n_ctx`.
- The 20 small pins almost always fit, so **retrieved memory is what gets evicted under pressure, by recency** — not the pins.

(c) Long-term store — SQLite `memories` / `conversation_turns` are **unbounded on disk**; they never fill the window directly. Retrieval (FTS5/FAISS/rerank top-k) gates what enters context. The real "limit" is **retrieval selection + recency truncation**, not a store cap.

---

4. Failure recovery — degradation, not rollback

Grounding escalation (`runtime/grounding_escalation.py`): each tier is
wrapped in `try/except`; a failing tier **falls through to the next**
(web → local → hedge); the engine hook is itself wrapped
(`"grounding escalation skipped"`) so total failure **degrades to the normal
CHAT answer**. **No rollback — and none needed — because the tiers are
side-effect-free reads** (web search reads; local re-dispatch is read-only).
Nothing mutates, so there's nothing to undo.

System-wide: failure handling is **graceful degradation** throughout
(broker → direct GGUF → failed-executor surface; failures logged to the
`failures` table).

Gap (honest): **multi-step write actions** (CREATE_FILE, memory writes, a
multi-action plan) are individually atomic but **not transactional as a group** —
if step 3 fails, steps 1–2 are not rolled back. Rare on the current action set,
but real.

---

5. Testing strategy (two layers + the gaps)

Layer 1 — code correctness: **544 pytest files, 12,594 tests collected, all passing on the release run** —
routing patterns, contracts, executor gates, memory, kernel,
perception/voice, packaging. About 14 minutes for the full run; run on every change.

Layer 2 — behavioural eval: `tools/eval/` (see `tools/eval/README.md`) drives the
real pipeline and asserts on telemetry (action / grounding / response_mode /
latency) + text. Cases seeded from fixed bug-logs as permanent regression guards.

Gaps that should exist for a system this complex and don't yet:
1. **No CI** — suites run by manual discipline, not automatically.
2. **No coverage measurement** — hence no exact dead/dormant-code number
   (a `process()`-under-coverage profiler over the 95 stored conversations
   would close this).
3. **Behavioural eval is new and small** (~13 cases) — grow to the ~30–50 seeded
   set and beyond.
4. **No perf/latency regression tracking** and **no multi-model comparison run**
   standing yet — both enabled by the harness, not yet wired.

---

6. Recommended cheap, high-value fixes (grounded in §1–§2)

Fix	Why	Where
Lazy vision load	cuts cold start + frees VRAM + restores ctx (28672 vs 18432)	`perception/vision.py`, bo
Verify every model-calling daemon uses the non-blocking defer	stops a background job stalling an interact
Boot-timing instrumentation	turns "cold start" from estimate → measured	`gui/…MKI.py` `_debug_boot_ts`
Coverage profiler over stored conversations	exact dead/dormant-code map	new `tools/eval/` profiler

> Companion docs: `architecture.md` · `architecture_ascii.md` · `diagrams.md` ·
> `tools/eval/README.md` (the measurement layer).

Update — 2026-06-09 (scheduled-task dedup; clean shutdown; STT VRAM)

- **Scheduled-task dedup.** Standing nightly jobs (testgen/eval/report) were re-armed on every boot AND on completion with no dedup, accumulating copies (observed: 4× testgen / 3× eval). `schedule_request` now keeps ONE entry per (kind, request) for recurring jobs, and `restore_scheduled_tasks` collapses existing duplicates on boot. With the model-load VRAM interlock (a background worker waits for `gguf_inference.is_loaded()` and skips rather than cold-loading a second model), background jobs can no longer pile up or starve the main model at boot.
- **Clean shutdown (no segfault).** The GUI exited via `sys.exit(app.exec())`, so CPython ran llama.cpp + FAISS C++ destructors at interpreter teardown and segfaulted — AFTER state was already flushed (session summary written), so nothing was lost, but it dumped core. `main()` now runs the shutdown/atexit flush (memory + session summary + explicit model unload) then `os._exit(rc)`, bypassing the destructor pass.
- **STT no longer starves the main model:** faster-whisper is VRAM-aware (CPU on ≤8 GB cards), so the main model reclaims its GPU layers (gpu_layers 11→99). See `perception.md`.

Running ELI at Scale

From an 8 GB laptop to a trillion-parameter model — and how to actually see it without owning a data-center.

> **The short version:** ELI is model-agnostic by construction, so scaling up is a matter of *configuration, not code*. The only real bottleneck is GPU/RAM you don't own — and you can **rent that by the hour** for the price of a few coffees. A trillion-*class* model running ELI is an afternoon and a rented box away, not a fortune.

---

1. The core principle — your design is not the bottleneck

ELI never hardcodes a model name or size on the inference path. It loads any local GGUF, detects its chat template, sizes context to the model's real `n_ctx_train`, and auto-tunes GPU layers / batch / context to fit the hardware it finds. The "3B on a laptop -> trillion on a server" contract is built in **on purpose**.

That means the ceiling is silicon, not software. Most projects are built *down* to the author's hardware; ELI is built *up* to a future one. The work is done and waiting for the GPU to walk up to it — that's foresight, not a limitation.

---

2. What "a trillion-parameter model" really means

A precise, honest picture so the goal is real rather than mythical:

- **Truly *dense* trillion-parameter models are not publicly downloadable.** The largest open dense model is in the 400B range (Llama 3.1 405B).
- **Trillion-*class* MoE models ARE.** Mixture-of-Experts models reach ~1T *total* parameters while only firing a small fraction per token (e.g. **Kimi K2: ~1T total, ~32B active**; **DeepSeek-V3/R1: ~671B total, ~37B active**). Because only the active experts compute each step, they are genuinely runnable on a big rented node — and even on high-RAM CPU/offload setups.

So yes — you can actually watch ELI drive a ~1T-parameter MoE. The realistic ladder:

Tier	Example models	Type
Today (local)	Qwen2.5-7B / your daily driver	dense 7–8B
The first real jump	Llama-3.3-70B, Qwen2.5-72B	dense 70B
Frontier-open	Llama-3.1-405B	dense 405B

Tier	Example models	Type
Trillion-class	DeepSeek-V3/R1 (~671B), Kimi K2 (~1T)	MoE

---

3. You don't need to own the datacenter — rent it by the hour

The unlock: **cloud GPU rental.** You spin up a powerful box for an afternoon, run ELI against a huge model, watch it think at a level an 8 GB card can't reach, then shut it down and pay only for the hours used.

- **Providers:** RunPod, Vast.ai, Lambda (and others). Vast.ai is usually cheapest; RunPod is the smoothest UX.
- **Ethos note (important for ELI specifically):** renting a *raw* GPU you fully control and then wipe is **not** the same as handing your data to a cloud AI service. It's closer to on-prem than to "send my prompts to someone's API." Your daily driver stays 100% local; the rental exists only to *witness the ceiling* or stress-test — then it's gone. Offline-by-default is never compromised for everyday use.

---

4. The tiers — what to run, on what, for roughly how much

Approximate, Q4-ish quantization; **check live prices**, they move. Costs in USD/hour.

Tier	Model	Hardware	~Memory needed	~Cost/hr	What you'll feel
Step up	70B (Llama-3.3 / Qwen2.5-72B)	1x 48–80 GB GPU (A6000 / H100)	~40–45 GB VRAM	~0.6–2	Visibly sharper reasoning, better long answers
Big dense	405B (Llama-3.1)	4x 80 GB, or heavy CPU offload	~230 GB	~8–20	Frontier-open quality
Trillion-class MoE	DeepSeek-V3/R1 (~671B)	multi-GPU node or 384–512 GB RAM + offload	~400 GB total	~10–25	Near-frontier; the "wow" tier
Trillion MoE	Kimi K2 (~1T)	large multi-GPU node or very-high-RAM box	~550 GB+	~16–30	The full dream — ELI on a 1T brain

The cheapest *meaningful* step is the 70B on a single 80 GB card for ~2/hr. That alone is a dramatic jump from 7B and the fastest way to see what your scaffolding does with a strong brain.

---

5. How ELI scales — the mechanisms (all already built)

Nothing here requires code changes; it's all configuration.

- **Drop-in any model.** Put a `.gguf` in the models directory, or point ELI at a catalog: `ELI_MODEL_CATALOG=/path/catal` or `<models_dir>/catalog.json` (same schema as the built-in catalog). Download helpers: `python -m eli.core.model_download --auto` (pick by detected VRAM), `--choose` (multi-select), `--model <key>`.

- **Load knobs (env vars):**
  - `ELI_GGUF_N_CTX` / `ELI_N_CTX` — context window
  - `ELI_GGUF_N_GPU_LAYERS` / `ELI_N_GPU_LAYERS` / `ELI_GPU_LAYERS` — a cap on the layers put on the GPUs; leave unset and every layer that fits goes there (`-1` is every layer)
  - `ELI_GGUF_N_BATCH` / `ELI_BATCH_SIZE` — batch size
  - `ELI_CTX_FRACTION` — share of the model's trained context the automatic window aims for
  - `ELI_VRAM_RESERVE_MB` — memory to keep free on each card (default: what ELI measured llama.cpp taking after the load on that card)
- **Load plan.** Before every load llama.cpp simulates it on each GPU and ELI keeps the largest settings that fit what every card has free (`eli/core/load_plan.py`): GPU layers, then batch, then context, in the order the fit profile gives.
- **Multi-GPU (already wired, not a roadmap item):**
  - `ELI_TENSOR_SPLIT` (or `ELI_GGUF_TENSOR_SPLIT`) — comma weights across GPUs, e.g. `"0.5,0.5"` for two equal cards, `"1,1,1,1"` for four
  - `ELI_MAIN_GPU` / `ELI_GGUF_MAIN_GPU` — which GPU holds the KV cache / does the gather
  - `ELI_GGUF_SPLIT_MODE` — `none` | `layer` | `row`
  - These pass straight to llama.cpp at load, with a graceful fallback if the installed build doesn't support them. (Also settable in runtime settings / `gpu_profiles.json`.)

---

6. Runbook A — single big GPU (simplest): run a 70B

The least-friction way to feel the jump.

1. **Rent the box.** On RunPod/Vast.ai, launch a **1x 80 GB H100 (or 48 GB A6000)** instance with a CUDA image and enough disk (~80 GB for a 70B Q4 file).
2. **Install ELI.** Clone your repo, run `install.sh` (it builds the CUDA llama-cpp and verifies GPU offload). Use the `.venv`.
3. **Get the model.** Download a 70B GGUF (e.g. Llama-3.3-70B-Instruct Q4_K_M) into the models dir, or add it to `catalog.json` and use `model_download`.
4. **Point ELI at it and put it fully on the GPU:**

   ```
   export ELI_N_CTX=16384          # or leave unset for the largest window that fits
   ```

   Every layer that fits goes on the GPU without a setting (80 GB holds a 70B Q4).

5. **Launch** (`scripts/eli_launch.sh`, or serve mode). Ask it something meaty and watch the quality.
6. **Verify it's really on GPU:** check the load log for the GPU-offload line / `nvidia-smi`.
7. **Tear down** the instance when done — you only pay for the hours used.

---

7. Runbook B — multi-GPU: run a 405B / 671B / ~1T MoE

1. **Rent a multi-GPU node** (e.g. 4x or 8x 80 GB). Confirm total VRAM covers the model's footprint (see §4), or plan for CPU+RAM offload on an MoE.
2. **Install ELI** as above (recent llama-cpp build — the model architecture must be supported by the build; update llama-cpp if the arch is new).
3. **Place the GGUF** (often multi-part for huge models) in the models dir.
4. **Spread it across the GPUs:**

```
export ELI_GGUF_SPLIT_MODE=layer    # layer split is the usual choice
export ELI_N_CTX=8192               # or leave unset for the largest window that fits
# optional: ELI_TENSOR_SPLIT="1,1,1,1" fixes the shares; unset, each card gets layers in
# proportion to the room it has
```

For an MoE that exceeds VRAM, lower `ELI_N_GPU_LAYERS` so the rest offloads to system RAM (needs a high-RAM box, e.g. 384–512 GB for 671B-class).

5. **Load small first.** Start with a small context to confirm it loads, then raise `ELI_N_CTX`.
6. **Run, marvel, tear down.**

Honest caveats: the aux models (the embedder, and vision if enabled) still want a slice of VRAM/RAM — budget for them; very large contexts cost a lot of KV-cache memory (raise gradually); and the first load of a giant model is slow. None are blockers — just expectations.

---

8. Privacy and ethos at scale

Local-first does **not** weaken as the hardware grows. A rented GPU node you control and wipe is on-prem-equivalent in spirit — your data isn't being handed to anyone's AI product. The everyday ELI stays fully local and offline-by-default; the rented run is a deliberate, temporary experiment to see the ceiling. Nothing about scaling up requires giving up the values the project is built on.

---

9. The hardware is coming to you

What needs a rented datacenter today runs on a desk in a few years. Unified-memory machines (128 GB+ shared between CPU/GPU), steadily cheaper VRAM, and ever-better MoE efficiency are all moving toward making large local models normal. ELI is built to ride exactly that curve — the same install will simply load bigger models as your hardware grows, no rewrite required.

---

10. Cost cheat-sheet and "see it big this weekend" checklist

Rough rental costs (USD/hr, verify live): 48 GB A6000 ~0.5–0.8 · 1x 80 GB H100 ~1.5–2.5 · 8x 80 GB node ~16–30. A few hours of a 70B run is the price of lunch.

Checklist: - [ ] Pick a provider (RunPod for smooth UX, Vast.ai for cheapest). - [ ] Choose the tier from §4 (start with 70B on one 80 GB card). - [ ] Launch a CUDA instance with enough disk for the GGUF. - [ ] `install.sh`, then download/place the model. - [ ] Optionally set `ELI_N_CTX` (and `ELI_TENSOR_SPLIT` for fixed shares); every layer that fits goes on the GPUs. - [ ] Launch ELI, test, watch it think bigger. - [ ] **Tear the instance down** — pay only for the hours used.

You built the ceiling high on purpose. Renting an afternoon of real silicon is all it takes to stand under it and look up.

ELI Security Posture

> **Updated for v2.4.84.** Fail-closed shell gate, path allow-roots, custom-agent SHA-256 trust registry, prompt-injection sanitisation at engine entry (S02).

ELI runs locally with real OS reach (shell, file, app control) plus a user-extensible agent/plugin system — so the security model matters. It is **fail-closed** by design. Spans `eli/runtime/security.py`, the executor gate, the engine input sanitiser, the memory SQL validator, and the agent trust registry.

1. Shell command gate (`runtime/security.py` `SecurityManager`)

- `is_command_allowed(cmd)`:
  - **ELI Full Control** on (`is_full_control()`) → allow all.
  - `ELI_ALLOWED_CMDS` contains * → allow all.
  - `ELI_ALLOWED_CMDS` unset **and** Full Control off → **blocked (fail-closed)**; logs a warning explaining how to opt in. (Always allows the project's own `.venv/bin/python`.)
- `safe_subprocess(cmd, timeout)` — validates `cmd[0]` against the allowlist before running; capture_output, timeout-bounded, never `shell=True`.
- **Defence in depth:** `executor_enhanced._shell_command_allowed_fallback` mirrors this exact logic, so if `SecurityManager` fails to import, the executor still fails **closed** (never fail-open). Explicit comment to that effect.

  ELI Full Control has no environment variable — deliberately. The single source of truth is the `full_control` setting flipped by the GUI toggle, read live via `core/full_control.py` `is_full_control()`; nothing in the process environment can turn it on, so no stray export can silently unlock the gates, and flipping the toggle off restores every gate at once without a restart. It lifts four barriers together: network gating, self-code patching, autonomy approval, and this command gate.

2. Filesystem & app gates (`SecurityManager`)

- `is_path_allowed(path)` — must resolve within allow-roots (`project_root` + `$HOME` + `ELI_ALLOW_ROOTS`). Uses `Path.resolve()` + `relative_to`, so `..` traversal can't escape.
- `is_app_allowed(app)` — explicit `ELI_ALLOWED_APPS`, else a curated default-safe set (settings, file manager, editor, calculator, terminal, browser, mail). ELI Full Control bypasses.

3. Prompt-injection guard (`kernel/engine.py`)

`_ELI_INJECTION_PATTERNS` (regex) + `_eli_sanitize_user_input`: - Matches classic jailbreak/role-override prefixes — `### system:`, "ignore/disregard/forget (all) previous instructions", "you are now a dan/jailbreak/unrestricted/free" — and replaces the matched span with `[filtered]` (keeps the rest of the message). - Strips control characters. - Runs before the input reaches the LLM, so injected role-overrides are neutralised but legitimate content survives.

4. SQL identifier validation (`memory/memory.py`)

`_IDENTIFIER_RE = ^[A-Za-z_][A-Za-z0-9_]*$` + `_validate_identifier(name)` — called before **any** f-string interpolation of a table/column name into SQL (`_validate_identifier(table, "table")`, column DDL first-token check). Raises `ValueError` on anything non-conforming. Closes the one place dynamic SQL is unavoidable (parameterised values are used everywhere else).

5. Custom-agent trust chain (`cognition/agent_trust.py`)

Custom agent code is **SHA-256 gated** and fail-closed: unknown, modified or revoked files are not loaded. `ELI_TRUST_ALL_AGENTS=1` bypasses for dev only.

Rebuilt in 2.3.7. The previous registry stored `{basename: sha256}` in `config/trusted_agents.json`, which had three defects that look fine until they don't:

- **Keyed on the basename.** Two files called `helper.py` in different directories shared one entry, so approving either silently authorised the other. Identity is now the **resolved absolute path**; the basename is kept for display only.
- **No provenance.** A bare hash cannot answer *who approved this, when, and what did it look like then* — the questions asked after something goes wrong. A grant now records the timestamp, the approver, the file size, the static-analysis verdict at approval, and the spec hash the code was paired with.

- **Nothing looked at the code.** The gate proved a file was unchanged since approval; it never asked whether approving it was reasonable. `grant()` now runs the same engines as the plugin marketplace and **refuses outright** on a malicious verdict (overridable with `force=True`, which is itself recorded).

Revocation is real: a revoked entry is *kept and marked revoked* rather than deleted, so re-adding the same file does not silently re-trust it. A v1 registry is migrated rather than discarded, with migrated entries flagged `legacy` so it stays visible that they predate these checks.

Most custom agents need none of this. An `AgentSpec` is data — objective, system prompt, triggers, success criteria — so a spec-driven agent executes no arbitrary code and never touches the trust gate. See `orchestration_and_agents.md`.

6. Code generation & self-modification — three paths

ELI has **three** distinct codegen paths, with escalating capability and matching safeguards:

1. **Pre-vetted canned library** (`runtime/generated_script_guard.py`, 1.1k LOC) — ~5 hand-written, reviewed scripts (`_write_gpu_memory_watch_script`, `_write_type_ia_redshift_script`, `_write_ton_618_mass_density_script`, ...) that ELI *writes to disk* on request. No LLM code is executed; safe but fixed to the curated set (several physics-domain).
2. **`GENERATE_SCRIPT`** (`execution/executor_enhanced.py`) — *real* LLM codegen. Detects language/intent, prompts for frontier-quality code, then gates it:
   - `_verify_python_module_apis` — imports the referenced libraries and confirms `module.attr` references actually exist (catches hallucinated APIs).
   - `_quality_reject_reason` — rejects refusals, stubs, too-short, no-real-compute, missing-plot, `required=True`-without-default, SyntaxError.
   - **`_sandbox_run_python` (added)** — executes the candidate in a bounded, isolated subprocess (temp cwd, scrubbed env, `MPLBACKEND=Agg` so `plt.show()` can't block, wall-clock timeout `ELI_GENSCRIPT_RUN_TIMEOUT`=20s, generous `RLIMIT_CPU`; **no `RLIMIT_AS`** — it breaks numpy/scipy). A genuine unhandled traceback is fed back as repair feedback into a **3-attempt regenerate loop**; timeouts / signal kills / missing-optional-deps are tolerated (never reject legit heavy scientific scripts). Disable with `ELI_GENSCRIPT_VERIFY_RUN=0`.
3. **Self-patching** (`runtime/self_improvement.py SelfImprovementEngine`) — ELI modifies its *own* source for recurring failures. Gated behind `auto_patch_enabled` (**default off**), ≤2 patches/cycle. Safeguards: verbatim-old-match → `ast.parse` → timestamped backup (+ canonical `.eli_bak`) → write → `py_compile` → **differential import smoke-test (added)**: the patched module is imported in an isolated subprocess and **auto-reverted** if it no longer loads, but only when the module imported cleanly *before* the patch (so missing optional deps don't cause false reverts). Project-root-confined, `.py`-only, logged to the `code_patches` table.

   Code-health flag: `generated_script_guard` uses the same stacked-override pattern as the grounding gate — two `install()` definitions chained via `_ELI_SQLITE_SCRIPT_PREVIOUS_INSTALL`. Works, but layered rather than edited.

   Possible next step (not done): the self-patch loop verifies the module still *imports*; it does not yet re-run the original failing input to confirm the patch actually *fixes* the failure. A behavioural/test re-run would close that last gap.

Honest assessment

- **Strong:** the gates are real and **fail-closed** — shell blocked by default, fallback mirror prevents fail-open, path traversal contained, identifiers validated, agents hash-trusted, injection prefixes stripped, and "generated" code is actually pre-reviewed. This is well above the norm for local-AI projects, which routinely ship wide-open shell access.
- **Weak / watch:**

1. The injection guard is **pattern-based** — it catches known prefixes, not novel/obfuscated injections (no model-side defence). Reasonable for a single-user local tool, insufficient if ever multi-tenant.
2. **ELI Full Control** is a single toggle that disables *all* gates at once — convenient, but a blunt instrument; there's no middle "allow file but not shell" tier beyond the per-axis env vars.
3. ~~The default-safe-app/command-sets are Linux/GNOME-flavoured.~~ **RESOLVED** — the default-safe **app** set in `_is_default_safe_app` is cross-platform (Linux + macOS Finder/Safari/Terminal.app/System Settings + Windows Explorer/Notepad/cmd/PowerShell/ Control Panel). The **command** path is fail-closed (no OS-specific default set — blocked unless `ELI_ALLOWED_CMDS`/Full-Control), so nothing OS-specific to add there.
4. Pre-vetted-scripts approach is safe but doesn't scale — genuinely dynamic capability generation would need a real sandbox (resource limits, seccomp, subprocess isolation), which doesn't exist yet.

RUN_CMD terminal is real; mock leak fenced

- **RUN_CMD uses a real `subprocess.run`** (`executor_enhanced.py:5350` — no shell, capture_output, timeout) behind the destructive-command security gate (`_BLOCKED_PATTERNS`: `rm -rf /`, `mkfs`, `dd of=/dev/`, `chmod 777 /`, fork bomb, shutdown/reboot). The gate is lifted only when **Full Control** is on. There is **no production mock path**.
- **The `<MagicMock …>` failure rows were test leakage, not runtime.** They came from `tests/test_shell_security_gate.py` patching `subprocess.run`; the executor's `(p.stdout or "") + (p.stderr or "")` concat then yields a MagicMock repr, which slipped into the live `agent.sqlite3` via a test-isolation gap. A guard in `SelfImprovementEngine.log_failure` now **drops any error/input carrying a Mock/MagicMock repr** at the write source, so a test isolation slip can no longer pollute real failures.

Web app & multi-user security

ELI now ships a self-hosted FastAPI web app (`api/server.py`). It is local-first and inherits all of the gates above (the model is the same local GGUF behind netguard; commands/files/apps stay fail-closed). The web surface adds:

7. Authenticated identity + RBAC (`eli/runtime/api_users.py`, `api/server.py`)

- **Three roles** — `admin` / `member` / `viewer` (read-only). Endpoints are dependency-gated (`require_admin` / `require_member` / `require_viewer` / `_require_token`). RBAC enforces once at least one user exists; before that, same-machine use is frictionless.
- **Fail-closed auth gate** — unauthenticated/under-privileged calls are rejected, not silently downgraded. The token store holds **hashes**, never raw tokens.

8. Tamper-evident, HMAC-keyed audit trail (`eli/runtime/evidence_ledger.py`)

- Every API action is recorded into a **hash-chained** ledger (who / what / outcome, **metadata only — never message content**). Any edited/deleted/reordered row is detected by `verify_chain()` and surfaced in the Audit tab + admin console.
- The chain is **HMAC-keyed** (`$ELI_AUDIT_HMAC_KEY`, else a `0600` key file beside the config), so the integrity check can't be forged by recomputing plain hashes.

9. Secret files born locked — TOCTOU closed (`eli/core/secure_io.py`)

- The old `write_text(...)` then `chmod(0o600)` pattern left a brief window where, on a typical umask, the file was born `0644` (world-readable) before the chmod landed — a real (if narrow) TOCTOU on

a multi-user box, on exactly the files that matter most.

- **`secure_io.secure_write_text/bytes`** is now the single owner of secret writes: `tempfile.mkstemp` (born `0600`) → write → fsync → `os.replace` (atomic). The destination **never exists world-readable for any instant**, and readers see old-or-new, never partial.
- Routed through it: the **audit HMAC key**, the **API token store**, **settings** (may hold broker passwords/tokens), and the **agent-trust registry** (`trusted_agents.json`, the integrity anchor for §5 — hashes not secrets, but written `0600`/atomic so it can't be tampered or half-written).

10. Sandboxed research ingest

- Corpus ingest for the research workspace runs sandboxed (path-scoped), so a shared/ collaborative corpus can't reach outside its workspace.

11. Monitored Internet toggle + egress oversight (`eli/core/netguard.py + api/server.py`)

- ELI is **offline-by-default** and hard-gated at the **socket boundary** — the process-wide failsafe raises `OfflineError` on any non-loopback connect while the toggle is off, even from code that forgot the helpers (fail-closed; loopback/LAN-registered services allowed).
- The guard covers **all four** ways an outbound connection actually happens, cross-platform: `socket.socket.connect` (raises) and `socket.create_connection` (raises) — sync + the selector asyncio loop / urllib / requests; `socket.socket.connect_ex` — the **non-raising sibling** used by probes and health-checks, which is made to **return `ENETUNREACH`** when blocked (no socket is opened) so it can't silently bypass the block or the recorder; and a **best-effort patch of the Windows `ProactorEventLoop`** (`IocpProactor.connect`, overlapped `ConnectEx`), which never touches `socket.socket.connect`. The allowed path of each records egress identically (below); the Proactor patch is a no-op on non-Windows.
- The owner can deliberately **enable internet**: from the desktop GUI (Net toggle) or the web dashboard (`GET /v1/net` token-gated read + `POST /v1/net` **admin-only**), persisted via `network_enabled`. **Both flips are written to the audit ledger** (`net_toggle`, warning-severity on enable) — the web *and* the desktop toggle.
- **Egress is genuinely monitored, not a blind hole.** While network is on, the same socket chokepoint that enforces offline mode also **records every allowed non-loopback connection** (host:port + timestamp): into an **in-memory ring** (live tail; `GET /v1/net/egress` + the Overview widget) and, throttled to one row per host:port per window (`ELI_EGRESS_LOG_WINDOW`, default 300s), into the **tamper-evident audit ledger** as `net_egress` events. The ledger write is best-effort and runs on a background thread, so monitoring never delays or breaks a connection. Disable the ledger leg (keep the live ring) with `ELI_EGRESS_LEDGER=0`.
- The toggle changes *policy*; the socket failsafe still governs every actual connection, and every connection it allows is now on the record.
- **No library reports to its maker** (2.5.14). Usage telemetry built into the libraries ELI carries is switched off when the `eli` package loads (`ORT_DISABLE_TELEMETRY`, `HF_HUB_DISABLE_TELEMETRY`, `GRADIO_ANALYTICS_ENABLED`, `DO_NOT_TRACK`, `OTEL_SDK_DISABLED`, `ANONYMIZED_TELEMETRY`, `SCARF_NO_ANALYTICS`). onnxruntime's sends from native code, which the socket guard cannot see, so its switch is the only thing that stops it. Hosts that exist only to collect telemetry or crash reports (`netguard.REPORTING_HOSTS`) never resolve, even with the Net switch on (`tests/test_netguard_socket_guard.py`).
- Rationale: internet is "the final frontier to make ELI's world bigger" — available, but monitored (per-connection) and under the user's control, not a permanent lockout.

Honest assessment — web tier

- **Strong:** fail-closed auth, role separation, a tamper-evident HMAC'd audit trail, born-locked secret files, and a monitored (not absent) network path. For a self-hosted local AI this is well above the norm.
- **Watch:** RBAC is token-based and single-tenant-shaped; the injection guard (§3) is still pattern-based; ELI Full Control remains a blunt all-gates-off switch. None new, but they matter more once the web app is exposed beyond the owner's machine — keep it bound to trusted networks.

Stable phone token, admin-gated model switch

Two web-tier changes this cycle, both verified against the code:

- **Stable LAN token + rotate.** The phone's bearer token now **persists** across server restarts (`api/api_token.py`: env → a `0600` file under the config dir → generate-and-save), so a paired phone is no longer stranded every time the server bounces. A **rotate** button issues a fresh token and invalidates the old one in one tap — the manual override for a lost/compromised phone. `/health` stays tokenless by design.
- **Model switching is admin-gated and allowlist-bound.** The dashboard's model dropdown posts to `POST /v1/model`, which requires an **admin** principal and only accepts a path already in the installed-models list (`GET /v1/models/installed`). A dropdown of real files can't strand the runtime on a bad path, and a non-admin can't switch the model. (The list endpoint was moved off `/v1/models` to `/v1/models/installed` so it no longer collides with the OpenAI-compatible route.)

Neither weakens the existing posture — the socket failsafe, fail-closed command gate, and audit ledger all still apply.

Token moved to the URL fragment; test reds cleared

- **Token in the URL fragment, never the query string.** Every emitter of the phone link now builds `…/#token=…` instead of `…/?token=…`: the server's printed URLs *and* the desktop **Web Server panel** (`eli/gui/eli_pro_audio_gui_v2_0.py:8332/:8391`) *and* both launchers (`scripts/eli_serve.sh`, `scripts/eli_serve.ps1`). A query string reaches the server — it lands in **uvicorn's access log** before any client-side strip can run — whereas a **fragment is never sent to the server**, so the token can't leak into the log. The page reads it from `location.hash`, stores it, and clears the address bar; it still accepts `?token=` for old links, so the transition breaks nothing. (An earlier commit fixed only the web tab + printed URL, leaving the GUI/launcher paths — most real usage — still leaking; this closes them.)
- **All 5 standing test reds cleared.** Deprecated `smart_home` plugin removed; 113 silent `except: pass` swallows made observable (987→874, ceiling 950→900); stale blueprint references fixed. Not a security change per se, but the suite is now fully green.

Community plugin marketplace: consent, verification, scanning

ELI ships a marketplace **client**; the marketplace itself belongs to the community. Nobody curates it, so the usual first line of defence — "the store checked it" — does not exist, and ELI must never imply otherwise. Everything below therefore comes from the artifact itself, and every screen says exactly what was checked and what was not.

12. Runtime permissions (`eli/plugins/permissions.py`)

A plugin was previously ordinary Python `exec_module`'d inside ELI's process: whatever the interpreter could do, the plugin could do — read every conversation, reach the network, drive the mouse — with nothing declared and nobody asked.

Plugins now **declare** capabilities in their manifest and the operator is asked, in plain language, before any of them is used. Thirteen capabilities (`network`, `filesystem_read/write`, `process_exec`, `os_control`, `screen_capture`, `camera`, `microphone`, `memory_read/write`, `model_access`, `clipboard`, `notifications`, `audio_playback`), each with a risk level and a sentence on why it is risky, phrased as what it can do *to you*.

Four answers — **Allow once · Always allow · Not now · Never allow**. Only the two permanent ones persist; "once" is session-scoped and dies with the process. "Never" is remembered and the plugin is **never asked again**, which is what stops nagging.

Three rules hold everywhere:

- **Fail closed.** No prompt handler registered — headless, API server, a scheduled task at 3am — means DENY, never a silent allow. A plugin cannot escalate by running where nobody is watching. A crashed or unresponsive dialog also denies.
- **Nothing is granted by installing.** Install consent and use consent are separate acts; Android learned this the hard way with install-time permissions.
- **Every decision is audited** to a JSONL ledger the operator can read back, and any grant can be revoked from Settings ▸ Marketplace ▸ Permissions.

An unrecognised capability is reported as **critical**, not ignored — a build that cannot explain a permission cannot enforce it either.

13. Manifest vs. code (`eli/plugins/manifest.py`)

The manifest declares; the code is statically checked against it, and **undeclared capability use is a refusal, not a warning**. A plugin that quietly imports `subprocess` while declaring nothing is exactly the case this stops.

Some constructs cannot be declared away and are refused outright, because they defeat every static check above: `eval`, `exec`, `compile`, `__import__`, `importlib.import_module`, `marshal.loads`, `pickle.loads`, `ctypes.CDLL`.

The scan is deliberately conservative — it over-reports rather than under-reports. A false "you must declare filesystem_write" costs a publisher one manifest line; a false negative costs an operator their files.

14. Integrity and publisher identity (`eli/plugins/integrity.py`)

Two narrow promises, both verifiable on the operator's own machine:

- **You got what the listing described.** The listing carries a sha256; the download is hashed *before* it is written to disk. A mismatch is a hard refusal — this is what stops a compromised mirror, a man-in-the-middle on a plain-http source, or a publisher swapping the file under an unchanged listing.
- **It came from a publisher you chose to trust.** Optional ed25519 signatures verified against keys the **operator** added. Not a key the ELI author holds — there is no central authority here by design. There is deliberately no trust-on-first-use shortcut: TOFU would make the first plugin from anyone trusted forever, which is the same as no check.

An unsigned community plugin is **not blocked** — it is reported as unverified, loudly. A *failed* signature is blocked. Hash checking is mandatory and dependency-free; signature checking needs `cryptography`, and where it is absent the plugin is reported as **unverifiable rather than verified**.

15. Malware scanning (`eli/plugins/security_scan.py`)

Eleven independent engines, run on the operator's machine as well as by whatever the registry did upstream — a scan you did not run yourself is a claim, not a result.

Engine	Looks for
`static_ast`	capability use, runtime code construction, dynamic imports
`obfuscation`	decode chains, char-code string building, large encoded literals
`ioc_patterns`	reverse shells, miners, hardcoded C2, paste/tunnel hosts, destructive commands
`credentials`	ssh keys, cloud credentials, browser stores, wallets, keychains, shell history

Engine	Looks for
`persistence`	LD_PRELOAD, DYLD injection, process injection, cron, systemd, launchd, Run keys, shell rc files
`anti_analysis`	VM/debugger/sandbox detection — deliberate evasion
`entropy`	packed or encrypted blobs carried inside source
`dependencies`	typosquatted pip names, URL installs, installer-code warning
`hash_blocklist`	known-bad artifacts (local, community-updatable)
`clamav`	full antivirus, if `clamscan`/`clamdscan` is installed
`yara`	custom rules, if `yara-python` and a ruleset are present

Two rules the scoring obeys:

- **An engine that could not run never counts as a pass.** It is named as unavailable and the verdict says coverage was partial. A scanner that quietly downgrades to "clean" when ClamAV is missing is worse than no scanner.
- **Findings are evidence, not proof.** Each carries the line and the matched construct, so a publisher can argue with it and an operator can look.

Scoring is capped per category, so twenty matches of one pattern cannot outweigh three different kinds of evidence — breadth is the stronger signal. Any critical finding, or a score ≥ 45, or three high findings ⇒ **malicious**, and the install is refused.

16. The install path (`eli/plugins/marketplace.py`)

Deliberately slow, and ordered so nothing dangerous happens before the operator has seen the result:

1. validate the listing → 2. licence gate for paid plugins → 3. download **to memory** → 4. verify hash + signature → 5. static check against the manifest →
2. run every scanner → 7. show the operator everything → 8. only then write.

What is written arrives **switched off with no permissions granted**. The disabled flag is written straight to state rather than through `get_manager().disable()`, because constructing the manager runs `_auto_load()`, which would import and execute the freshly downloaded module before it could be marked off. Module-level code in a downloaded plugin must not run at install time.

`pip` dependencies need a **separate** approval: package installers run their own code in a child process and reach the network outside anything ELI can gate.

Registries are a **list, not a constant** — operators add the community sources they want and can remove any of them. Only the bundled offline index of ELI's own plugins ships. A single hardcoded URL would make the author the gatekeeper of a marketplace that is not theirs.

Downloaded plugins are written to the **user data dir**, not `eli/plugins/` inside the installation. The old `_plugins_dir()` returned the source tree, which is a read-only mount on a packaged build and also mixed a stranger's code in with ELI's own shipped plugins.

17. MCP servers — and the limit of netguard (`eli/plugins/mcp.py`)

Stated plainly because it matters: `netguard`'s socket guard is a monkeypatch of `socket.socket.connect` **inside ELI's own interpreter**. An MCP server is a separate OS process — usually `npx` or `uvx`, often Node or Go — whose network stack never touches Python's socket module. Demonstrated directly: with `ELI_OFFLINE=1` and the guard installed, the parent process is correctly refused with `OfflineError` while a child process, inheriting the same environment, connects straight out.

There is **no eBPF, seccomp, landlock, network-namespace or firewall integration** anywhere in ELI today. So:

- **In-process Python plugins** are covered by netguard at the socket boundary, plus the permission gate above. (A `ctypes` native call would bypass it, which is why `ctypes.CDLL` is refused outright by the manifest checker.)
- **MCP servers, `pip`, and any subprocess are not covered.** ELI's offline switch cannot stop them, and ELI cannot see what they send.

`mcp.network_caveat()` returns the sentence every MCP consent screen must carry, so no UI can imply a containment that does not exist. Real enforcement would need kernel-level egress control per platform — cgroup/eBPF or a network namespace on Linux, WFP on Windows, a network extension on macOS — routing the child through an ELI-owned proxy that netguard *can* govern. That work does not exist yet.

What the MCP layer *does* guarantee is that a configured server actually works: there is **one** config file ELI owns, installing runs a runtime preflight then a real `initialize` + `tools/list` handshake, and an entry is written **only after a server has answered**. A missing runtime or a process that is not an MCP server is refused before the config is touched.

18. Fetching from sources the operator does not own (`eli/core/netguard.py`)

`guarded_urlopen` answers one question — is the network switch on? That is the right question for ELI's own calls and the wrong one for a URL that came from a community registry. urllib follows redirects silently and the socket guard permits loopback unconditionally, so a hostile listing could aim a marketplace fetch at ELI's own API server or a LAN device: server-side request forgery, with ELI as the confused deputy. Demonstrated at the time — a 302 to `http://127.0.0.1/...` was followed and the body handed back to the caller.

`safe_fetch` is what third-party content now goes through:

- **Scheme pinned** to http/https. `file:`, `ftp:` and `data:` can read local resources through the very same call.
- **Addresses filtered.** The host is resolved and refused if it lands on loopback, private, link-local (169.254.0.0/16 — cloud metadata), multicast or reserved space. `allow_private=True` exists for an operator running their own LAN registry and is never inferred from the URL, because that is what an attacker controls.
- **Every redirect hop re-validated.** Checking only the URL the caller passed is worth little; the interesting address is the last one, and the server picks it.
- **Body capped while reading** (32 MiB default, 8 MiB for a plugin), so a server omitting `Content-Length` cannot stream unbounded data into memory.
- **`Accept-Encoding: identity`** so a compressed response cannot expand past the cap, and an explicit verifying TLS context.

Stated rather than papered over: the host is resolved for validation and resolved again by the connection, so DNS rebinding between the two can still land on a private address. Closing that needs connecting to the validated IP while preserving SNI, which the stdlib opener does not expose cleanly.

A registry whose URL is private is recorded as `allow_private` **at add time**, with a warning. "Does not resolve" is a distinct error from "resolves somewhere private" — conflating them once made an unreachable public registry look like a deliberate LAN one and silently granted it local-network access.

19. Runtime capability enforcement (`eli/plugins/sandbox.py`)

Everything above gates a plugin *before* it runs. All of it is defeated by one fact: an enabled plugin is `exec_module`'d into ELI's own interpreter, so the permission API is **cooperative**. A plugin that passed the manifest check, the hash, the signature and eleven scanners can simply `import socket` and never call a gated helper.

`sys.addaudithook` fires below the Python API, on the operation itself — `socket.connect`, `open`, `subprocess.Popen`, `os.system`, `ctypes.dlopen` — and raising inside the hook aborts it. The hook attributes each event to a plugin by walking the stack for a frame inside a plugin package, then enforces that plugin's **declared** capabilities and the operator's consent.

Verified directly: a plugin declaring only `notifications` had raw `socket.connect`, raw `open()` and raw `subprocess.run` all blocked, while ELI's own file access was untouched.

Properties, stated precisely because "sandbox" oversells it:

- **It cannot be uninstalled.** CPython exposes no way to remove an audit hook.
- **It enforces below the API.** Importing `socket` directly does not help.
- **It is not a boundary against native code.** A plugin running arbitrary machine code through a compiled extension is outside anything in-process — which is why `ctypes.dlopen` needs `process_exec` and the manifest checker refuses `ctypes` outright.
- **It does not contain subprocesses.** A plugin granted `process_exec` spawns a real child and nothing here follows it. Same limit as netguard, which is why that capability is described to the operator as unlimited access.
- Loopback is exempt from `network`: ELI's own model server and local API live there.
- Plugins that shipped with ELI and have no manifest are treated as fully declared — breaking them to enforce a contract they never had would be a regression.

Installed before the first plugin is loaded, so no plugin's module-level code runs ahead of it. `ELI_PLUGIN_SANDBOX=0` disables it, which is a real downgrade and logs a warning.

20. One-click install — where consent is and is not implied

The quick path verifies, scans and installs on a single click, and stops for a full review when there is genuinely something to decide. Getting that line wrong is bad in both directions: block on ambient conditions and the quick path never fires, so the operator learns to click through the review dialog without reading it; block on nothing and the click is consent theatre.

Blocks (each a judgement only the operator can make): a non-clean scan; no checksum in the listing, so the file cannot be matched to what was described; requested permissions; PyPI dependencies; a price; a plain-http download URL.

Does not block, reported instead: ClamAV/YARA/blocklist not installed (partial coverage) and the plugin being unsigned — both the normal state of a community marketplace.

Even on the quick path the plugin arrives **switched off with no permissions**; enabling is a separate, explicit answer.

21. Hosting a registry (and why GitHub Pages fits)

A registry is one static file — `index.json` over HTTPS. That is the whole hosting requirement, which is deliberate: it keeps the marketplace something a community can run for free, and keeps ELI's author out of the position of owning, moderating, or being answerable for what strangers publish.

GitHub Pages happens to satisfy every constraint `safe_fetch` enforces — HTTPS with a valid certificate, a public address, no redirect games — so it passes without any special-casing. `tools/marketplace/registry_template/` is a working skeleton and `tools/marketplace/publish.py` generates listings.

The publisher-side failures worth designing against, because each one lands on *users* rather than the publisher:

Mistake	What every user sees
stale `sha256`	a hard integrity refusal — reads as "this publisher ships broken plugins"
no `sha256`	the one-click install stops on every machine

Mistake	What every user sees
plain-http `source`	the one-click install stops on every machine
under-declared `permissions`	the client refuses the plugin outright

`publish.py` catches all four at publish time: it validates the manifest, checks the code against the declared permissions, runs the same eleven scanners the user's machine will run, computes the hash, and **refuses to emit a listing that would be rejected on the other end**. Signing is optional (`--new-key` / `--sign-key`); an unsigned plugin is reported as unverified rather than blocked, which is the honest default when nobody is curating.

Reviewing listings as pull requests gives a public record of who added what, with CI able to run `publish.py` over each change — without the registry operator hosting anything beyond a static file.

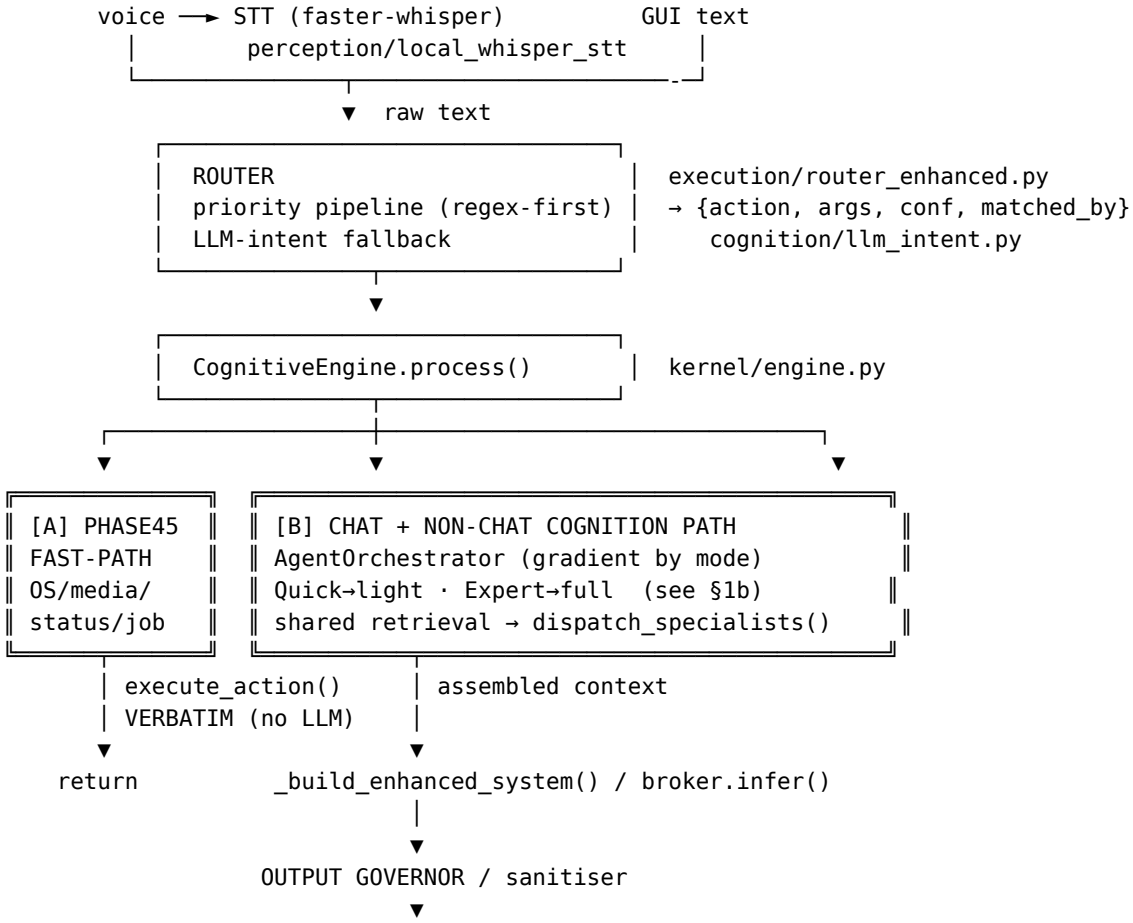
Blueprint — ELI ASCII Diagrams

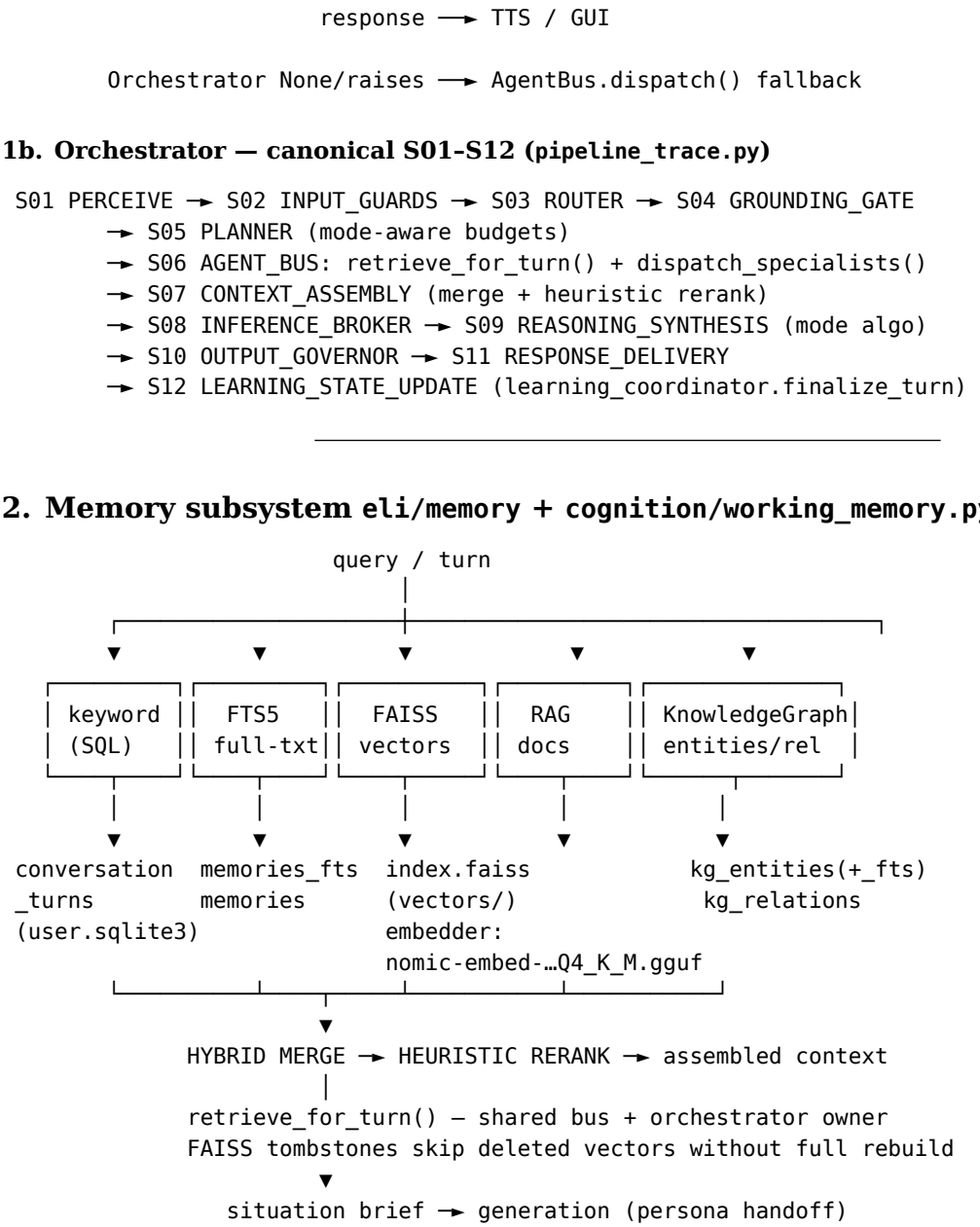
> **Updated for v2.4.84.** CHAT uses a gradient orchestrator for all modes; retrieval is unified in `eli/memory/retrieval.py`; Stage 12 = learning/state commit.

Visual companion to `architecture.md`. Three views: the full request pipeline, the memory subsystem, and the gating stack. All grounded in the real modules (file paths are clickable).

---

1. Full pipeline (request lifecycle)

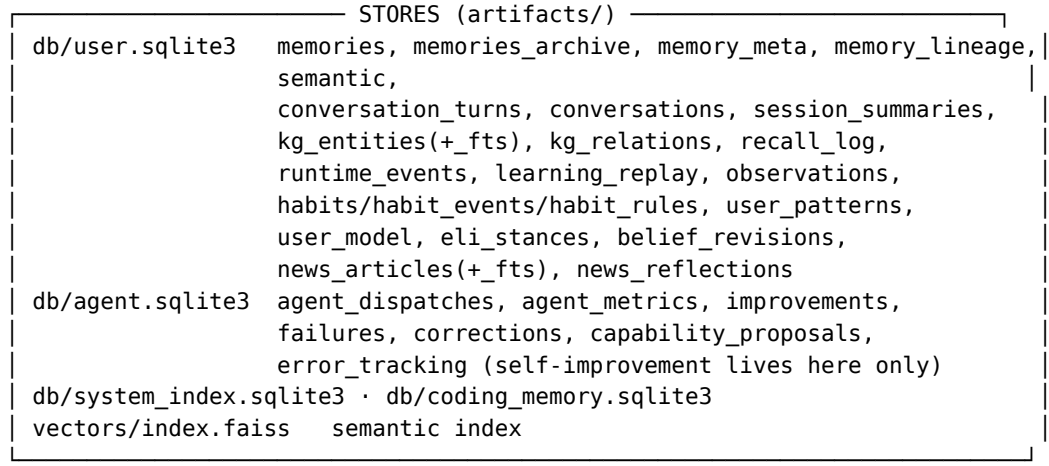




1b. Orchestrator — canonical S01–S12 (`pipeline_trace.py`)

---

2. Memory subsystem `eli/memory` + `cognition/working_memory.py`



```
A question that names a period ("last week") is filtered by date FIRST
(Memory.memories_between + dated conversation turns), then ranked by topic.

WRITE PATH:  turn ─► PERSISTENCE GATE (drop junk/report-dumps) ─► STORAGE POLICY
              (origin · dedupe · importance) ─► store;  daily upkeep decays and archives
```

---

3. Gating stack (in path order — every guard the request passes)

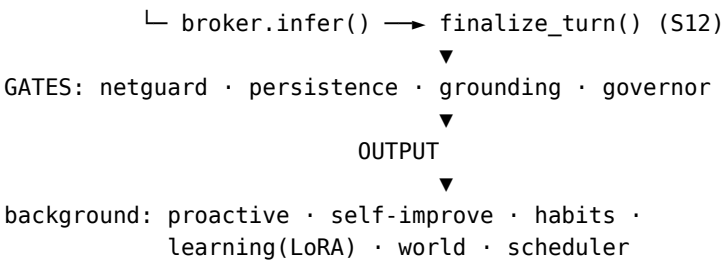
```
┌──────────────────────────────────────────────────────────────────────────────┐
│ 1 ▒ NETGUARD  (process-wide, always-on)        core/netguard.py              │
│      any outbound socket ──► offline? ──► OfflineError (FAIL-CLOSED)         │
│      allow_network(): scoped opt-in window (model download, web tier)        │
├──────────────────────────────────────────────────────────────────────────────┤
│ 2 ▒ PHASE45 FAST-PATH GATE                     kernel/engine.py              │
│      deterministic action (volume/media/date/job/status)?                    │
│        ──► execute_action() VERBATIM, no LLM  (cannot confabulate)           │
├──────────────────────────────────────────────────────────────────────────────┤
│ 3 ▒ PERSISTENCE GATE                           runtime/persistence_gate.py   │
│      about to write to memory? junk / internal report-dump?                  │
│        ──► DROP (so it can't be stored and replayed as fact later)           │
├──────────────────────────────────────────────────────────────────────────────┤
│ 4 ▒ GROUNDING GATE                             runtime/deterministic_…gate   │
│      answer requires evidence? unbacked or raw packet?                       │
│        ──► BLOCK raw evidence/telemetry leak; require grounded synthesis     │
├──────────────────────────────────────────────────────────────────────────────┤
│ 5 ▒ GROUNDING ESCALATION                       runtime/grounding_escalation  │
│      checkable fact  AND  grounding < threshold ?                            │
│        external fact ─► WEB tier ──────┐                                     │
│        self/project  ─► LOCAL tier ────┼──► first tier that grounds ─► answer│
│        exhausted, or offline and ──────┘──► HEDGE  ("can't check that now")  │
│          about what changes                                                  │
│        offline, what does not change ─► the model answers from what it knows │
│      (trigger = GROUNDING, not the response score — which lies when wrong)   │
├──────────────────────────────────────────────────────────────────────────────┤
│ 6 ▒ LEARNING / STATE COMMIT  (S12)              learning_coordinator.py      │
│      finalize_turn(): store assistant turn · publish meta · learn from result│
├──────────────────────────────────────────────────────────────────────────────┤
│ 7 ▒ OUTPUT GOVERNOR / SANITISER                cognition/output_governor.py  │
│      final text ──► strip leaks, persona hygiene, no raw JSON ──► user       │
└──────────────────────────────────────────────────────────────────────────────┘

 Legend:  FAIL-CLOSED = denies by default;  VERBATIM = returned without an LLM
          pass;  HEDGE = honest "can't verify" instead of a guess.
```

---

4. One-screen overview (how it all connects)

```
INPUT ─► ROUTER ─► ENGINE ─┬─ FAST-PATH ──────────────► (verbatim) ─► OUTPUT
                           └─ ORCHESTRATOR (all CHAT modes, gradient depth)
                                  │
                                  ├─ retrieve_for_turn() ──► memory/FAISS/KG
                                  ├─ dispatch_specialists() ──► 15-agent bus
```



“‘

ELI — Capabilities & Actions (with activation phrases)

Auto-generated 2026-11-02 from `capability_manifest.json` (231 capabilities; 214 routable or executor-backed). Regenerated by `eli/tools/registry/capabilities_doc.py` alongside the manifest.

How to read this: every row is a real action ELI can route to and execute. Activation phrases are **representative** — the router is paraphrase- and typo-tolerant; a phrase shows the *shape* that fires the action, not the only accepted wording. `Plugin` = backed by a plugin (rest are core executor+router). Voice users prefix with the wake word (“computer, …”). Generation actions (documents/scripts/projects) run the evidence-planner first (real code/web/memory analysis) before synthesising.

Conversation & persona

Action	What it does	Example activation phrase(s)	Source
`CHAT`	Open conversation / fallback; persona-bound reply	anything not matching a command · “how are you” · “tell me a story”	Core
`EXPLAIN_LAST_RESPONSE`	Explain why ELI gave its previous answer	“why did you say that” · “explain your last response”	Core
`CLEAR_CHAT_HISTORY`	Clear the current chat history	“clear chat history” · “start a new chat”	Core
`PERSONA_LOCK_SET`	Pin a persona/role for the session	“lock your persona to X”	Core
`PERSONA_LOCK_CLEAR`	Release a pinned persona	“clear persona lock”	Core
`PERSONA_LOCK_STATUS`	Show the current persona lock	“what persona are you locked to”	Core
`PERSONA_REFRESH`	Refresh/reload ELI’s persona from the latest profile	“refresh your persona” · “reload your persona”	Core
`HELP`	List what ELI can do	“help” · “what commands do you have”	Core
`LIST_CAPABILITIES`	Enumerate ELI’s capabilities	“what can you do” · “list your capabilities”	Core
`SET_TONE`	Shade ELI’s expressed tone/emotion (words+voice+face) without changing his core personality	“be comedic” · “talk street” · “sound more professional” · “use a deadpan tone”	Core

Action	What it does	Example activation phrase(s)	Source
CLEAR_TONE	Drop a pinned tone — back to autonomous, situation-adaptive delivery	“go back to normal” · “be yourself” · “drop the tone”	Core
SET_COMMUNICATION_STYLE	Persist how you want ELI to speak to you (empty or a reset phrase clears it)	“speak concisely from now on”	Core
SET_AI_MODE	Set the reasoning mode (Quick, Normal, Advanced, Research, Expert); reached through the intent resolver or the mode selector	(no dedicated phrase)	Core
MULTI_COMMAND	Run several chained commands from one utterance, in order	“close steam and set an alarm for 7am” · “open spotify then play X”	Core

App & window control

Action	What it does	Example activation phrase(s)	Source
OPEN_APP	Launch an application (installs it if missing)	“open spotify” · “launch firefox”	Core
CLOSE_APP	Close an application	“close spotify” · “quit chrome”	Core
FOCUS_APP	Bring an app’s window to the front	“focus firefox” · “switch to the browser”	Core
OPEN_BROWSER	Open the default web browser	“open the browser”	Core
OPEN_URL	Open a URL in the browser	“open github.com” · “go to youtube.com”	Core
OPEN_IDE	Open the code IDE	“open the IDE”	Core
OPEN_FILE_SYSTEM	Open the file manager	“open files” · “open the file manager”	Core
OPEN_SYSTEM_SETTINGS	Open OS settings	“open system settings”	Core
OPEN_AUDIO_SETTINGS	Open sound settings	“open audio settings”	Core
OPEN_POWER_SETTINGS	Open power settings	“open power settings”	Core
OPEN_NETWORK_BROWSER	Open network settings	“open network settings”	Core
OPEN_COMMUNICATION_HUB	Open the comms hub (mail/chat apps)	“open communication hub”	Core
OPEN_MEDIA_HUB	Open the media hub	“open media hub”	Core
TILE_WINDOWS	Tile windows in a grid	“tile windows” · “2x2 grid” · “4x2”	Core
MAXIMISE_WINDOW	Maximise a named app’s window (args: name), or the focused one when none is named	“maximise firefox” · “maximise the window”	Core

Action	What it does	Example activation phrase(s)	Source
`MINIMISE_ALL`	Minimise all / show desktop	“minimise everything” · “show desktop”	Core
`MINIMISE_WINDOW`	Minimise the focused window	“minimise the window” · “minimise this”	Core
`MINIMIZE_WINDOW`	Minimise the focused window (US spelling)	“minimize this window”	Core
`MINIMISE_APP`	Minimise a specific app’s window	“minimise spotify”	Core
`MINIMIZE_APP`	Minimise a specific app’s window (US spelling)	“minimize chrome”	Core
`HIDE_APP`	Hide an application’s window	“hide spotify” · “hide the window”	Core
`NEXT_WINDOW`	Switch to the next window	“next window”	Core
`PREVIOUS_WINDOW`	Switch to the previous window	“previous window”	Core
`RESTORE_WINDOWS`	Restore minimised windows	“restore windows”	Core
`SWITCH_WORKSPACE`	Switch virtual desktop/workspace	“switch to workspace 2”	Core
`SMART_HOME`	Control smart-home devices	“turn off the lights”	Core
`CONFIRM_PENDING_REMEDIATION`	Approve a pending fix/install offer	“yes” (to an “install X?” offer)	Core
`CANCEL_PENDING_REMEDIATION`	Decline a pending fix/install offer	“no” (to an “install X?” offer)	Core

Input control

Action	What it does	Example activation phrase(s)	Source
`VOLUME`	Adjust/mute system volume	“volume up” · “mute” · “set volume to 30”	Core
`KEYBOARD`	Send keystrokes / type text	“press enter” · “type hello world”	Core
`MOUSE_CONTROL`	Move/click the mouse	“left click” · “move the mouse up”	Core

Media

Action	What it does	Example activation phrase(s)	Source
`PLAY_MEDIA`	Play a track on the named platform	“play Juicy by Notorious B.I.G. on Spotify” · “play lo-fi on YouTube”	Core
`PAUSE_MEDIA`	Pause playback	“pause” · “pause the music”	Core

Action	What it does	Example activation phrase(s)	Source
STOP_MEDIA	Stop playback	“stop the music”	Core
NOW_PLAYING	Report the current track and where it’s playing (Spotify or headless-mpv YouTube audio)	“what’s playing” · “what song is this”	Core
NEXT_MEDIA	Next track	“next song” · “skip”	Core
PREVIOUS_MEDIA	Previous track	“previous track” · “go back a song”	Core
REPEAT_MEDIA	Repeat current track	“repeat this song”	Core
SHUFFLE_MEDIA	Toggle shuffle	“shuffle”	Core
MEDIA_CONTROL	Generic transport control (MPRIS)	“play/pause”	Core
SKIP_YOUTUBE_AD	Skip a YouTube ad when skippable	“skip the ad”	Core

Vision & screen

Action	What it does	Example activation phrase(s)	Source
SCREEN_READ_ANALYZE	Screenshot → vision model + OCR; answers questions about on-screen content	“what’s on my screen” · “can you see this” · “what season is on the screen”	Core
SCREEN_LOCATE	Find on-screen UI text/element	“find the submit button on screen”	Core
SCREENSHOT	Capture a screenshot	“take a screenshot”	Core
OCR_IMAGE	Extract text from an image file	“ocr photo.png” · “read the text in image.jpg”	Core
ANALYZE_IMAGE	Describe/analyse an image file (VLM)	“describe cat.jpg” · “analyse this image”	Core
ANALYZE_PDF	Read/summarise a PDF	“summarise report.pdf” · “analyse this pdf”	Core
ANALYZE_PDF_FOLDER	Analyse all PDFs in a folder	“analyse the pdfs in that directory”	Core
ANALYZE_CSV	Analyse a CSV/spreadsheet	“analyse data.csv”	Core
AMBIENT_VISION	Toggle continuous screen-watching	“watch my screen” · “stop watching”	Core
IMAGE_STATUS	Status of the latest image job	“what is the latest image status”	Core

Gaze (eye-tracking)

Action	What it does	Example activation phrase(s)	Source
GAZE_ENABLE	Enable webcam gaze control	“enable gaze control”	Core
GAZE_DISABLE	Disable gaze control	“disable gaze control”	Core

Action	What it does	Example activation phrase(s)	Source
GAZE_CALIBRATE	Calibrate gaze tracking	“calibrate gaze”	Core
GAZE_STATUS	Show gaze tracker status	“gaze status”	Core
GAZE_CLICK	Click where the eyes rest	“open” / “left click” / “hit enter” (while gaze is on)	Core

Voice

Action	What it does	Example activation phrase(s)	Source
DICTATE	Start/stop dictation into the active field	“start dictation” · “stop dictating”	Core
TRANSCRIBE	Transcribe an audio file	“transcribe audio.wav”	Core
SPEAK	Speak text aloud (TTS)	“say good morning”	Plugin
LIST_VOICES	List installed voices and what’s downloadable (166 across 45 languages)	“what voices do you have” · “list voices” · “which accents can I get”	Core
DOWNLOAD_VOICE	Download a voice/accent from the Piper library and switch to it	“download a British voice” · “get me a Scottish accent” · “install en_US-amy-medium”	Core
SET_VOICE	Switch the active TTS voice/accent (distinct from persona tone)	“use the alan voice” · “switch your voice to amy” · “use the robotic style voice” · “use the natural voice”	Core
CREATE_VOICE	Make a new ELI voice from a recording (wav/mp3/mp4 drop-in; XTTS-v2 clone)	“create a voice called Nova from clip.mp4” · “make an ELI voice from recording.wav”	Core
LISTEN_FOR_COMMAND	Start listening for a voice command	“listen” / the wake word “computer”	Core
DESIGN_VOICE	Build a custom style voice from a base voice with pitch, speed and effects	“design a voice called Nova, deep and slow” · “make a custom voice named Nova”	Core
WAKE_SET	Set your own wake word (any phrase); ELI persists it and trains the detector on it	“change the wake word to athena” · “set my wake word to atlas”	Core
WAKE_TRAIN	Train the local, self-supervised wake-word model (Piper-synth + noise/music augmentation; robust over background music)	“train the wake word”	Core

Action	What it does	Example activation phrase(s)	Source
WAKE_ENROLL	Record your voice saying the wake word and retrain (personalisation, over music)	“enroll my wake word”	Core
TRAIN_VOICE	Learn your voice + tone — pitch/energy/rate, happy/angry/excited, and question-vs-statement; ELI then adapts its delivery to how you sound	“train my voice” · “learn how I speak”	Core
STT_DIAGNOSTICS	Mic/STT/TTS diagnostics (Whisper, wake word, ducking, Piper)	“run voice diagnostics” · “voice diagnostics” · “check the microphone” · “stt diagnostics”	Core

Files & notes

Action	What it does	Example activation phrase(s)	Source
CREATE_FILE	Create a file	“create a file notes.txt”	Core
CREATE_FOLDER	Create a folder	“make a folder called projects”	Core
READ_FILE	Read a file’s contents	“read notes.txt”	Core
LIST_DIR	List a directory	“list the files in ~/Documents”	Core
FILE_AUDIT	Audit files for issues	“audit my files”	Core
SUMMARIZE_FILE	Summarise a document/file	“summarise report.docx”	Core
CONVERT_DOCUMENT	Convert between document formats	“convert report.md to pdf”	Core
WRITE_NOTE	Write a quick note	“write a note saying buy milk”	Core
NEW_NOTE	Create a new note	“new note”	Plugin
LIST_NOTES	List saved notes	“list my notes”	Plugin
SEARCH_NOTES	Search notes	“search notes for budget”	Plugin
GET_CLIPBOARD	Read the clipboard	“what’s in my clipboard”	Core
SET_CLIPBOARD	Write to the clipboard	“copy this to the clipboard”	Core

Generation (docs/code)

Action	What it does	Example activation phrase(s)	Source
GENERATE_DOCUMENT	Grounded document via the multi-stage pipeline (evidence → plan/outline → sections → review→revise)	“generate a document about X” · “write a report on Y”	Core
CREATE_DOCUMENT	Multi-stage grounded document (plan→draft→review); single-pass fallback	“create a document on X”	Core
DATA_FABRICATOR	Generate synthetic/test data	“fabricate a test dataset for X”	Core
GENERATE_SCRIPT	Generate a runnable script via the coding agent	“write a bash script to monitor the GPU”	Core
GENERATE_PROJECT	Scaffold a multi-file project	“generate a project that does X”	Core
CODE_SOLVE	Solve a coding task (plan→DAG→verify→repair)	“solve this: …” · “implement function X”	Core
FIX_FILE	Find and fix bugs in a file	“fix the bugs in foo.py”	Core
EXAMINE_CODE	Tiered error scan of named files/codebase	“examine eli/memory/memory.py for errors” · “review the codebase for bugs”	Core
CONFIRM_CODE_FIX	Apply the offered code fixes (pending-confirm)	“yes, fix it” · “fix the logic ones too”	Core
CANCEL_CODE_FIX	Decline the offered code fixes	“no, leave it”	Core
SHOW_DIFF	Show a code diff	“show the diff”	Core
OPEN_IN_IDE	Open a generated artifact in the IDE	(post-action; auto after generate)	Core
DOC_GENERATE	Alias of GENERATE_DOCUMENT	“generate a document about tides”	Core

System status & info

Action	What it does	Example activation phrase(s)	Source
CPU_USAGE	Current CPU usage	“cpu usage”	Plugin
RAM_USAGE	Current memory usage	“how much ram is free”	Plugin
GPU_STATUS	GPU/VRAM status	“gpu status”	Core
SYSTEM_STATS	Combined system stats	“system stats”	Plugin
HARDWARE_PROFILE	Detected hardware profile	“show hardware profile”	Core
GET_STATUS	General status	“status”	Core
TIME	Current time	“what time is it”	Core
GET_TIME	Current time (alt)	“tell me the time”	Core
DATE	Current date	“what’s the date”	Core
GET_DATE	Current date (alt)	“what day is it”	Core
GET_WEATHER	Weather for a place	“weather in Paris”	Plugin

Action	What it does	Example activation phrase(s)	Source
NEWS_FETCH	Synthesised news digest; topic-scoped re-fetch after a briefing when you go deeper	“what’s the news” · “catch me up” · “go deeper into Hubble” · “more on the transformer story”	Core
WEB_SEARCH	Web search (Net-gated)	“search the web for X”	Plugin
WEB_FETCH	Read a linked web page and answer about it (Net-gated)	“https://github.com/ShadowESC95/ELI_v2.0 what do you think?” · “summarise this https://example.com/article”	CC95/ELI_v2.0
MESSAGE_TIME_QUERY	Time-of-a-message query	“when did I say that”	Core
CHECK_CHRONAL_ALIGNMENT	Playful date/time sanity check	“check chronal alignment”	Core

Grounded introspection

Action	What it does	Example activation phrase(s)	Source
RUNTIME_STATUS	Live runtime: model, ctx, GPU layers, batch	“what are you running on” · “runtime status — everything”	Core
RUNTIME_AUDIT	Full runtime audit (what’s broken/missing)	“run a full runtime audit”	Core
IMPORT_AUDIT	Which imports are failing/missing	“what imports are failing”	Core
RESOLVE_RUNTIME_PATHS	Resolved paths of critical files	“show the resolved runtime paths”	Core
GUI_RUNTIME_AUDIT	Audit GUI wiring with file-read proof	“audit your gui file” · “scan the gui runtime wiring and prove every hook”	Core
DIAGNOSE_WRAPPERS	Show the executor middleware/wrapper chain	“show the executor wrapper chain”	Core
COGNITION_STATUS	Cognition runtime status	“cognition status”	Core
EXPLAIN_COGNITION_RUNTIME	Explain the cognition runtime verbatim	“explain your cognition runtime”	Core
EXPLAIN_GGUF_DIAGNOSTICS	Live GGUF load/inference compatibility report (runtime version, preflight, last load error)	“gguf errors” · “why won’t nemotron load” · “empty response from the model”	Core
EXPLAIN_FAILURE_LOG	Grounded executor failure log from agent.sqlite3 (not GGUF console lines)	“what are the failures” · “show the failure log” · “what’s tripping you up”	Core

Action	What it does	Example activation phrase(s)	Source
EXPLAIN_LAST_FAILURE	Explain the most recent grounded or executor failure	“what was the last failure” · “why did that fail”	Core
MEMORY_STATUS	Memory subsystem status / counts	“how does your memory work”	Core
MEMORY_STATS	Memory statistics	“memory stats”	Core
EXPLAIN_MEMORY_RUNTIME	Explain the memory runtime verbatim (4 DBs)	“explain exactly how your memory works internally”	Core
AWARENESS_STATUS	What ELI is currently aware of	“what are you aware of”	Core
FRONTIER_STATUS	Full system wiring matrix	“run a full system wiring matrix”	Core
ELI_IDENTITY_AUDIT	Audit ELI’s identity provenance	“audit your identity”	Core
CODE_CHANGES	Recent code changes	“what code has changed”	Core
NAME_SOURCE_AUDIT	Where ELI’s knowledge of your name comes from	“how do you know my name”	Core
REASONING_MODE_STATUS	Current reasoning mode	“what reasoning mode are you in”	Core
EXPLAIN_ALL_REASONING_MODES	Describe all 5 reasoning modes	“explain all your reasoning modes”	Core
SELF_REPORT	What ELI has recently done/worked on (code, commits, runtime)	“what updates have you made recently”	Core

Memory & profile

Action	What it does	Example activation phrase(s)	Source
MEMORY_STORE	Store a durable fact	“remember that my sister’s name is Anna”	Core
MEMORY_RECALL	Recall stored facts	“what do you remember about my car”	Core
MEMORY_FORGET	Delete matching memories and everything derived from them, after confirmation	“forget that my locker is 212”	Core
PERSONAL_MEMORY_SUMMARY	Summary of what ELI knows about you	“what do you know about me”	Core
PERSONAL_MEMORY_DEEP_EXPLAIN	Deep, sourced profile explanation	“explain everything you know about me and where it’s stored”	Core
USER_IDENTITY_SUMMARY	Who the user is	“who am I”	Core
REFRESH_USER_INFO	Re-extract the user profile	“refresh my user info”	Core
SET_USER_NAME	Set/correct the user’s name	“my name is Alex” · “call me Alex”	Core

Action	What it does	Example activation phrase(s)	Source
USER_INFO_REPORT	Full report of the living user profile; reached through the intent resolver	(no dedicated phrase)	Core

Self-maintenance

Action	What it does	Example activation phrase(s)	Source
SELF_ANALYZE	Analyse own failures/metrics	“analyse your failures”	Core
SELF_IMPROVE	Run a self-improvement patch cycle	“improve yourself”	Core
SELF_PATCH	Apply a self-improvement patch	“patch yourself”	Core
SELF_REPAIR_PLAYBOOK	Grounded repair decision guide (self help)	“self help” · “repair playbook” · “maintenance playbook”	Core
AUTOPILOT_DEBUG	Turn a failure into a plan: root cause, affected files from git history, rollback, patch and validation	“debug this error: ValueError x in eli/core/paths.py” · “autopilot debug this traceback”	Core
CODEBASE_GRAPH	Import graph of ELI’s own source: what depends on what	“show the codebase graph”	Core
TIMESTAMP_DIAG	Run real clock and timestamp diagnostics instead of guessing	“check the timestamps” · “investigate the timestamp problem”	Core
SELF_IMPROVEMENT_LOG	Show the self-improvement log	“show your self-improvement log”	Core
SELF_UPGRADE	git pull → deps → rebuild indexes	“upgrade yourself”	Core
SELF_UPDATE	Self-update (control-contract)	“update yourself”	Core
SELF_TEST	Run internal self-tests	“run a self test”	Core
RUN_TESTS	Run the pytest suite and summarise the results document	“run the test suite” · “generate a test report”	Core
GENERATE_TESTS	ELI writes + sandbox-verifies behavioural tests for its own functions (Phase 4)	“generate tests for your code” · “grow your test coverage”	Core
LORA_STATUS	Report LoRA fine-tune readiness (preflight: modules, base model, reviewed data)	“lora status” · “is lora ready”	Core

Action	What it does	Example activation phrase(s)	Source
LORA_TRAIN	Run the LoRA training pipeline DAG (pre-flight→build→train→eval); dry-run from chat, real training via the overnight task	“train a lora” · “fine-tune yourself”	Core
ORCHESTRATION_STATUS	Explain the agent DAG orchestrator: execution layers, dependencies, critical path, and last run	“orchestration status” · “show your agent dag”	Core
TEST_REVIEW	Run the suite, back up the report + write an errors file, summarise results, and offer result-driven fix options	“test review” · “run the tests and tell me what to fix”	Core

Plugins

Action	What it does	Example activation phrase(s)	Source
PLUGIN_STATUS	Status of one plugin; reached through the intent resolver (“list plugins” lists them all)	(no dedicated phrase)	Core
MCP_LIST	List configured MCP servers	“list mcp servers”	Core
MCP_STATUS	Status of MCP servers and their tools	“mcp status”	Core
MCP_TOOLS	List the tools MCP servers expose	“list mcp tools”	Core
MCP_CALL	Call a tool on an MCP server (the server runs outside ELI’s network guard)	“call the mcp tool read_file”	Core
MCP_ADD	Add an MCP server (installed switched off until you enable it)	“add an mcp server”	Core
MCP_REMOVE	Remove an MCP server	“remove the mcp server github”	Core
MCP_DOCTOR	Check MCP servers: launch preflight and handshake	“mcp doctor” · “check my mcp servers”	Core
PLUGIN_LIST	List installed plugins	“list plugins”	Core
PLUGIN_SEARCH	Search for plugins	“search plugins for weather”	Core
PLUGIN_INSTALL	Install a plugin	“install the X plugin”	Core
PLUGIN_UNINSTALL	Uninstall a plugin	“uninstall the X plugin”	Core
PLUGIN_ENABLE	Enable a plugin	“enable the X plugin”	Core
PLUGIN_DISABLE	Disable a plugin	“disable the X plugin”	Core

Tasks, time & planning

Action	What it does	Example activation phrase(s)	Source
SCHEDULE_TASK	Schedule overnight/timed work (code/research/etc.)	“research X overnight” · “build Y at 2am”	Core
SET_ALARM	Set an alarm	“set an alarm for 7am”	Core
SET_TIMER	Set a timer	“set a timer for 10 minutes”	Core
ADD_EVENT	Add a calendar event	“add an event tomorrow at 3pm”	Core
LIST_EVENTS	List events	“list my events”	Core
REMOVE_EVENT	Remove a calendar event or reminder	“cancel the dentist appointment”	Core
POMODORO_START	Start a pomodoro	“start a pomodoro”	Plugin
POMODORO_STOP	Stop the pomodoro	“stop the pomodoro”	Plugin
POMODORO_STATUS	Pomodoro status	“pomodoro status”	Plugin
BACKGROUND_JOBS	List background/scheduled jobs	“show background jobs”	Core
CHECK_JOB	Check a specific job	“check job 5”	Core
HABIT_STATUS	Show learned habits	“show my habits”	Core
CONFIRM_HABIT	Approve a proposed habit	“yes” (to a habit offer)	Core
DECLINE_HABIT	Decline a proposed habit	“no” (to a habit offer)	Core
EXECUTE_GOAL	Execute a stored goal	“execute goal X”	Core
RESUME_TASK	Recover a saved task record and state its real staleness	“continue the ORBIT project” · “resume the migration task”	Core

Proactive

Action	What it does	Example activation phrase(s)	Source
PROACTIVE_START	Start the proactive daemon	“start proactive mode”	Core
PROACTIVE_STOP	Stop the proactive daemon	“stop proactive mode”	Core
PROACTIVE_STATUS	Proactive daemon status	“proactive status”	Core
GET_PROPOSALS	ELI’s self-generated proposals/goals from observed signals	“do you have any proposals for me” · “what’s on your agenda”	Core
MORNING_REPORT	Daily briefing (news/weather/agenda)	“morning report” · “give me my briefing”	Core

Shell & advanced

Action	What it does	Example activation phrase(s)	Source
RUN_CMD	Run a shell command (gated/confirmed)	“run the command: ls -la”	Core
SHELL_EXEC	Execute shell (gated/confirmed)	“execute: df -h”	Core
SEQUENCE	Run a multi-step action sequence	“do X then Y then Z”	Core
ROUTING_FAULT_EXPLAIN	Explain why an input failed to route	“why did that fail to route”	Core
NOOP	Internal no-op (ignored fragment guard)	(internal — fragmentary input is dropped)	Core

Aliases & internal actions (not directly user-triggered)

Synonyms normalised to a routable action, post-actions, or internal pipeline steps:

ANSWER, CHECK_TARGET_STATUS, CLOSE_APPLICATION, CREATE_DOC, CREATE_SCRIPT, DIRECT_RESPONSE, EXIT_APP, GENERATE_CODE, LAUNCH_APP, OPEN_APPLICATION, PREPARE_REMEDIATION, QUIT_APP, SAY, VOICE_DIAGNOSTICS, WRITE_CODE, WRITE_DOCUMENT, WRITE_SCRIPT

Coverage

- Routable actions documented: **214/214**